

Глава 17. Интеграция с внешними системами

17.1. Общая информация

Часто встречаются ситуации, когда корпоративные информационные комплексы состоят из различных информационных систем. Эти системы могут быть написаны как одним, так и различными производителями. В такой ситуации очевидным и естественным желанием является обеспечение эффективного взаимодействия всех этих систем. Можно выделить два подхода к интеграции: интеграция данных и функциональная интеграция. В первом случае обеспечивается обмен информацией между системами, во втором - обеспечивается возможность одной системы вызвать функцию другой системы с передачей данных и получением обратных результатов обработки. При решении задачи интеграции нескольких информационных систем необходимо обеспечивать:

- Достаточную оперативность - данные должны попадать потребителю как можно раньше и никак не позднее того, когда они должны потребителем использоваться.
- Достаточную надежность - данные должны дойти до потребителя даже в случае временного отказа инфраструктуры или временной недоступности самого потребителя.
- Минимально возможную связанность систем - системы должны продолжать работать независимо друг от друга, обновление одной системы не должно требовать обновления всех остальных и т. д.

Для организации взаимодействия различных информационных систем, как правило, можно использовать следующие способы:

- Обмен файлами.
- Использование общей базы данных.
- Вызов (в том числе удаленный) методов программного интерфейса одной системы из другой системы. Вызовы могут выполняться различными способами, в том числе и посредством сети Интернет.
- Обмен сообщениями систем класса сервисная шина предприятия (СШП или ESB - **E**nterprise **S**ervice **B**us).
- Другие способы интеграции, которые не подпадают под вышеприведенную классификацию, но, тем не менее, позволяют обмениваться данными с «внешним миром». Например, это может быть работа через буфер обмена используемой ОС.

Кратко опишем способы интеграции и особенности этих способов.

Файловый обмен.

Системы-участники по расписанию генерируют файлы, содержащие данные для передачи другим участникам, и по расписанию же ищут подготовленные для них файлы. Оперативность, с которой данные оказываются в системе-потребителе, зависит от расписания выгрузки и загрузки. Можно увеличивать частоту выгрузки данных и добиваться большей оперативности передачи данных, но интенсивная работа с большим количеством файлов может привести к потере производительности. Также производительность теряется, если требуется организовать файловый обмен с использованием низкоскоростных каналов передачи данных. Таким образом, файловый обмен позволяет выполнить интеграцию данных, но не позволит выполнить функциональную интеграцию. Файловый обмен не требует сильного связывания, но не обеспечивает высокой скорости обмена.

Для реализации такого обмена в системе «1С:Предприятие» существуют объекты для работы с файлами и различными файловыми форматами. В качестве примеров можно рассмотреть работу с форматами XML/XDTO (см. [здесь](#)), JSON (см. [здесь](#)), двоичные данные (см. [здесь](#)), DBF (объект `XBase`), табличные форматы MXL/XLS/XLSX/ODS (объект `ТабличныйДокумент`) и текст (объекты `ТекстовыйДокумент`, `ЧтениеТекста`, `ЗаписьТекста`).

Использование общей базы данных

Прямое обращение к базе данных другой системы плохо тем, что схема данных системы часто не является документированным контрактом, который сторонняя система обязуется выполнять. Соответственно, при смене версии часто теряется работоспособность интеграции из-за изменений в схеме или изменений способа работы системы со своими данными. В некоторых случаях могут возникать сложности с лицензиями СУБД, которые могут явно запрещать ее использование сторонними системами. Для проприетарных СУБД важно также увеличение числа используемых подключений, которое может быть ограничено. Таким образом, обмен через общую базу данных позволяет выполнить интеграцию данных, но не позволит выполнить функциональную интеграцию. Обмен через общую базу данных требует сильного связывания, но позволяет обеспечить высокую скорость обмена.

Для реализации такого обмена можно воспользоваться механизмом внешних источников данных (см. [здесь](#)).

Вызовы методов программного интерфейса

Вызов методов программного интерфейса одной системы из другой системы обеспечивает функциональную интеграцию, но приводит к еще более сильной связанности взаимодействующих систем. Функциональная интеграция может реализовываться разными способами, например через сеть Интернет (с помощью Интернет-сервисов) или с помощью специальных механизмов взаимодействия компонентов (COM-модель, доступна только при работе под управлением ОС Windows).

Интернет-сервис - это программная система, которая обладает стандартизированным интерфейсом и идентифицируется уникальным адресом (URL) в сети Интернет (или локальной сети). Стандартизированный интерфейс предполагает некоторый набор заранее описанных действий, которые можно применять к этой программной системе. Выполнение этих действий в определенной последовательности и позволяет реализовать тот или иной бизнес-процесс. Доступность в сети Интернет предполагает, что программная система может использоваться практически любыми сторонними системами, которые умеют получать и передавать данные через сеть и формировать данные в формате (например, XML или JSON), который принят для общения с этой программной системой.

Интернет-сервисы могут быть реализованы различными способами, например с помощью протокола SOAP (**S**imple **O**bject

Access Protocol, простой протокол доступа к объектам) и формата WSDL (**Web Services Description Language**, язык описания Web-сервисов) или используя подход, который называется REST (**R**epresentational **S**tate **T**ransfer, передача состояния представления). Для Интернет-сервисов, построенных с учётом REST (то есть не нарушающих накладываемых этим подходом ограничений), применяют термин «RESTful».

Протокол SOAP определяет то, каким образом будет оформляться взаимодействие между участниками взаимодействия (клиент и сервер сервиса). Но этот протокол не описывает с какими данными работает сервис и какие команды сервис «понимает». Для описания этих подробностей служит формат WSDL. По сути, перечень действий и форматов данных ничем не ограничен и определяется во время разработки сервиса. REST-сервис - это сервис, при разработке которого применялся подход, который подразумевает ограниченный набор действий (ограничен списком HTTP-методов) и разветвленный набор ресурсов. Состав ресурсов определяется в процессе разработки сервиса. Набор ресурсов определяет функциональные возможности сервиса. При этом структуры данных, которые используются REST-сервисом - жестко не определяются.

Если внешняя программная система предоставляет программный интерфейс, который ориентирован на использование протокола SOAP, то для использования этого протокола система «1С:Предприятие» предоставляет механизм Web-сервисов (см. [здесь](#)) протокол обеспечения вызовов и механизм XDTO (см. [здесь](#)) как формат описания методов сервиса и используемых данных. Прикладное решение может быть и клиентом, и сервером в таком обмене.

Если в сторонней программной системе для построения программного интерфейса применен подход REST, то для того, чтобы выступать в роли клиента REST-сервиса (т. е. получать данные из сторонней системы), следует использовать объект **HTTPСоединение** (и связанные объекты). Если необходимо, чтобы прикладное решение на платформе «1С:Предприятие» выступало в качестве REST-сервера (т. е. предоставляло данные, используя подход REST), то следует использовать механизм HTTP-сервисов (см. [здесь](#)). Если нужно получить унифицированный доступ к данным информационной базы «1С:Предприятие» без реализации какого-либо кода на встроенном языке, то следует использовать стандартный интерфейс OData (см. [здесь](#)).

Обмен данными через сообщения

Сервисная шина предприятия - это программное обеспечение, которое обеспечивает централизованный и унифицированный событийно-ориентированный обмен сообщениями между различными информационными системами на принципах сервисно-ориентированной архитектуры. Обмен сообщениями сохраняет слабую связанность взаимодействующих систем аналогично файловому обмену. Обмен сообщениями дает возможность частого и быстрого взаимодействия небольшими сообщениями. При помощи обмена сообщениями можно интегрировать функциональность систем, используя паттерн «запрос-ответ». Передаваемые сообщения, при необходимости, могут быть трансформированы без участия и уведомления отправителя и получателей. При обмене сообщениями предоставляется возможность доставить одно сообщение множеству потребителей в оригинальном или измененном виде без изменения логики работы отправителя. Работа через сообщения меняет логику взаимодействия систем, обмен становится асинхронным. В этой схеме отправитель не должен ожидать ответа на свое сообщение, а должен быть готовым в любой момент времени принять какое-либо сообщение и адекватно на него отреагировать.

«1С:Предприятие» подключается к некоторой системе класса СШП с помощью протокола AMQP (<https://www.amqp.org/>). Адаптером для этого продукта в объектной модели «1С:Предприятие» является объект **Сервис интеграции** (см. [здесь](#)).

Прикладное решение, которое написано на платформе «1С:Предприятие», может одновременно использовать все перечислены выше механизмы интеграции. В результате прикладное решение может с одной стороны получать различную информацию, которую предоставляют сторонние системы (быть клиентом в каком-либо обмене), а с другой стороны - «делиться» необходимой информацией со сторонними системами (быть сервером в обмене).

В данной главе будут рассмотрены вопросы, касающиеся различных механизмов интеграции и их реализации в рамках платформы «1С:Предприятие».

17.2. Web-сервисы

17.2.1. Общая информация

Конфигурация системы «1С:Предприятие» может экспортировать свою функциональность через Web-сервисы. Определения Web-сервисов задаются в дереве конфигурации и становятся доступны произвольным информационным системам благодаря публикации их на веб-сервере.

Кроме этого, система «1С:Предприятие» может обращаться к Web-сервисам сторонних производителей как через статические ссылки, определенные в дереве конфигурации, так и с помощью динамических ссылок, создаваемых средствами встроенного языка.

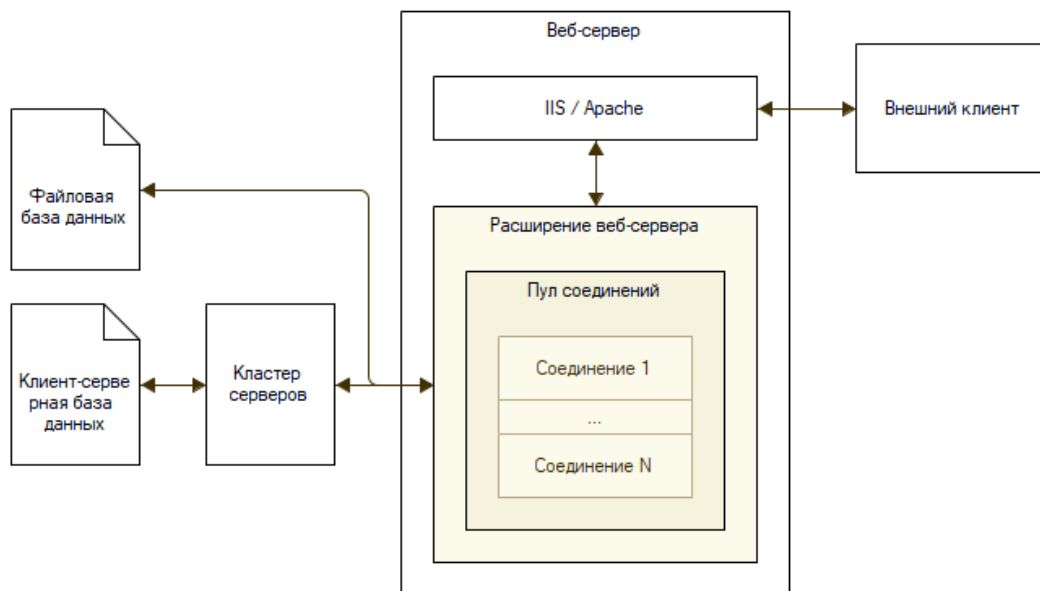


Рис. 507. Web-сервисы

Взаимодействие клиентов веб-сервисов с системой «1С:Предприятие» выполняется при помощи расширения веб-сервера. Расширение веб-сервера выполняется в процессе сервисного хоста, который выполняет функцию приема/передачи сообщений из/в менеджер сервисов. В качестве сервисного хоста может использоваться веб-сервер IIS или Apache.

Расширение веб-сервера содержит в себе пул соединений, а также другие механизмы, через которые идет взаимодействие с базой данных системы «1С:Предприятие».

Механизм Web-сервисов, реализованный в системе «1С:Предприятие», поддерживает следующие стандарты:

- SOAP 1.1,
- SOAP 1.2,
- WSDL 1.1,
- WS-I Basic Profile 1.1,
- HTTP 1.1,
- TLS 1.x (TLS 1.1 и 1.2 поддерживаются, если не требуется передача клиентского сертификата на сервер), включая криптографические алгоритмы, соответствующие ГОСТ Р 34.10-94, Р 34.10-2001, Р 34.10-2012, Р 34.11-94, Р 34.11-2012 и 28147-89;
- MTOM;
- Аутентификация: Basic, NTLM/Negotiate.

Механизм Web-сервисов, реализованный в системе «1С:Предприятие», не поддерживает:

- Механизм WS-Policy;
- Механизм WS-Addressing;
- Протокол WS-Security;
- Задание уже используемого XML пространства имен в качестве целевого пространства имен (targetNamespace).

Для получения доступа к Web-сервису необходимо использовать адрес, который формируется следующим образом: <http://host/base/ws/ИмяWebСервиса> или <http://host/base/ws/АдресWebСервиса>. Более подробно рассмотрим составные части адреса:

- <http://host/base> - обычный URL, по которому выполняется доступ, например, к информационной базе с помощью веб-клиента. При наличии разделителей, не поддерживается указание значений разделителей с помощью параметра [Z](#) командной строки запуска клиентского приложения.
- [ws](#) - признак того, что выполняется обращение к Web-сервису (в отличие от [hs](#), который определяет доступ к HTTP-сервису, см. [здесь](#)).
- [ИмяWebСервиса](#) - имя Web-сервиса. Задается в свойстве объекта [Web-сервис](#).
- [АдресWebСервиса](#) - описывает альтернативное имя для доступа к Web-сервису. Задается в свойстве [Имя файла публикации](#) объекта [Web-сервис](#). Может быть изменено при публикации Web-сервиса.
- Обращения по имени и адресу Web-сервиса являются равносильными.

17.2.2. Предоставление функциональности через Web-сервисы

Для того чтобы функциональность системы «1С:Предприятие» стала доступна внешним потребителям Web-сервисов, нужно

выполнить следующие действия:

- создать в конфигурации необходимое количество Web-сервисов,
- опубликовать Web-сервисы с помощью специального инструмента конфигулятора.

Описание процедуры публикации Web-сервисов [приведено в книге](#). Описание использования Reverse Proxy для доступа к «1С:Предприятию» [приведено в книге](#).

Создание Web-сервиса заключается:

- в добавлении в дерево метаданных объекта конфигурации Web-сервис,
- описании операций, которые может выполнять создаваемый Web-сервис,
- описании параметров операций Web-сервиса.

Объект конфигурации [Web-сервис](#) содержит модуль, в котором создаются процедуры на встроенном языке, выполняемые при вызове тех или иных операций Web-сервиса. Типы параметров операций Web-сервиса описываются с помощью типов XDTO и могут представлять собой либо значения XDTO, либо объекты XDTO.

Вызов Web-сервиса происходит следующим образом:

- из пула соединений выбирается подходящее соединение с информационной базой; при отсутствии необходимого соединения соединение создается;
- создается новый сеанс и для созданного сеанса вызывается событие [УстановкаПараметровСеанса](#) (в модуле сеанса);
- выполняется вызов затребованного метода Web-сервиса, при этом происходит вызов обработчика [УстановкаПараметровСеанса\(\)](#) (в модуле сеанса) каждый раз, когда происходит обращение к неинициализированному параметру сеанса.

СОВЕТ. Не рекомендуется выполнять ресурсоемкие операции в обработчике события [УстановкаПараметровСеанса](#).

Событие [УстановкаПараметровСеанса](#) модуля сеанса вызывается на сервере в привилегированном режиме. Модуль вызванного сервиса исполняется на сервере в обычном режиме.

Модуль сеанса (см. [здесь](#)) служит для инициализации параметров сеанса и выполнения некоторого набора команд при вызове любого Web-сервиса системы «1С:Предприятие».

17.2.3. Пример реализации Web-сервиса

Например, требуется создать Web-сервис системы «1С:Предприятие», который должен по переданному номеру расходной накладной возвращать состав ее табличной части. Аналогичный пример с помощью HTTP-сервисов см. [здесь](#).

Для описания возвращаемого значения создадим пакет XDTO [ДанныеРасходнойНакладной](#) с пространством имен <http://www.MyCompany.ru/shipment>, содержащий три типа объектов XDTO:

- **Номенклатура** - для передачи данных элемента справочника [Номенклатура](#). Этот тип объекта XDTO будет содержать следующие свойства:
 - **Наименование** - тип `string` из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - **ПолноеНаименование** - тип `string` из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - **ШтрихКод** - тип `string` из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - **ЗакупочнаяЦена** - тип `int` из пространства имен <http://www.w3.org/2001/XMLSchema>.
- **СтрокаРасходнойНакладной** - для передачи данных одной строки расходной накладной. Этот тип объекта XDTO будет содержать следующие свойства:
 - **Номенклатура** - тип [Номенклатура](#) из пространства имен <http://www.MyCompany.ru/shipment>; представляет собой ссылку на объект XDTO, который мы определили выше;
 - **Количество** - тип `int` из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - **Цена** - тип `int` из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - **Сумма** - тип `int` из пространства имен <http://www.w3.org/2001/XMLSchema>.
- **РасходнаяНакладная** - для передачи данных всех строк расходной накладной. Этот тип объекта XDTO будет содержать единственное свойство:
 - **Состав** - тип [СтрокаРасходнойНакладной](#) из пространства имен <http://www.MyCompany.ru/shipment>. Представляет собой ссылку на объект XDTO, который мы определили выше. Для того чтобы это свойство могло содержать неограниченное множество значений, необходимо установить его свойство [Верхняя граница](#) в значение -1.

После того как необходимые типы XDTO созданы, следует добавить в конфигурацию новый Web-сервис [ДанныеРасходнойНакладной](#) со следующими значениями свойств:

- **URI Пространства имен** - <http://www.MyCompany.ru/shipment>;
- **Пакеты XDTO** - [ДанныеРасходнойНакладной](#);

- **Имя файла публикации** - `shipment.lcws`.

У созданного Web-сервиса следует определить операцию **Получить** со следующими значениями свойств:

- **Тип возвращаемого значения** - `РасходнаяНакладная` из пространства имен `http://www.MyCompany.ru/shipment`;
- **Возможно пустое значение** - `установлен`;
- **Имя процедуры** - `Получить`.

У операции **Получить** следует определить параметр **НомерДокумента** со следующими значениями свойств:

- **Тип значения** - тип `string` из пространства имен `http://www.w3.org/2001/XMLSchema`;
- **Направление передачи** - `Входной`.

После этого следует открыть модуль созданного Web-сервиса и разместить в этом модуле функцию `Получить()`, которая будет выполняться при вызове данного Web-сервиса.

[Копировать в буфер обмена](#)

Функция `Получить(НомерДокумента)` Экспорт

```
// Получить объект расходной накладной по переданному номеру
ДокументСсылка = Документы.РасходнаяНакладная.НайтиПоНомеру(НомерДокумента, ТекущаяДата());

Если ДокументСсылка.Пустая() Тогда

Возврат Неопределено;

КонецЕсли;

Документ = ДокументСсылка.ПолучитьОбъект();

// Получить типы объектов XDTO
НоменклатураТип = ФабрикаXDTO.Тип("http://www.MyCompany.ru/shipment", "Номенклатура");
РасходнаяНакладнаяТип = ФабрикаXDTO.Тип("http://www.MyCompany.ru/shipment", "РасходнаяНакладная");
СтрокаРасходнойНакладнойТип = ФабрикаXDTO.Тип("http://www.MyCompany.ru/shipment",
"СтрокаРасходнойНакладной");

// Создать объект XDTO расходной накладной
РасходнаяНакладная = ФабрикаXDTO.Создать(РасходнаяНакладнаяТип);

Для Каждого СтрокаДокумента Из Документ.Состав Цикл

// Создать объекты XDTO строки расходной накладной
// и номенклатуры
СтрокаРасходнойНакладной = ФабрикаXDTO.Создать(СтрокаРасходнойНакладнойТип);
Номенклатура = ФабрикаXDTO.Создать(НоменклатураТип);

// Заполнить свойства номенклатуры
Номенклатура.Наименование = СтрокаДокумента.Номенклатура.Наименование;
Номенклатура.ПолноеНаименование = СтрокаДокумента.Номенклатура.ПолноеНаименование;
Номенклатура.ШтрихКод = СтрокаДокумента.Номенклатура.ШтрихКод;
Номенклатура.ЗакупочнаяЦена = СтрокаДокумента.Номенклатура.ЗакупочнаяЦена;

// Заполнить свойства строки расходной накладной
СтрокаРасходнойНакладной.Номенклатура = Номенклатура;
СтрокаРасходнойНакладной.Количество = СтрокаДокумента.Количество;
СтрокаРасходнойНакладной.Цена = СтрокаДокумента.Цена;
СтрокаРасходнойНакладной.Сумма = СтрокаДокумента.Сумма;

// Добавить строку расходной накладной
РасходнаяНакладная.Состав.Добавить(СтрокаРасходнойНакладной);

КонецЦикла;

// Вернуть расходную накладную
Возврат РасходнаяНакладная;

КонецФункции
```

В заключение следует опубликовать созданный Web-сервис на веб-сервере, например <http://www.MyCompany.ru>, в каталоге shipment.

17.2.4. Работа с веб-сервисами сторонних поставщиков

17.2.4.1. Общая информация

Система «1С:Предприятие» может использовать веб-сервисы, предоставляемые другими поставщиками, несколькими способами:

- с помощью статических ссылок, создаваемых в дереве конфигурации;
- с помощью динамических ссылок, создаваемых средствами встроенного языка;
- комбинацией предыдущих способов.

Преимущество использования статических ссылок заключается в большей скорости работы, т. к. описание веб-сервиса поставщика получается один раз, при создании ссылки. В дальнейшем при обращении к данному веб-сервису используется существующее описание веб-сервиса.

При использовании динамических ссылок описание веб-сервиса поставщика будет получаться системой «1С:Предприятие» каждый раз при вызове веб-сервиса, что, естественно, будет замедлять работу с данным веб-сервисом. Однако преимуществом такого подхода является возможность получения актуального описания веб-сервиса поставщика. При использовании же статических ссылок для получения актуального описания веб-сервиса следует выполнить повторный импорт WSDL-описания средствами конфигуратора и сохранение измененной конфигурации.

При эксплуатации прикладных решений может возникать ситуация, когда один и тот же веб-сервис предоставляется по разным адресам (URL), однако имеет абсолютно одинаковое описание (WSDL). В этом случае возникает потребность загрузить описание веб-сервиса в конфигурацию (создать объект в дереве объектов конфигурации), но во время использования указать конкретный адрес, по которому расположен веб-сервис. Комбинированный способ позволяет работать таким образом. В качестве примера можно рассмотреть следующую ситуацию: есть тиражируемый веб-сервис, выполняющий некоторую функцию. Прикладное решение, написанное на «1С:Предприятии», пользуется услугами данного сервиса, при этом адрес сервиса может быть различным (сервис тиражируемый), а описание - фиксированное. Тогда в прикладное решение может загрузить описание веб-сервиса, а в настройках прикладного решения предусмотреть ввод адреса конкретного экземпляра сервиса, который (адрес) и будет использоваться при работе.

Также доступен и другой способ: используется динамическая ссылка, но адрес расположения веб-сервиса получается не из файла описания (WSDL), а непосредственно указывается при создании объекта.

При попытке загрузить описание Web-сервиса в конфигураторе (создание статической ссылки) или при использовании динамической ссылки (при помощи объекта **WSОпределения**), система выполняет проверку загружаемого описания Web-сервиса (WSDL). Если в описании Web-сервиса присутствует ошибка (с «точки зрения» системы «1С:Предприятия»), то описание не будет загружено и будет сгенерировано исключение. В тексте исключения будет находиться подробная диагностика причин отказа в загрузке. Ошибки WSDL располагаются в порядке их обнаружения. Каждая ошибка WSDL содержит запись с детальным описанием следующего вида:

Копировать в буфер обмена

```
<Тип элемента с ошибкой>.<Имя>
  [<Тип элемента с ошибкой>.<Имя> [...]]
  <Описание ошибки>
  <Тип элемента с ошибкой>
  <Имя>
```

В этом описании:

- **Тип элемента с ошибкой** - тип элемента WSDL. Чаще всего соответствует имени тега в xml-файле, описывающем Web-сервис.
- **Описание ошибки** - описание конкретной ошибки.
- **Имя** - имя объекта WSDL, в котором произошла ошибка.

В нижеследующей таблице приведены ошибки, которые могут возникнуть при проверке описания Web-сервиса, и рекомендации по их (ошибок) исправлению:

Описание ошибки	Рекомендации по исправлению
Тип не найден	Проверить корректность пространств имен и имен типов и элементов
Сообщение не найдено	Проверить корректность пространств имен и имен сообщений
Привязка не найдена	Проверить корректность пространств имен и имен привязок
Тип порта не найден	Проверить корректность пространств имен и имен типов портов
Абстрактная операция не найдена	Проверить корректность имени абстрактной операции
Содержимое операции не найдено	Необходимо в операции указать содержимое. Это может быть вход или выход операции
Описание ошибки не найдено	Необходимо указать имя описания ошибки
Абстрактная операция не задана	Необходимо указать абстрактную операцию

Имя не задано	Необходимо указать имя объекта WSDL согласно схемы: http://schemas.xmlsoap.org/wsdl/2003-02-11.xsd
Вход операции не соответствует входу абстрактной операции	Необходимо, чтобы операция и абстрактная операция либо имели вход (обе), либо не имели его (обе). Нельзя, чтобы абстрактная операция имела вход, а операция нет и наоборот.
Выход операции не соответствует выходу абстрактной операции	Аналогично сообщению про вход операции
Тип порта не задан	Необходимо указать тип порта, проверить пространства имен
Объект не уникален	Необходимо проверить уникальность объекта по пространству имен и имени
Расположение не задано	Необходимо указать расположение Правило R2007 - для элемента import Правило R2711 - для элемента port
Пространства имен отличаются	Необходимо проверить и исправить отличия пространств имен импорта и пространства имен импортируемой схемы
Тип части сообщения не задан	Для части сообщения необходимо задать тип (тип или элемент, в зависимости от стиля WSDL), проверить пространства имен
Сообщение не задано	Для объекта WSDL необходимо указать или исправить имя сообщения, проверить его пространство имен
Абстрактной операции требуется вход	Необходимо указать вход для абстрактной операции
Абстрактной операции требуется отсутствие входа	Абстрактная операция не должна иметь вход
Абстрактной операции требуется выход	Необходимо указать выход для абстрактной операции
Абстрактной операции требуется отсутствие выхода	Абстрактная операция не должна иметь выход
Ошибка абстрактной операции не задана	Необходимо указать или проверить корректность имени абстрактной ошибки, пространства имен
Привязка не задана	Привязка порта не задана, некорректное имя привязки, некорректное пространство имен
Абстрактное описание операции не задано	Абстрактное описание операции не задано, некорректно указано имя или пространство имен
Пространство имен операции в rpc-literal стиле должно быть задано	Необходимо указать элемент namespace для операции. Правило R2717
Пространство имен операции в document-literal стиле не должно быть задано	Необходимо удалить элемент namespace для операции. Правило R2216
В rpc-literal стиле часть сообщения должна ссылаться на определение типа в схеме	Для указания типа части необходимо у соответствующего объекта message в частях сообщений использовать тип, а не элемент (тег type , а не element). Правило R2203
В document-literal стиле часть сообщения должна ссылаться на объявление элемента в схеме	Для указания типа части необходимо у соответствующего объекта message в частях сообщений использовать элемент, а не тип (тег element , а не type). Правило R2204
Неизвестный стиль привязки	Необходимо указать стиль привязки и проверить правильность его написания. Спецификация: http://www.w3.org/TR/wsdl#_bindings . Правила R2705, R2706
Транспорт в привязке должен быть задан как http://schemas.xmlsoap.org/soap/http	Необходимо проверить, чтобы транспорт был именно http://schemas.xmlsoap.org/soap/http . Правило R2702
Часть сообщения не задана	Необходимо для описания ошибки задать часть сообщения. Правило R2205
В схеме ошибка заголовка должна быть описана как объявление элемента	Необходимо в части сообщения, которая используется для описания ошибки использовать тип заданный как элемент (использовать тег element). Правило R2205
В схеме заголовок должен быть описан как объявление элемента	Необходимо в части сообщения, которая используется для описания заголовка использовать тип заданный как элемент (использовать тег element). Правило

	R2205
Схема должна содержать атрибут <code>targetNamespace</code>	Необходимо для схемы добавить атрибут <code>targetNamespace</code> и заполнить его. Это требование связано с особенностями реализации платформы
Ошибка загрузки импортируемой схемы	Проверить корректность пространств имен, расположения импортируемой схемы

При указании в таблице выражения вида [Правило RXXXX](#) понимается следующее - в спецификации Basic Profile Version 1.1 (<http://www.ws-i.org/profiles/basicprofile-1.1-2004-08-24.html>, на английском языке) необходимо найти правило [RXXXX](#).

17.2.4.2. Пример использования статической WS-ссылки

В качестве примера использования веб-сервисов стороннего поставщика рассмотрим обращение к Web-сервису, который ранее был создан в примере (см. [здесь](#)).

Прежде всего, следует добавить в дерево конфигурации новый объект конфигурации WS-ссылка с именем `ДанныеРасходнойНакладной`, ссылающийся на опубликованный сервис. Для этого следует выполнить импорт WSDL-описания опубликованного сервиса и в качестве URL указать <http://www.MyCompany.ru/shipment/ws/Shipment.1cws?wsdl>. Описание импорта WSDL-описания см. [здесь](#).

После этого, например, в модуле приходной накладной, можно создать процедуру, приведенную ниже. Она заполняет табличную часть документа данными расходной накладной поставщика, полученными с помощью веб-сервиса поставщика.

[Копировать в буфер обмена](#)

Процедура `ПолучитьДанныеРасходнойНакладной(НомерНакладнойПоставщика)`

```
// Создать WS прокси на основании ссылки
Прокси = WSCсылки.ДанныеРасходнойНакладной.СоздатьWSПрокси("http://www.MyCompany.ru/shipment",
"ДанныеРасходнойНакладной", "ДанныеРасходнойНакладнойSoap");

ДанныеНакладной = Прокси.Получить();

Если ДанныеНакладной = Неопределено Тогда

Возврат;

КонецЕсли;

// Заполнить приходную накладную полученными данными
Для Каждого СтрокаНакл Из ДанныеНакладной.Состав Цикл

НоваяСтрока = ДокументОбъект.Состав.Добавить();

НоваяСтрока.Количество = СтрокаНакл.Количество;
НоваяСтрока.Цена = СтрокаНакл.Цена;
НоваяСтрока.Сумма = СтрокаНакл.Сумма;

// Найти элемент номенклатуры по переданным данным
// (например, по штрихкоду)
НоваяСтрока.Номенклатура = Справочники.Номенклатура.НайтиПоРеквизиту("ШтрихКод",
СтрокаНакл.Номенклатура.ШтрихКод);

КонецЦикла;
```

КонецПроцедуры

Если адрес реального расположения сервиса отличается от адреса, который использовался во время загрузки описания веб-сервиса в конфигурацию, то новый адрес необходимо явно указать при создании объекта `WSПрокси`:

[Копировать в буфер обмена](#)

```
// Создать WS прокси на основании ссылки
Прокси = WSCсылки.ДанныеРасходнойНакладной.СоздатьWSПрокси("http://www.MyCompany.ru/shipment",
"ДанныеРасходнойНакладной", "ДанныеРасходнойНакладнойSoap", , , , "http://www.realURL/realPath");
```

17.2.4.3. Пример использования динамической WS-ссылки

Использование динамической ссылки отличается от использования статической ссылки только способом создания WS-прокси и отсутствием необходимости создавать WS-ссылку в дереве конфигурации.

Если провести сравнение с примером, представленным в предыдущем разделе, то, в отличие от создания прокси на основе статической ссылки, при использовании динамической ссылки WS-прокси создается с помощью конструктора следующим образом:

```
// Создать WS прокси на основании WS определения
```

```
Определение = Новый WSOпределения("http://www.MyCompany.ru/shipment/ws/Shipment.1cws?wsdl");
```

```
Прокси = Новый WSПрокси(Определение, "http://www.MyCompany.ru/shipment", "ДанныеРасходнойНакладной",  
"ДанныеРасходнойНакладнойSoap");
```

В то время как создание WS-прокси на основе статической ссылки выглядит следующим образом:

Копировать в буфер обмена

```
// Создать WS прокси на основании ссылки
```

```
Прокси = WSCсылки.ДанныеРасходнойНакладной.СоздатьWSПрокси("http://www.MyCompany.ru/shipment",  
"ДанныеРасходнойНакладной", "ДанныеРасходнойНакладнойSoap");
```

Если адрес реального расположения сервиса отличается от адреса, который указан в WSDL-файле, используемом при создании определения веб-сервисов, новый адрес необходимо явно указать при создании объекта **WSПрокси** на основании определения веб-сервисов:

Копировать в буфер обмена

```
// Создать WS прокси на основании WS определения
```

```
Определение = Новый WSOпределения("http://www.MyCompany.ru/shipment/ws/Shipment.1cws?wsdl");
```

```
Прокси = Новый WSПрокси(Определение, "http://www.MyCompany.ru/shipment", "ДанныеРасходнойНакладной",  
"ДанныеРасходнойНакладнойSoap", , , , "http://www.realURL/realPath");
```

17.2.5. Редактирование свойств Web-сервиса

На закладке **Основные** вводится имя, синоним и комментарий объекта.

На закладке **Операции** создаются подчиненные объекты **Операции**, которые, в свою очередь, могут иметь подчиненные объекты **Параметры**, необходимые для работы с объектами данного типа. Описание подчиненных объектов выполняется в палитре свойств.

17.2.5.1. Свойства «Операции»

Помимо общих свойств объектов конфигурации операция Web-сервиса содержит следующие свойства:

- **Тип возвращаемого значения** - тип значения, которое возвращает операция Web-сервиса. Может являться типом значения XDTO или типом объекта XDTO.
- **Возможно пустое значение** - показывает, может ли возвращаемое значение принимать неопределенное значение.
- **В транзакции** - показывает, будет ли выполняться код модуля Web-сервиса в транзакции или нет. Если свойство установлено, то при вызове Web-сервиса автоматически будет начата транзакция, а при завершении работы Web-сервиса транзакция будет либо зафиксирована, либо произойдет откат транзакции (в зависимости от результатов выполнения). Если свойство не установлено, при начале исполнения модуля Web-сервиса транзакция не будет начата.
- **Имя метода** - имя экспортируемой процедуры модуля Web-сервиса, которая будет выполнена при вызове данного свойства.

На закладке **Подсистемы** указывается, к каким подсистемам относятся объекты данного типа.

На закладке **Прочее** определяются следующие свойства:

- **URI пространства имен** - содержит URI пространства имен Web-сервиса. Каждый Web-сервис может быть однозначно идентифицирован по своему имени и URI пространству имен, которому он принадлежит. Пространство имен сервиса не должно совпадать с известными пространствами имен, которые уже используются или зарезервированы другими организациями. Рекомендуется в пространство имен сервиса включать фрагмент, уникальный для организации, которая ведет разработку Web-сервиса. Например, для организации с названием **Промресурс** имеет смысл начинать все пространство имен с префикса, например, <http://promresurs.com>. Тогда URI пространства имен Web-сервиса будет иметь вид <http://promresurs.com/public/services/OurService>.
- **Пакеты XDTO** - перечень пакетов XDTO, типы которых могут использоваться в качестве типов возвращаемого значения операций и типов параметров операций Web-сервиса.
- **Имя файла публикации** - имя файла описания Web-сервиса, который расположен на веб-сервере.

По кнопке **Модуль** открывается редактор модуля Web-сервиса.

17.2.5.2. Свойства «Параметр»

Помимо общих свойств объектов конфигурации параметр операции Web-сервиса содержит следующие свойства:

- **Тип значения** - тип значения параметра операции Web-сервиса. Может являться типом значения XDTO или типом объекта XDTO.
- **Возможно пустое значение** - показывает, может ли значение параметра операции принимать неопределенное значение.
- **Направление передачи** - определяет направление передачи данных с помощью данного параметра. Возможные значения:
 - **Входной** - означает, что параметр используется для передачи данных Web-сервису;

- **Выходной** - означает, что параметр используется для получения данных от Web-сервиса;
- **Входной-Выходной** - означает, что параметр может использоваться как для передачи данных, так и для их получения от Web-сервиса.

17.3. HTTP-сервисы

17.3.1. Использование HTTP-сервисов

17.3.1.1. Общая информация

HTTP-сервисы (и REST-интерфейсы) могут предоставлять не только системы, созданные на платформе «1С:Предприятие», но и (в общем случае) любые системы, которые доступны в сети Интернет. В широком смысле, при упоминании REST-интерфейса, можно говорить о некотором клиент-серверном взаимодействии. На стороне сервера реализован некоторый программный интерфейс (REST-интерфейс), которым пользуется клиент (конфигурация на базе «1С:Предприятие»). Для получения данных клиент использует HTTP-запросы. В качестве транспорта этих запросов (и ответов сервера на них) выступает протокол HTTP.

При рассмотрении HTTP-сервисов мы будем постоянно оперировать следующими понятиями:

- **Ресурс** - любая информация, которая может быть получена при обращении к REST-интерфейсу.
- **URI** - универсальный идентификатор ресурса. Фактически это адрес в сети, обращение к которому приведет к какому-либо действию.
- **Запрос** - аналогичен вызову функции. Что-бы выполнить какое-либо действие - необходимо отправить запрос к необходимому ресурсу. В протоколе HTTP определены варианты запросов, которые называются «методами». Возможно определение собственных вариантов запросов. Описание стандартных методов HTTP-запросов приведено тут (<https://datatracker.ietf.org/doc/html/rfc7231#section-4>) или тут (<https://developer.mozilla.org/ru/docs/Web/HTTP/Methods>). В частности, с помощью HTTP-запросов можно реализовать аналог концепции CRUD работы с базой данных:
 - **Create** - создать запись в базе данных. HTTP-аналогом выступает метод **POST**.
 - **Read** - прочитать запись из базы данных: метод **GET**.
 - **Update** - обновить запись в базе данных: метод **PUT**.
 - **Delete** - удалить запись в базе данных: метод **DELETE**.

Приведенное соответствие не является единственно возможным. Например, для того чтобы выполнить изменение данных, используемый сервер будет требовать использовать метод **PATCH**, а не метод **PUT**.

- **Код состояния (или код ответа)** - число, с помощью которого клиент может определить результат запроса и «понять», какие действия необходимо выполнить в дальнейшем. Код состояния является целым числом, состоящим из 3-х цифр и формируемым по специальным правилам. Описание стандартных кодов состояния приведено тут (<https://datatracker.ietf.org/doc/html/rfc7231#section-6>) или тут (<https://developer.mozilla.org/ru/docs/Web/HTTP/Status>).

Таким образом, общая схема работы выглядит следующим образом:

- Клиент устанавливает HTTP-соединение с сервером.
- Клиент выполняет запрос определенного вида к ресурсу, имеющему конкретный идентификатор.
- Сервер обрабатывает запрос, формирует ответ и код состояния.
- Ответ и код состояния возвращаются на клиента.
- Клиент анализирует код состояния и выполняет обработку тела ответа (если нужно).

При клиент-серверном взаимодействии используются сообщения, которые состоят из следующих частей:

1. **Стартовая строка** - определяет тип сообщения. В стартовой строке, в том числе, указывается и то, какой HTTP-запрос (метод) передается в данном сообщении.
2. **Заголовки** - различные вспомогательные параметры, которые используются для служебных целей. Каждый заголовок представляет из себя пару «ключ-значение». Стандартные заголовки описываются тут (<https://www.iana.org/assignments/message-headers/message-headers.xhtml>) или тут (<https://developer.mozilla.org/ru/docs/Web/HTTP/Headers>).
3. **Тело сообщения** - если присутствует, используется для передачи произвольных данных, связанных в запросом или ответом. Необходимость тела сообщения определяется используемым методом и прикладной реализацией самого метода на стороне сервера. Формат содержимого тела сообщения не стандартизируется.

Система «1С:Предприятие» предоставляет определенный набор объектов, который позволяет разработчику выполнить все вышеперечисленные действия:

- Объект **HTTPСоединение** предназначен для установки соединения с внешней системой и выполнения HTTP-запросов к этой системе.
- Объект **HTTPЗапрос** предназначен для описания HTTP-запросов, отправляемых через объект HTTPСоединение. Позволяет задать адрес запрашиваемого ресурса, заголовки и тело запроса.
- Объект **HTTPОтвет** предоставляет доступ к содержимому ответа, который сформирован сервером в результате обработки HTTP-запроса.

Таким образом, разработчику нет необходимости вручную формировать текст сообщения. Достаточно воспользоваться объектной моделью, а платформа «1С:Предприятие» автоматически сформирует тело сообщения в соответствии с используемыми стандартами и отправит серверу. Ответное сообщение также автоматически будет преобразовано в соответствующий объект при получении ответа от сервера.

В зависимости от того, где выполняется работа с HTTP-запросом, система «1С:Предприятие» предоставляет два варианта работы:

- Синхронный вариант использования - доступен только на стороне сервера система «1С:Предприятие».
- Асинхронный вариант - доступен только на стороне клиентского приложения. При этом работа на стороне веб-клиента накладывает дополнительные ограничения (обусловленные веб-браузером и принятой в нем модели безопасности), которые будут отражены в описании того или иного метода в синтаксис-помощнике. Асинхронный метод отличается от синхронного наличием суффикса **Асинх** в конце имени метода: **ВызватьHTTPМетод()** - синхронный метод, **ВызватьHTTPМетодАсинх()** - асинхронный метод.

17.3.1.2. Общая схема работы с HTTP-запросами

Для того, чтобы использовать HTTP-запросы, в первую очередь необходимо создать HTTP-соединение с ресурсом, который будет использоваться. Для этого предназначен объект **HTTPСоединение**. Для примеров будем использовать сайт <http://httpbin.org> (исходный код сервиса: <https://github.com/postmanlabs/httpbin>), который предоставляет простой REST-интерфейс для проверки своего кода (можно, также, использовать сервис <https://reqres.in/>). Также все примеры будут работать на стороне клиентского приложения, а поэтому будут использовать асинхронную модель работы.

Рассмотрим пример создания объекта **HTTPСоединение**:

[Копировать в буфер обмена](#)

```
Соединение = Новый HTTPСоединение("httpbin.org", , , , 5);
```

В данном примере создается соединение с сервером <http://httpbin.org> и устанавливается тайм-аут на выполнение действий с сервером равный 5 секунд. В данном примере важно обратить внимание на следующие моменты:

- В адресе сервера не требуется указывать схему доступа к серверу (HTTP или HTTPS) и путь на самом сервере (идентификатор ресурса). Схема доступа указывается с помощью параметра конструктора **ЗащищенноеСоединение**. Идентификатор ресурса будет указываться при создании HTTP-запроса. При создании HTTP-соединения требуется указать только адрес самого сервера.
- Для доступа к серверу используется стандартный порт 80. Если сервер требует указания другого порта, то для этого следует использовать параметр **Порт**.
- Для доступа к тестовому серверу не требуется указывать имя пользователя и пароль на самом сервере, поэтому параметры конструктора HTTP-соединения **Пользователь** и **Пароль** не указаны. Если сервер выполняет аутентификацию пользователя - параметры следует использовать.
- Для доступа к серверу используются параметры прокси-сервера по умолчанию. Если прокси-сервер требует указания имени пользователя и пароля, то тогда необходимо создать объект **ИнтернетПрокси** и заполнить соответствующие параметры объекта. Прокси-сервер - это транзитный сервер, перенаправляющий HTTP-трафик. Прокси-серверы используются для ускорения выполнения запросов путем кэширования веб-страниц, а также как средство управления HTTP-трафиком (например, для блокирования доступа к некоторым ресурсам) и межсетевой экран.
- В данном примере используется обычное, не защищенное соединение.

Таким образом, выполнив данный программный код, мы создадим HTTP-соединение с сервером. Это соединение мы будем использовать далее, при обращении к серверу.

Теперь нам необходимо выполнить запрос **GET** к ресурсу **/get** используемого сервера. Запрос не имеет параметров. Рассмотрим, как это делается:

[Копировать в буфер обмена](#)

```
Запрос = Новый HTTPЗапрос("/get");  
ОбещанныйРезультат = Соединение.ВызватьHTTPМетодАсинх("GET", Запрос);  
Результат = Ждать ОбещанныйРезультат;
```

В данном примере используется асинхронный запрос **GET** к используемому серверу. Следует отметить следующие особенности примера:

- Запрос будет выполняться асинхронно. Так как это асинхронный запрос, то после отправки запроса серверу, разработчик может реализовать еще какой-то код, который будет выполнен пока выполняется HTTP-запрос на сервер.
- Запрос должен быть выполнен за время, не превышающее 5 секунд. Именно такой тайм-аут был установлен на HTTP-соединение, которое используется для работы с нашим запросом. Если требуется изменить тайм-аут выполнения запроса - следует использовать параметр **ТаймАут** конструктора объекта **HTTPСоединение**.

Таким образом, если не произошло каких-то инфраструктурных ошибок (которые приведут к формированию исключения), в переменной **Результат** будет значение типа **HTTPОтвет**. После получения ответа можно проанализировать результат выполнения запроса.

Во-первых, необходимо проверить статус выполнения нашего запроса. Для этого следует использовать свойство **КодСостояния** объекта **HTTPОтвет**. Если данное свойство установлено в значение 200, то это значит, что запрос выполнен успешно. В этом случае можно выполнять анализ тела ответа, если сервер вернул какие-то данные в теле ответа.

[Копировать в буфер обмена](#)

```
Если Результат.КодСостояния = 200 Тогда
```

```
// удачное завершение
Иначе
// что-то пошло не так
КонецЕсли;
```

Если код состояния отличен от 200 (а более широко - от кодов группы 2XX), это означает, что запрос выполнен unsuccessfully. При этом код состояния, отличный от 2XX не означает, что запрос выполнен с ошибкой. Например, сервер сообщает, что запрос следует выполнить, указав другой URI, т. к. у ресурса установлена переадресация. Об этом будут сообщать коды состояния группы 3XX. В этом случае прикладной код должен проанализировать заголовки ответа и получить из них информацию о том, как надо выполнить запрос еще раз.

Для того, чтобы проанализировать заголовки ответа сервера, нам потребуется свойство **Заголовки** объекта **HTTPОтвет**. В этом свойстве находится соответствие, каждый элемент которого представляет собой один элемент заголовка ответного сообщения сервера. Следующий пример выведет все заголовки, которые доступны в ответе сервера:

Копировать в буфер обмена

```
Для каждого Заголовок Из Результат.Заголовки Цикл
    Сообщить (" " + Заголовок.Ключ + " = " + Заголовок.Значение);
КонецЦикла;
```

При отправке запроса может также потребоваться заполнять заголовки сообщения. Это можно сделать с помощью свойства **Заголовки** объекта **HTTPЗапрос**. Это свойство также является соответствием. Например, если нам необходимо, чтобы сервер вернул ответ в формате JSON, то нам потребуется установить заголовок **Content-type** следующим образом:

Копировать в буфер обмена

```
Запрос.Заголовки.Вставить ("Content-type", "application/json");
```

Так делать имеет смысл только в том случае, если сервер анализирует данный заголовок и позволяет вернуть ответ в нескольких различных форматах. При этом надо понимать, что формат возвращаемого значения может определяться не только заголовком сообщения, но и параметрами запроса.

Параметры запроса устанавливаются при формировании объекта HTTP-запрос. Параметры запроса (в отличие от заголовков запроса) - предназначены для обработки непосредственно прикладным кодом сервера и эти параметры имеют прикладную специфику. Параметры запроса также описываются парой «ключ-значение» и указываются в тексте идентификатора ресурса. При этом список параметров отделяется от URI символом «?», а между собой параметры разделяются символом «&».

В принципе, HTTP-сервис может получать параметры как в составе URI, так и с помощью параметров запроса. Например, если требуется получить документ с некоторым номером, то это можно будет сделать следующими способами (приведен полный URI для запроса **GET** на мифический сервис):

- <http://my.example.com/documents/16> - получить документ с номером 16.
- <http://my.example.com/documents&id=16> - получить документ с номером 16.

То, каким образом принимает параметры тот или иной сервис - полностью зависит от его (сервиса) реализации и подходов разработчика сервиса. Формирование HTTP-запроса будет практически одинаковым:

Копировать в буфер обмена

```
Запрос = Новый HTTPЗапрос ("/documents/16");
Запрос = Новый HTTPЗапрос ("/documents?id=16");
```

Если рассматривать формирование URI и определение параметров запроса с точки зрения бизнес-логики, то можно сформулировать следующее условное разделение:

- Если «что-то» попадает в состав URI - скорее всего это является бизнес-сущностью (документы, пользователи т .д.).
- Если «что-то» попадает в параметры запроса - скорее всего это фильтр для объектов используемой бизнес-сущности. Например, только проведенные документы или только внешние пользователи.

В приведенном выше примере используется универсальный метод выполнения любого HTTP-запроса: **ВызватьHTTPМетодАсинх()**. В этом методе указывается выполняемый метод и описание запроса. Однако, если вам по каким-то причинам не подходит такой универсальный метод, то можно использовать набор специализированных методов, которые также предоставляются объектом **HTTPСоединение**. Эти методы объекта **HTTPСоединение** позволяют выполнить только некоторое подмножество HTTP-запросов:

Метод объекта HTTPСоединение	Метод HTTP
УдалитьАсинх()	DELETE
ПолучитьАсинх()	GET
ПолучитьЗаголовкиАсинх()	HEAD
ИзменитьАсинх()	PATCH
ОтправитьДляОбработкиАсинх()	POST
ЗаписатьАсинх()	PUT

В заключение этого раздела осталось рассмотреть способы работы с телом запроса. Еще раз повторим, что содержимое и формат тела запроса определяется тем интерфейсом, который выставляет REST-сервис, для использования которого мы пишем программный код. Тем не менее, платформа «1С:Предприятие» предоставляет несколько универсальных методов для получения доступа к телу запроса:

- Использовать тело запроса как строку:

- `HTTPЗапрос.ПолучитьТелоКакСтроку()` / `УстановитьТелоИзСтроки()`.
- `HTTPОтвет.ПолучитьТелоКакСтроку()`.
- Использовать тело запроса как двоичные данные:
 - `HTTPЗапрос.ПолучитьТелоКакДвоичныеданные()` / `УстановитьТелоИзДвоичныхДанных()`.
 - `HTTPОтвет.ПолучитьТелоКакДвоичныеданные()`.
- Использовать тело запроса как поток:
 - `HTTPЗапрос.ПолучитьТелоКакПоток()`.
 - `HTTPОтвет.ПолучитьТелоКакПоток()`.
- Использовать файл в качестве тела запроса:
 - `HTTPЗапрос.ПолучитьИмяФайлаТела()` / `УстановитьИмяФайлаТела()`.
 - `HTTPОтвет.ПолучитьИмяФайлаТела()`.

Так, если какой-то запрос возвращает в качестве результат тело запроса, которое содержит данные в формате JSON, то следующий пример кода на встроенном языке будет возвращать соответствие, которое содержит данные из тела ответа (в переменной `Данные`):

```
Читатель = Новый ЧтениеJSON;
Читатель.УстановитьСтроку(Результат.ПолучитьТелоКакСтроку());
Данные = ПрочитатьJSON(Читатель, Истина);
Читатель.Закреть();
```

[Копировать в буфер обмена](#)

Аналогичным образом можно формировать тело и для HTTP-запросов к используемому сервису.

17.3.1.3. Работа через защищенное соединение

В предыдущем разделе мы рассмотрели использование HTTP-запросов через HTTP-соединение. Такое соединение не является безопасным, а значит передавать по нему чувствительные данные - не очень хорошая идея. В этом случае следует использовать защищенное соединение, которое использует протокол HTTPS. Рассмотрим, что необходимо изменить для возможности использования HTTP-запросов в защищенном соединении.

Для того, чтобы использовать защищенное соединение для работы с HTTP-запросами, нам будет необходимо указать значение параметра `ЗащищенноеСоединение` конструктора `HTTPСоединение`. Значением этого параметра может быть объект типа `ЗащищенноеСоединениеOpenSSL` или `ЗащищенноеСоединениеКриптоПро`. В зависимости от используемого объекта будет использоваться библиотека криптопровайдеров OpenSSL или КриптоПро.

```
//ЗащищенноеСоединение = Новый ЗащищенноеСоединениеКриптоПро(, СертификатыУдостоверяющихЦентровОС);
ЗащищенноеСоединение = Новый ЗащищенноеСоединениеOpenSSL(, СертификатыУдостоверяющихЦентровОС);
Соединение = Новый HTTPСоединение("httpbin.org", , , , , 5, ЗащищенноеСоединение);
```

[Копировать в буфер обмена](#)

Собственно, на этом все изменения будут закончены. Следует обратить внимание на следующие свойства, которые изменятся в свойствах объекта `HTTPСоединение`, если соединение установлено по протоколу HTTPS:

- Свойство `ЗащищенноеСоединение` установлено в значение типа `ЗащищенноеСоединениеOpenSSL` (как в примере выше) или `ЗащищенноеСоединениеКриптоПро`. Для обычного соединения это свойство будет установлено в значение `Неопределено`.
- Свойство `Порт` установлено в значение 443 (порт по умолчанию для HTTPS-соединения). Для обычного соединения это свойство установлено в значение 80. Если порт явно указывается в конструкторе, то в свойстве `Порт` будет содержаться явно указанное значение.

При создании любого объекта `ЗащищенноеСоединение*` можно управлять тем, откуда будет получен клиентский сертификат (свойство `СертификатКлиента`) для установки соединения и откуда будут получаться сертификаты удостоверяющих центров, с помощью которых будет выполняться проверка валидности сертификата сервера (свойство `СертификатыУдостоверяющихЦентров`). Также предоставляется возможность управлять поведением системы при недоступности сервера проверки отзыва сертификата (свойство `РежимПроверкиОтзываTLSCертификатаСервера`).

Местоположение клиентского сертификата зависит от того, какое значение находится в свойстве `СертификатКлиента` (или соответствующем параметре конструктора):

- `Неопределено` - в этом случае соединение будет установлено только в том случае, если сервер не требует предъявления клиентского сертификата.
- `СертификатКриптографии` - клиентский сертификат получен из двоичных данных или файла. Значение данного типа может быть получено в результате поиска сертификата (по различным критериям) в хранилище сертификатов криптографии (семейство методов `НайтиПо*` объекта `ХранилищеСертификатовКриптографии`).
- `СертификатКлиентаФайл` - клиентский сертификат находится в файле, указанном при создании объекта `СертификатКлиентаФайл`.
- `СертификатКлиентаОС` - в этом случае клиентский сертификат получается из системного хранилища той операционной системы, на которой выполняется код на встроенном языке. Именно таким образом рекомендуется указывать хранилище сертификатов при написании кроссплатформенного кода.

- `СертификатКлиентаLinux`, `СертификатКлиентаMacOS` и `СертификатКлиентаWindows` - клиентский сертификат находится в системном хранилище соответствующей операционной системы.

Если клиентский сертификат выбирается в системном хранилище сертификатов ОС и будет найдено несколько подходящих сертификатов, то способ выбора сертификата может быть задан с помощью параметра `СпособВыбораСертификата*` для объектов `СертификатКлиентаWindows`, `СертификатКлиентаMacOS` и `СертификатКлиентаОС`.

Местоположение набора сертификатов удостоверяющих центров, которые используются для проверки серверного сертификата, задается с помощью свойства `СертификатыУдостоверяющихЦентров` и соответствующего параметра конструктора:

- `Неопределено` - в этом случае устанавливается HTTPS-соединение без проверки серверного сертификата.
- `СертификатыУдостоверяющихЦентровФайл` - для проверки серверного сертификата используются сертификаты удостоверяющих центров, которые находятся в файле, указанном при создании объекта `СертификатыУдостоверяющихЦентровФайл`.
- `СертификатыУдостоверяющихЦентровОС` - в этом случае для проверки серверного сертификата используются системные хранилища сертификатов удостоверяющих центров той операционной системы, на которой выполняется код на встроенном языке. Именно таким образом рекомендуется указывать хранилище сертификатов при написании кроссплатформенного кода.
- `СертификатыУдостоверяющихЦентровLinux`, `СертификатыУдостоверяющихЦентровMacOS` и `СертификатыУдостоверяющихЦентровWindows` - в этом случае для проверки серверного сертификата используются системные хранилища сертификатов удостоверяющих центров соответствующей операционной системы.

Свойство `РежимПроверкиОтзываTLSCертификатаСервера` (или соответствующий параметр конструктора) управляет поведением системы при невозможности проверки отзыва предоставленного сертификата сервера. Значением данного параметра может быть значение системного перечисления `РежимПроверкиОтзываTLSCертификатаСервера`:

- `Авто` - в этом случае поведение системы определяется параметром `IgnoreServerCertificatesChainRevocationSoftFail` конфигурационного файла `conf.cfg` (см. [здесь](#)). При настройках по умолчанию, в этом случае поведение будет соответствовать значению `Нестрогий` данного перечисления. Если параметр не указан - будет использовано это значение.
- `Строгий` - невозможность проверки отзыва сертификата приводит к исключению и невозможности установки соединения.
- `Нестрогий` - в случае невозможности проверки отзыва сертификата сервера соединение будет установлено.
- `НеПроверять` - проверка отзыва сертификата сервера не выполняется.

Не рекомендуется в промышленных системах работать с сертификатом сервера, для которого не выполняется проверка отзыва. Это может привести к проблемам с безопасностью.

17.3.1.4. Использование JWT-аутентификации

17.3.1.4.1. Общая информация

Токен - средство идентификации пользователя, группы пользователей или отдельного сеанса работы в приложении. **JSON Web Token** (сокращенно **JWT**) - стандарт создания токенов доступа, основанный на формате JSON. JWT описывается в стандарте RFC 7519 (<https://datatracker.ietf.org/doc/html/rfc7519>). JWT используется для аутентификации пользователя в клиент-серверных приложениях. Использование JWT можно описать следующим порядком действий:

1. Пользователь аутентифицируется на сервере аутентификации, например, с использованием имени пользователя и пароля.
2. В ответ сервер аутентификации предоставляет пользователю JWT.
3. Пользователь обращается к приложению, представляясь этому приложению путем предъявления JWT.
4. Если предъявленный JWT проходит проверку приложением, приложение выполняет запрошенное действие и возвращает результат.

Кратко рассмотрим устройство JWT и то, каким способом этот инструмент может быть использован в системе «1С:Предприятие».

Логически JWT состоит из трех частей: заголовок (header), полезная нагрузка (payload), подпись (signature). Заголовок и полезная нагрузка представляют из себя объекты в формате JSON. Заголовок содержит служебную информацию, которая должна позволить проверяющей стороне корректно обработать данные JWT. Полезная нагрузка состоит из **претензий** (или требований, заявок, в оригинале - **JWT claims**), которые описывают прикладные данные, которые требуются для приложения, которое выполняет аутентификацию (выполняет запрос, аутентификация для которого выполняется с помощью JWT). Стандартные имена претензий перечислены в разделе 4.1 стандарта RFC 7519. Разработчик может создать собственное имя претензии. Важно, чтобы имена претензий, в рамках одного токена, были уникальными.

Подпись формируется на основании заголовка и полезной нагрузки с помощью специальной функции, описанной в заголовке и преобразованной в формат `Base64URL`. Физически JWT представляет собой строку, которая состоит из трех частей, разделенных символами «.». Заголовок и полезная нагрузка также подвергаются кодированию с помощью алгоритма `Base64URL`.

Рассмотрим пример.

Заголовок содержит следующую информацию:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Полезная нагрузка содержит следующую информацию:

[Копировать в буфер обмена](#)

```
{
  "id": "1234567890",
  "name": "Иван Иванов",
  "role": "administrator"
}
```

JWT будет выглядеть следующим образом (токен разделен на три строки):

[Копировать в буфер обмена](#)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEyMzQ1Njc4OTAiLCJ1eWl1Ijo0JjQstCw0L0g0JjQstCw0L3QvtCyIiwicm9sZSI6ImFkbWluaXN0cmF0b3IifQ.n-0muDfzGhlMQbzaCxeIU8JItOnlYr4RdJq4G8RUnck
```

В этом примере разделение по строкам приведено для удобства восприятия, а собственно разделение выполнено по символу «.». Если средство просмотра документации не позволяет второй строке (начинается с символов «eyJpZCI6I») целиком поместиться по ширине просмотра, то пример будет отображаться более, чем в 3 строки.

Очевидно, что чем больше информации находится в разделе полезной нагрузки, тем больший размер будет занимать JWT.

В системе «1С:Предприятие» JWT может использоваться в следующих случаях:

- В качестве одного из способов аутентификации пользователей информационной базы «1С:Предприятия».
- В качестве одного из способов аутентификации при использовании HTTP- и Web-сервисов.
- При настройке публикации информационной базы на веб-сервере.

Для того, чтобы поэкспериментировать с различным наполнением JWT, можно использовать, например, сайт <https://jwt.io>.

17.3.1.4.2. Формирование JWT из встроенного языка

Для того, чтобы создать JWT с помощью встроенного языка, предназначен объект **ТокенДоступа**. Строковое представление экземпляра значения данного типа будет являться текстом JWT (в формате **Base64URL**). Получившуюся строку можно использовать в тех случаях, когда удаленная система требует указать JWT в качестве параметра метода или значения параметра командной строки запуска.

Для того, чтобы указать заголовок токена, предназначено свойство **Заголовки** объекта **ТокенДоступа**. В этом заголовке расположено соответствие, где каждый элемент соответствия описывает один параметр заголовка. Ключ элемента соответствия выступает имя параметра, а значение параметра записывается в значение элемента соответствия. Для формирования корректного токена, всегда должны быть установлены заголовки **typ** и **alg**. Если JWT формируется для использования платформой «1С:Предприятие», то заголовок **typ** может принимать только значения **JWT** и **JWS**. Поддерживаемые алгоритмы формирования подписи содержатся в системном перечислении **АлгоритмПодписиТокенаДоступа**. Не рекомендуется отключать использование подписи JWT (заголовок **alg** установленный в значение **none** отключает подпись и не позволяет проверить источник JWT).

Для того, чтобы сформировать полезную нагрузку, в объекте **ТокенДоступа** используется свойство **ПолезнаяНагрузка**, которое позволяет задать любую претензию. Кроме того, есть несколько претензий, значения которых можно изменять через свойства объекта **ТокенДоступа**. Далее приведено соответствие свойств объекта **ТокенДоступа** и соответствующих претензий:

Свойство объекта	Имя и краткое описание претензии
ВремяЖизни	exp . Время в Unix-формате (число секунд, прошедших с 1.01.1970), определяющее момент, когда истечет срок действия токена.
ВремяСоздания	iat , nbfc . Время в Unix-формате (число секунд, прошедших с 1.01.1970), определяющее момент, когда создан токен (iat) и когда для токена начинается срок его действия (nbfc). Система «1С:Предприятие» изменяет одновременно обе претензии при изменении свойства ВремяСоздания . Если претензии iat и nbfc необходимо заполнять разными значениями - надо это делать через свойство ПолезнаяНагрузка .
Идентификатор	jti . Уникальный идентификатор сформированного токена.
КлючСопоставленияПользователя	sub . Уникальный идентификатор субъекта, для которого сформирован токен. Уникальность должна обеспечиваться в рамках той системы, которая обслуживает запрос. Можно сказать, что эта претензия содержит имя пользователя, от имени которого будет выполняться действие в приложении.
Получатели	aud , передается массивом. Список уникальных идентификаторов приложений, для которых сформирован данный токен. Если текущее приложение не входит в этот список - токен должен быть проигнорирован.
Эмитент	iss . Идентификатор приложения, выдавшего токен.

Если изменять претензии через свойства объекта и коллекцию претензий, то действуют следующие правила:

- Изменение свойства объекта `ТокенДоступа` приводит к автоматическому созданию или изменению соответствующей претензии.
- Изменение именованной претензии не приводит к изменению «связанного» свойства объекта `ТокенДоступа`.

Из перечисленных правил следует простой вывод: манипуляции с полезной нагрузкой рекомендуется выполнять или через конкретные свойства объекта `ТокенДоступа` (перечисленные в таблице выше) или через свойство `ПолезнаяНагрузка`. Одновременное использование и свойств объекта и имен конкретных претензий (из стандарта RFC 7519) может привести к неожиданным последствиям. Рекомендуется использовать свойства `Заголовки` и `ПолезнаяНагрузка` в том случае, когда интерфейса объекта `ТокенДоступа` не достаточно для того, чтобы сформировать JWT для какого-либо используемого сервиса.

Таким образом для формирования JWT можно использовать такой код на встроенном языке:

[Копировать в буфер обмена](#)

```
АлгоритмПодписи = АлгоритмПодписиТокенаДоступа.HS256;
ТокенДоступа = Новый ТокенДоступа;
ТокенДоступа.Заголовки.Вставить("alg", Строка(АлгоритмПодписи));
ТокенДоступа.Эмитент = "ERP";
ТокенДоступа.Получатели.Добавить("Д08");
ТокенДоступа.КлючСопоставленияПользователя = "ИвановИИ";
ТокенДоступа.ВремяСоздания = ТекущаяУниверсальнаяДата() - Дата(1970,1,1,0,0,0);
ТокенДоступа.ВремяЖизни = 3600;
ТокенДоступа.Идентификатор = Новый УникальныйИдентификатор;
ТокенДоступа.ПодписатьАсинх(АлгоритмПодписи, КлючПодписи);
ТекстТокена = Строка(ТокенДоступа); // получим токен строкой
```

17.3.1.4.3. Использование JWT

Если сервер, к которому выполняется HTTP-запрос, требует указания JWT, то это можно осуществить несколькими различными способами. Но технически, все возможности опираются или на формирование заголовка HTTP-запроса, или на формирование параметра запроса.

Самым простым способом выглядит явное указание «токена на предъявителя» (bearer token) в заголовке `Authorization` HTTP-запроса к серверу:

[Копировать в буфер обмена](#)

```
Запрос = Новый HTTPЗапрос("/get");
Запрос.Заголовки.Вставить("Authorization", "Bearer " + ТекстТокена);
```

Аналогичный заголовок будет сформирован также в том случае, если будет использован метод `ДобавитьТокенДоступа()` объекта `HTTPЗапрос`. Разница с предыдущим способом заключается в том, что в качестве параметра метода выступает не строка, а значение типа `ТокенДоступа`:

[Копировать в буфер обмена](#)

```
Запрос = Новый HTTPЗапрос("/get");
Запрос.ДобавитьТокенДоступа(ТокенДоступа);
```

Если сервер требует токен доступа в виде параметра запроса, то сделать это можно следующим образом:

[Копировать в буфер обмена](#)

```
Запрос = Новый HTTPЗапрос("/get?AccessToken=" + ТекстТокена);
```

Если необходимо передать токен доступа при запуске клиентского приложения системы «1С:Предприятия», то сделать это с использованием командной строки запуска. Команда `AccessToken` предназначена для передачи в клиентское приложение JWT в текстовом виде. Для того, чтобы такое действие имело смысл, для пользователей информационной базы должна быть включена возможность аутентификации токеном доступа.

Смотри также:

- Параметры пользователя информационной базы (см. [здесь](#)).
- Файл `default.vrd` (см. [здесь](#)).
- Командная строка запуска клиентского приложения «1С:Предприятия» (см. [здесь](#)).
- Командная строка запуска веб-клиента (см. [здесь](#)).

17.3.2. Создание собственных HTTP-сервисов

17.3.2.1. Общая информация

Обращение к HTTP-сервису, который реализован в рамках прикладного решения, выполняется по некоторому URL. В качестве примера можно привести HTTP-сервис, который при обращении по одному URL возвращает список каких-либо документов (например, расходных накладных), а при обращении по другому URL будет возвращена конкретная накладная. Действие, которое следует выполнить серверу, определяется тем, какой HTTP-метод адресован серверу. Например, если на сервер отправляется GET-запрос, то выполняется получение каких-либо данных, DELETE-запрос приводит к удалению каких-либо данных и т. д.

При реализации HTTP-сервиса прикладной разработчик определяет следующие элементы сервиса:

1. Определяет базовую часть URL, по которому будет выполняться обращение к сервису.
2. Определяет состав предоставляемой функциональности и структуру ресурсов, на которые эта функциональность будет

отображаться.

3. Определяет действия (HTTP-методы), которые можно будет выполнить при обращении к тому или иному ресурсу.

4. Для каждого выбранного действия реализуется специальный метод, написанный на встроенном языке системы «1С:Предприятие», который и реализует необходимую функциональность. Метод расположен в специальном модуле, связанным с HTTP-сервисом.

После реализации HTTP-сервиса, необходимо опубликовать его на веб-сервере с помощью стандартного механизма публикации.

При обращении к HTTP-сервису формируется URL, который выглядит следующим образом: <http://host/base/hs/корневойURL/относительныйURL>. Более подробно рассмотрим составные части адреса:

- [http://host/base](#) - обычный URL, по которому выполняется доступ, например, к информационной базе с помощью веб-клиента. При наличии разделителей, не поддерживается указание значений разделителей с помощью параметра [Z](#) командной строки запуска клиентского приложения.
- [hs](#) - признак того, что выполняется обращение к HTTP-сервису (в отличие от [ws](#), который определяет доступ к Web-сервису).
- [корневойURL](#) - имя ресурса, которое определяет группу ресурсов, связанных общим смыслом. Задается в свойстве объекта [HTTP-сервис](#).
- [относительныйURL](#) - определяет ресурс, к которому будет выполняться обращение. Относительный URL, указанный в запросе, будет использован для определения конкретного ресурса, к которому выполнялось обращение. Правило сопоставления задается в объекте [Шаблон URL](#).

17.3.2.2. Разработка HTTP-сервиса

Создание HTTP-сервисов выполняется в ветке [Общие - HTTP-сервисы](#) дерева объектов конфигурации. Вначале создается собственно сервис, затем в каждом сервисе необходимо создать определенный шаблон фрагмента URL (группу ресурсов), и затем для каждой группы ресурсов необходимо определить HTTP-метод, который будет реализован для данной группы. Также надо понимать, что один сервис может включать в себя более одного шаблона URL (отношение «один ко многим»). Фактически, шаблон URL определяет группу ресурсов, обрабатываемых по единым правилам.

17.3.2.2.1. Редактирование HTTP-сервиса

При создании HTTP-сервиса следует обратить внимание на свойство [Корневой URL](#). Определяет группу ресурсов, объединенных общим смыслом. Например, если необходимо создать несколько ресурсов, которые работают с заказами, то корневой URL в этом случае может выглядеть как [order](#). Тогда начало URL при обращении к такому сервису будет выглядеть следующим образом: <http://host/base/hs/order>.

17.3.2.2.2. Редактирование шаблона URL

При создании шаблона URL описываются возможные адреса ресурсов, которые можно будет использовать для обращения к HTTP-сервису. Собственно шаблон задается в свойстве [Шаблон](#). При создании шаблона можно использовать следующие символы:

- Любые символы, допустимые в идентификаторах языка «1С:Предприятие».
- Символ [«/»](#);
- Символы [«{»](#) с непустым текстом между ними;
- Символ [*](#).

Например, шаблон может выглядеть следующим образом:

```
/query  
/documents/{id}/props/{PropertyName}/*
```

[Копировать в буфер обмена](#)

При описании шаблона будут использоваться следующие термины:

- Сегмент - часть URL между двумя символами [«/»](#) Например, [props](#), из примера выше, является сегментом.
- Параметризованный сегмент - сегмент, заключенный в фигурные скобки ([«{сегмент}»](#)). Фактически, параметризованный сегмент описывает переменную с указанным именем, к которой можно получить доступ из встроенного языка.

Сегмент (не параметризованный) из шаблона должен быть перенесен в URL дословно. На месте параметризованного сегмента может располагаться любое значение, допустимое в URL. Символ [«*»](#) может находиться только в конце шаблона. На месте символа [«*»](#) может находиться любое количество сегментов, включая отсутствие сегмента. В одном шаблоне не может встречаться параметризованных сегментов с одинаковым именем. Имя параметризованного сегмента может состоять только из букв, цифр и символа [«_»](#).

Шаблон вида [/documents/*/number](#) не является допустимым. Если задан шаблон вида [/documents/{kind}/{number}](#), то являются допустимыми следующие URL сервиса:

```
/documents/trade/13  
/documents/12/test
```

[Копировать в буфер обмена](#)

При получении запроса система пытается определить, какой шаблон следует использовать. Для этого выполняется попытка сопоставить текст входящего запроса с заданными шаблонами. Сопоставление выполняется по следующим правилам:

1. Все строки сравниваются регистрозависимо.
2. Точное совпадение всегда более приоритетно, чем соответствие шаблону с параметризованными сегментами или символом «*».
3. Символ «*» сопоставляется с любым количеством сегментов (включая отсутствие сегмента).
4. Параметризованный сегмент имеет приоритет над символом «*».
5. Если переданному запросу не удалось однозначно сопоставить ни один шаблон - будет сгенерирована ошибка.

Данные правила поясняют следующие примеры:

1. При сопоставлении URL [/Накладная](#) с шаблонами [/Накладная/](#) и [/{EntityName}/](#) для использования будет выбран первый шаблон;
2. Шаблон [/Document/*/ {Num}](#) не является допустимым;
3. URL [/Накладная/500/Строки/2/Товар](#) может быть сопоставлен шаблону [/Накладная/*](#);
4. При сопоставлении URL [/Накладная/500](#) с шаблонами [/{Document}/{Num}](#) и [/{Document}/*](#), для использования будет выбран первый шаблон;
5. При сопоставлении URL [/Накладная/500/](#) шаблонам [/{Document}/{Num}](#) и [/{Document}/{Number}](#) произойдет ошибка, т. к. не удалось однозначно выбрать шаблон.

17.3.2.2.3. Редактирование метода

Для созданного шаблона необходимо определить HTTP-методы, которые могут быть использованы при работе с ресурсом, совпадающим с шаблоном. Для этого необходимо создать нужное количество объектов [Метод](#), подчиненных объекту [Шаблон URL](#). В каждом методе необходимо указать свойство [HTTP-метод](#) и создать обработчик. Имя обработчика автоматически формируется из имени шаблона URL и собственно имени метода. Так, если в шаблоне URL указано имя [ПримерДляДокументации](#), и для метода указано имя [GET](#), то для метода встроенного языка, с помощью которого будет обрабатываться HTTP-запрос, использовано имя обработчика [ПримерДляДокументацииGET\(\)](#).

СОВЕТ. Рекомендуется называть объекты [HTTP-метод](#) именем обслуживаемого метода на английском языке.

Имеется возможность создать обработчики для следующих методов:

- GET;
- POST;
- PUT;
- DELETE;
- PATCH;
- MERGE;
- CONNECT;
- OPTIONS;
- TRACE;
- PROPFIND;
- PROPPATCH;
- MKCOL;
- COPY;
- MOVE;
- LOCK;
- UNLOCK.

Если создается обработчик для метода [Любой](#), то обработчик этого метода будет вызван для всех HTTP-методов (из приведенного выше списка), которые явно не описаны в том шаблоне, который «обрабатывает» входящий HTTP-запрос.

После получения необходимого шаблона, будет предпринята попытка вызвать метод, обслуживающий нужный HTTP-запрос (GET, POST и т. д.). Если обработчик для необходимого запроса не был обнаружен, вызывающей стороне будет возвращена ошибка 405. Если в процессе исполнения метода обнаружена ошибка встроенного языка - вызывающей стороне будет возвращена ошибка 500, а в тело ответа будет помещен текст исключения.

При формировании ответа параметр [Content-Length](#) может быть не задан. В этом случае он будет определен автоматически.

17.3.2.2.4. Входные и выходные данные обработчика метода

Обработчиком того или иного HTTP-метода является функция, которая получает на вход один параметр типа `HTTPСервисЗапрос` и должна вернуть объект типа `HTTPСервисОтвет`.

Входной параметр полностью описывает запрос, поступивший в систему. С помощью объекта можно определить:

- Какой HTTP-метод используется в запросе (свойство `HTTPМетод`);
- На какой URL поступил запрос (свойство `БазовыйURL`) - часть полного URL запроса, не включающая относительный URL (см. [здесь](#)) и параметры запроса;
- Относительный URL (свойство `ОтносительныйURL`) - часть полного URL запроса, которая использовалась для определения группы ресурсов и правил обработки входящего HTTP-запроса;
- Какие заголовки (свойство `Заголовки`) содержит HTTP-запроса;
- Какие параметризованные сегменты (свойство `ПараметрыURL`) выделены из входящего HTTP-запроса и их значения.
- Какие параметры (свойство `ПараметрыЗапроса`) перечислены в URL HTTP-запроса после символа "?" и их значения.

Также существует возможность получать тело запроса в виде двоичных данных или строки (в зависимости от передаваемой информации).

Таким образом, прикладному разработчику не требуется самостоятельно «разбираться» с входящим запросом. С помощью примера, приведенного далее в этой главе (см. [здесь](#)), можно смотреть, каким образом входящий HTTP-запрос распределяется по свойствам объекта `HTTPСервисЗапрос`.

После того, как прикладное решение выполнило все действия, которые определены для данного ресурса и HTTP-метода, необходимо сформировать ответ сервиса. Для этого необходимо создать объект типа `HTTPСервисОтвет`. В зависимости от результата обработки, следует указать свойство `КодСостояния`, которое описывает стандартный код возврата HTTP (<https://datatracker.ietf.org/doc/html/rfc2616#section-10>, на английском языке). Затем, при необходимости, следует сформировать тело ответа HTTP-сервиса. Тело может быть установлено как двоичные данные, как строка или с помощью указания имени файла, откуда будет загружено тело ответа. Если тип возвращаемого функцией (которая реализует реакцию на HTTP-метод) значения отличается от `HTTPСервисОтвет`, то клиенту будет возвращена 500 ошибка.

17.3.2.3. Примеры реализации HTTP-сервисов

ПРИМЕЧАНИЕ. Примеры, приведенные ниже, не являются законченными. Они предназначены для демонстрации некоторых возможностей HTTP-сервисов.

17.3.2.3.1. Отображение входящего запроса

В качестве примера реализуем HTTP-сервис, который будет возвращать в качестве ответа все параметры входящего запроса.

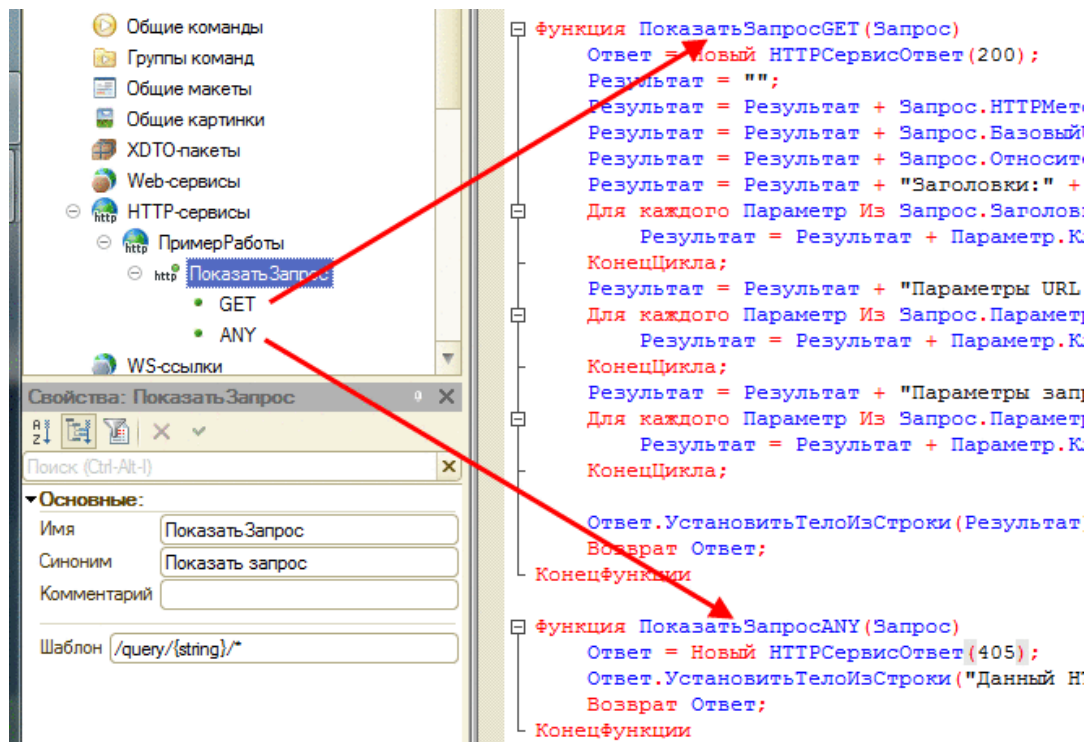


Рис. 508. Тестовый пример

Для этого нужно создать HTTP-сервис `ПримерРаботы`, указав для него корневой URL `example`. Затем следует для сервиса создать шаблон URL `ПоказатьЗапрос` с шаблоном `/query/{string}/*`. Теперь необходимо в шаблон `ПоказатьЗапрос` добавить следующие методы:

- Метод **GET** для HTTP-запроса **GET**;
- Метод **ANY** для HTTP-запроса **Любой**.

Обработчики должны выглядеть следующим образом:

Копировать в буфер обмена

```

Функция ПоказатьЗапросGET (Запрос)
    Ответ = Новый HTTPСервисОтвет(200);
    Результат = "";
    Результат = Результат + Запрос.HTTPМетод + Символы.ПС;
    Результат = Результат + Запрос.БазовыйURL + Символы.ПС;
    Результат = Результат + Запрос.ОтносительныйURL + Символы.ПС;
    Результат = Результат + "Заголовки:" + Символы.ПС;
    Для каждого Параметр Из Запрос.Заголовки Цикл
        Результат = Результат + Параметр.Ключ + ":" + Параметр.Значение + Символы.ПС;
    КонiecЦикла;
    Результат = Результат + "Параметры URL:" + Символы.ПС;
    Для каждого Параметр Из Запрос.ПараметрыURL Цикл
        Результат = Результат + Параметр.Ключ + ":" + Параметр.Значение + Символы.ПС;
    КонiecЦикла;
    Результат = Результат + "Параметры запроса:" + Символы.ПС;
    Для каждого Параметр Из Запрос.ПараметрыЗапроса Цикл
        Результат = Результат + Параметр.Ключ + ":" + Параметр.Значение + Символы.ПС;
    КонiecЦикла;

    Ответ.УстановитьТелоИзСтроки(Результат);
    Возврат Ответ;
КонiecФункции

Функция ПоказатьЗапросANY (Запрос)
    Ответ = Новый HTTPСервисОтвет(405);
    Ответ.УстановитьТелоИзСтроки("Данный HTTP сервис не поддерживает метод " + Запрос.HTTPМетод);
    Возврат Ответ;
КонiecФункции

```

После публикации HTTP-сервисов, [описанной в книге 1С:Предприятие 8.3. "Руководство администратора"](#), сервис должен возвращать полное «описание» поступившего на вход запроса.

При использовании следующего запроса: <http://localhost/httpservice/hs/example/query/value/another?p1=1&p2=2> будет получен следующий ответ (значение некоторых заголовков зависит от используемого веб-браузера):

Копировать в буфер обмена

```

GET
http://localhost/httpservice/hs/example
/query/value/another
Заголовки:
Accept-Encoding: gzip, deflate, sdch
Connection: keep alive
Accept-Language: ru RU, ru; q=0.8, en US; q=0.6, en; q=0.4
DNT: 1
Accept: text/html, application/xhtml+xml, application/xml; q=0.9, image/webp, */*; q=0.8
Content-Length: 0
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.107
Safari/537.36
Host: localhost
Параметры URL:
*/another
string: value
Параметры запроса:
p1: 1
p2: 2

```

Рассмотрим, каким образом был разобран запрос. В примере было указано, что корневой URL сервиса - **example**, шаблон URL - **/query/{string}/*** и был сформирован обработчик HTTP-запроса GET.

В результате видно, что система определила следующие параметры URL:

- Параметр ***** со значением **/another**;
- Параметр **string** со значением **value**.

Это полностью соответствует указанному шаблону URL. Если шаблон URL установить в значение **/query/{string}**, то демонстрационный запрос закончится сообщением об ошибке 404, т. к. такой шаблон не будет допускать наличие в URL информации, после параметризованного сегмента.

Также в обработчике метода будут переданы все параметры запроса, которые были указаны в URL.

Любой другой вид запроса (отличный от GET) будет приводить к ошибке 405 с указанием, какой именно запрос не поддерживается сервисом.

17.3.2.3.2. Работа с документами

Данный пример является некоторым аналогом Web-сервиса, который должен по переданному номеру расходной накладной возвращать состав ее табличной части (см. [здесь](#)).

Данный пример будет предоставлять возможность получить список «документов» и получить «документ» по номеру. Также будет реализована реакция на неверный номер «документа». Для упрощения примера реальных объектов типа *Документ* в примере создано не будет.

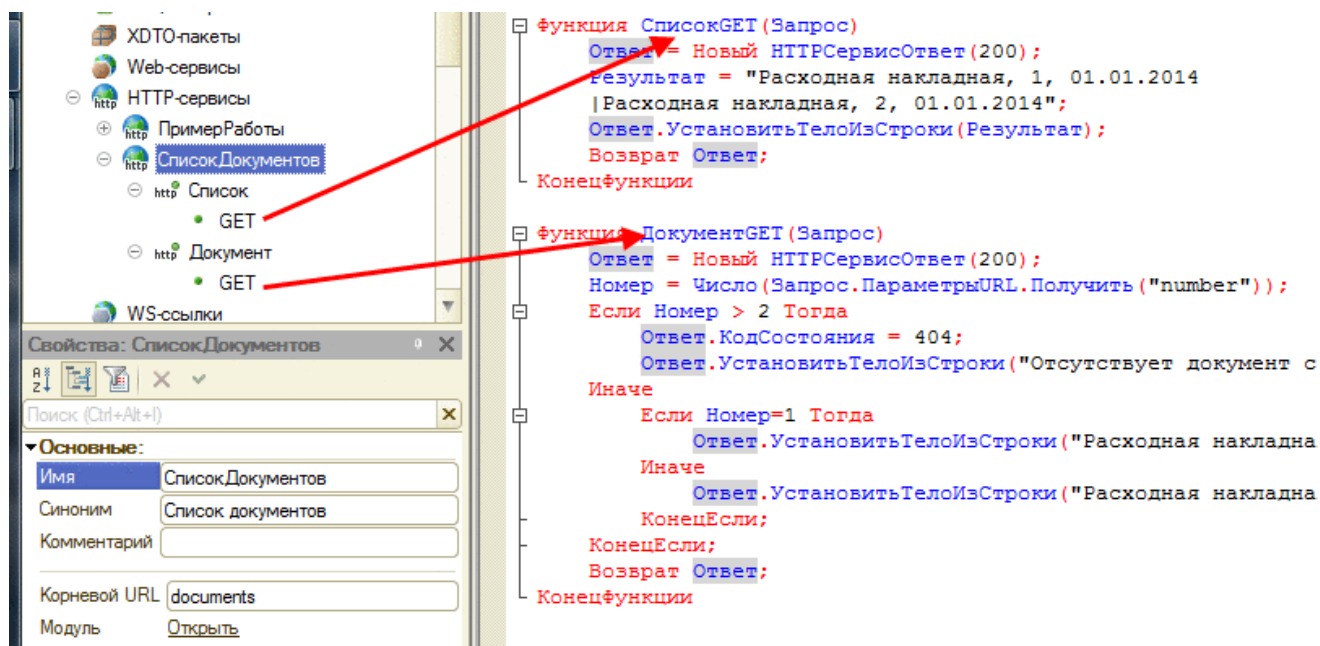


Рис. 509. Тестовый пример

Для этого нужно создать HTTP-сервис *СписокДокументов*, указав для него корневой URL *documents*. Затем следует для сервиса создать шаблон URL *Список* с шаблоном */** и шаблон URL *Документ* с шаблоном */{number}*. Для каждого шаблона URL следует добавить реализацию метода GET.

Обработчики должны выглядеть следующим образом:

```
функция СписокGET (Запрос)
    Ответ = Новый HTTPСервисОтвет(200);
    Результат = "Расходная накладная, 1, 01.01.2014
|Расходная накладная, 2, 01.01.2014";
    Ответ.УстановитьТелоИзСтроки(Результат);
    Возврат Ответ;
Конецфункции

функция ДокументGET (Запрос)
    Ответ = Новый HTTPСервисОтвет(200);
    Номер = Число(Запрос.ПараметрыURL.Получить("number"));
    Если Номер > 2 Тогда
        Ответ.КодСостояния = 404;
        Ответ.УстановитьТелоИзСтроки("Отсутствует документ с номером: " + Номер);
    Иначе
        Если Номер=1 Тогда
            Ответ.УстановитьТелоИзСтроки("Расходная накладная, 1, 01.01.2014");
        Иначе
            Ответ.УстановитьТелоИзСтроки("Расходная накладная, 2, 01.01.2014");
        КонецЕсли;
    КонецЕсли;
    Возврат Ответ;
Конецфункции
```

После публикации HTTP-сервисов, [описанной в книге 1С:Предприятие 8.3. "Руководство администратора"](#), сервис должен возвращать некоторый набор данных о документах, «хранящихся» в системе.

Рассмотрим, каким образом работает созданный сервис:

- Запрос <http://localhost/httpservice/hs/documents> вернет список «документов»:

```
Расходная накладная, 1, 01.01.2014
Расходная накладная, 2, 01.01.2014
```

- Запрос вида <http://localhost/httpservice/hs/documents/get/list> вернет список «документов», т. к. этот запрос соответствует шаблону *Список* и не соответствует шаблону *Документ*.
- Запрос вида <http://localhost/httpservice/hs/documents/list> вернет ошибку времени исполнения, т. к. приведенный запрос соответствует шаблону *Документ*, но в функции *ДокументGET()* нет проверки на то, что переданный номер является числом.

- Запрос вида <http://localhost/httpservice/hs/documents/2> вернет «документ» с номером 2:

[Копировать в буфер обмена](#)

Расходная накладная, 2, 01.01.2014

- Запрос вида <http://localhost/httpservice/hs/documents/3> вернет ошибку:

[Копировать в буфер обмена](#)

Отсутствует документ с номером: 3

Обработка корректности номера документа явно реализована в обработчике метода.

17.3.2.4. Установка значений разделителей

Значения разделителей можно устанавливать только следующими способами:

- С помощью фрагментов URL при обращении к REST-сервису;
- С помощью файла описания публикации [default.vrd](#).

Указанием разделителей с помощью параметры командной строки **Z** не поддерживается.

17.3.2.5. Публикация HTTP-сервиса

Публикация HTTP-сервисов выполняется с помощью диалога публикации на веб-сервере ([Администрирование - Публикация на веб-сервере](#)) и [описана в книге 1С:Предприятие 8.3. "Руководство администратора"](#).

17.4. Стандартный интерфейс OData

17.4.1. Общая информация

Стандартный интерфейс OData в «1С:Предприятии» предназначен для получения доступа к данным системы из внешнего приложения без модификации кода прикладного решения (например, если прикладное решение стоит на поддержке). Для получения такого доступа необходимо особым образом опубликовать приложение на веб-сервере и указать, какие объекты конфигурации будут использоваться таким образом (см. [здесь](#)).

Для доступа к данным используется протокол OData (<http://www.odata.org/>, на английском языке) версии 3 (<http://www.odata.org/documentation/odata-version-3-0/odata-version-3-0-core-protocol>, на английском языке). Поддерживаются следующие форматы представления данных: **atom+xml** и **json**. Для доступа к данным, при публикации, автоматически генерируется стандартный интерфейс OData, который позволяет читать данные «1С:Предприятия», изменять их, создавать новые объекты данных и удалять существующие. Прикладное решение на базе «1С:Предприятия» может выступать как клиентом, так и сервером при работе со стандартным интерфейсом OData. Для работы сервером практически никаких дополнительных действий осуществлять не надо (эта возможность предоставляется автоматически). Для того, чтобы стать клиентом стандартного интерфейса OData, необходимо в прикладном решении реализовать программный слой, который будет использовать данные сервера с использованием стандартных интерфейсов «1С:Предприятия», например объекта **HTTPСоединение**.

Стандартный интерфейс OData можно использовать для следующих задач:

- Интеграция прикладного решения с различными сайтами (например, на базе Microsoft SharePoint);
- Реализация сторонними средствами дополнительной функциональности прикладного решения без изменения его конфигурации;
- Загрузка данных в прикладное решение и выгрузка данных из него;
- Интеграция прикладного решения с корпоративными системами, возможно даже без дополнительного программирования.

Типичные операции, выполняемые с помощью стандартного интерфейса OData:

- Получение списка объектов системы с установленным отбором;
- Получение данных конкретного объекта системы;
- Создание нового и запись изменений существующего объекта системы;
- Проведение одного документа, старт бизнес-процесса.

17.4.2. Общие принципы работы

При работе с протоколом OData используется специальная терминология. При применении протокола используются следующие термины:

- **Сущность** - нечто, обладающее идентичностью. Сущность также обладает набором свойств. Некоторые свойства описывают ее идентичность, комбинация которых определяет ключ сущности. По этому ключу можно получить конкретную сущность.
- **Набор сущностей** - коллекция сущностей определенного типа.
- **Составной тип** - набор свойств, не обладающий идентичностью.
- **Функция** - набор некоторых операций, выполняемых на стороне сервера, возвращающий данные (не обязательно сущность или набор сущностей) и не приводящий к наблюдаемым побочным эффектам (изменениям данных). Функция

обязательно связана с сущностью или набором сущностей.

- **Действие** - функция, которая может изменять данные.

Термины протокола некоторым образом соответствуют терминам, принятым в системе «1С:Предприятие»:

- **Сущность** - в качестве сущности может выступать один из 4 групп объектов системы «1С:Предприятие»: объектные типы, наборы записей регистров, записи регистров и строки табличных частей объектных типов. Записи регистров и строки табличных объектных типов (как отдельные сущности) доступны только на чтение.

- Реквизиты объектов «1С:Предприятия» представляются как **свойства** сущностей. В некоторых случаях (например, реквизит объекта конфигурации составного типа) реквизит может быть представлен несколькими свойствами, одно из которых будет **навигационным**. Такое свойство содержит в качестве значения ссылку (URL) на сущность, описывающую объект «1С:Предприятия». Свойство, описывающее тип такого реквизита, называется **диспетчеризационным**. Название такого свойства завершается суффиксом **_Type**. Более подробное описание диспетчеризационного свойства приведено далее.

- Если реквизит объекта имеет ссылочный тип, то объект, на который ссылается такой реквизит, будет называться **связанной сущностью**.

Обращение к стандартному интерфейсу OData выполняется с помощью HTTP-запроса, выполнено по определенному URL. URL формируется специальным образом и состоит из следующих частей:

1. Адрес информационной базы;
2. Признак обращения к стандартному интерфейсу OData;
3. Имя ресурса, к которому выполняется обращение;
4. Параметры запроса обращения к ресурсу.

Само обращение выполняется с помощью HTTP-запроса определенного вида. При более подробном описании работы со стандартным интерфейсом OData вид запроса будет указываться отдельно.

Рассмотрим части URL более подробно:

Адрес информационной базы

Это обычный URL, по которому выполняется доступ, например, к информационной базе с помощью веб-клиента. Например, <http://host/base> или <http://host.server.zone/data-base>. Также необходимо помнить, что при использовании информационной базы, в которой настроено разделение, допустимо использовать только один способ указания значений разделителей, а именно: значения разделителей можно указывать только в URL информационной базы. Указание разделителей с помощью параметра **Z** не поддерживается. Более подробную информацию про настройку указания значений разделителей в URL информационной базы можно получить в описании файла [default.vrd](#) в книге [1С:Предприятие 8.3. "Руководство администратора"](#).

Признак обращения к стандартному интерфейсу OData

В качестве такого признака выступает последовательность [/odata/standard.odata](#).

Имя ресурса, к которому выполняется обращение

Особым образом сформированный идентификатор ресурса (возможно, с параметром) или предопределенные ресурсы. Например, [\\$metadata](#), [Catalog_Контрагент\(guid'value'\)](#).

Параметры обращения к ресурсу

В качестве параметров обращения выступают параметры в виде, принятом для HTTP-запросов: [?ключ=значение&ключ2=значение2](#).

При обращении к ресурсу могут использоваться специальные ключевые слова, имеющие специальное назначение:

- **\$format** - указывает, в каком формате необходимо получить данные. Если ключевое слово не указано, данные получаются в формате atom+xml.
- **\$format=atom** - возвращает данные в формате **atom+xml**.
- **\$format=json** - возвращает данные в формате **json**. Для указания того, что данные должны возвращаться в формате **json**, можно указать MIME-тип application/json в заголовке **Accept** HTTP-запроса на получение данных.
- **\$metadata** - указывает, что требуется получить описание стандартного интерфейса OData. Подробнее см. [здесь](#).
- **\$filter** - описывает отбор, применяемый при получении данных. Подробнее см. [здесь](#).
- **\$select** - описывает перечень свойств сущности, которые получаются при обращении к стандартному интерфейсу OData. Подробнее см. [здесь](#).

После того, как сформирован URL необходимого ресурса, следует выполнить HTTP-запрос нужного вида. В зависимости от того, какая операция выполняется, используется соответствующий HTTP-метод:

- Получение данных - метод **GET**;
- Создание объекта - метод **POST**;

- Обновление данных:
 - метод `PATCH` - в этом случае можно указывать только те свойств, которые необходимо обновить;
 - метод `PUT` - в этом случае необходимо указывать все свойства сущности;
- Удаление данных - метод `DELETE`.

В результате выполнения запроса клиентское приложение получает ответ сервера, который кроме кода состояния может содержать различные данные, предоставленные сервером, в виде XML-документа.

При обращении к стандартному интерфейсу OData следует учитывать, что для указания значений некоторых типов необходимо соблюдать специальную нотацию:

- Тип `УникальныйИдентификатор` (тип `Edm.Guid`) указывается как `guid'value'`. Например: `guid'8d04ce3c-6360-11e9-dd92-fa163e8ba11d'`.
- Тип `Дата` (тип `Edm.DateTime`) указывается как `datetime'yyyy-mm-ddThh:mm:ss'`. Например: `datetime'1945-05-09T06:00:00'`.

17.4.3. Представление данных

Данные, возвращаемые стандартным интерфейсом OData, могут быть представлены в виде XML-документа или JSON-документа. Это зависит от того, в каком формате запрашиваются данные.

Для различных типов данных используется следующее соответствие между типом «1С:Предприятия» и типом OData:

Тип «1С:Предприятия»	Тип свойства OData
Строка	<code>Edm.String</code>
Дата	<code>Edm.DateTime</code>
Число (целое)	<code>Edm.Int16</code> , <code>Edm.Int32</code> , <code>Edm.Int64</code>
Число (дробное)	<code>Edm.Double</code>
Булево	<code>Edm.Boolean</code>
Ссылка, УникальныйИдентификатор	<code>Edm.Guid</code>
Перечисление	<code>Edm.String</code>
ХранилищеЗначения	Состоит из трех свойств*: 1. Навигационное свойство; 2. <code><Имя свойства>_Base64Data</code>; 3. <code><Имя свойства>_Type</code>.
Ссылочный тип	Два свойства: • Собственно значение ссылки на объект типа <code>Edm.Guid</code>; • Навигационное свойство.
Составные типы	Два свойства типа <code>Edm.String</code> : • Значение фактического типа, выраженное в виде строки; • Имя фактического типа.

В таблице символ * означает, что:

- С помощью навигационного свойства можно получить данные, хранящиеся в реквизите:
 - Для типа `Картинка` - будет получена собственно картинка с соответствующим значением HTTP-заголовка `content-type`;
 - Для типа `ДвоичныеДанные` - будет получен байтовый поток;
 - Для остальных типов - XDTO-сериализованное значение хранимых данных.
- Свойство `<Имя свойства>_Base64Data` хранит данные, которые могут быть получены с помощью навигационного свойства, только закодированные в `Base64`. Редактировать данные реквизита типа `ХранилищеЗначения` можно только с помощью этого свойства.
- Свойство `<Имя свойства>_Type` описывает тип данных, хранимых в реквизите. Может принимать одно и трех значений:
 - `application/octet-stream` - двоичные данные;
 - `application/xml+xdto` - XDTO-сериализованный объект;
 - значение HTTP-заголовка `content-type`, соответствующего картинке, хранящейся в реквизите, например, `image/jpeg` для картинки формата JPEG.

Остальные типы не поддерживаются, и при попытке их чтения будет сгенерирована ошибка с кодом 501.

Имена свойств могут оканчиваться на различные суффиксы. Такие свойства имеют специальный смысл. Далее будут более подробно рассмотрены возможные суффиксы:

- [Key](#);
- [Type](#);
- [Base64Data](#);
- [____Presentation](#).

[Key](#)

Свойство с таким суффиксом содержит значение ключа соответствующего ссылочного реквизита объекта (без такого суффикса) или независимого регистра сведений без измерений. Имя с таким суффиксом используется для установки отбора в качестве имени реквизита, по которому выполняется отбор. Например, для установки отбора по ссылочному полю [Контрагент](#), условие будет выглядеть следующим образом: [Контрагент_Key=guid'value'](#). При этом свойство [Контрагент](#) будет содержать представление контрагента с указанным значением ссылки.

[Type](#)

Данный суффикс используется для описания реквизита составного типа. Так, если в данных есть поле составного типа [Контрагент](#), то в документе, который возвращает стандартный интерфейс OData, этому полю будет соответствовать два свойства:

- [Контрагент_Type](#) - будет содержать описание типа значения реквизита в виде строки (тип [Edm.String](#), диспетчеризационное свойство);
- [Контрагент](#) - будет содержать значение реквизита (соответствующего типа).

Перечень допустимых типов, которые могут быть использованы в поле с таким суффиксом, определяется схемой сервиса, который можно получить при запросе полного описания стандартного интерфейса OData (см. [здесь](#)). Таким образом, при необходимости установить тип [Документ.РасходТовара](#), в элемент с суффиксом [_Type](#) должно быть записано значение [StandardODATA.Document_РасходТовара](#).

Если значение реквизита составного типа в информационной базе «1С:Предприятия» имеет значение [Неопределено](#), то диспетчеризационное свойство будет иметь значение [StandardODATA.Undefined](#), а само значение свойства должно игнорироваться.

Пример представления реквизита составного типа:

```
<d:РеквизитСоставной/>
<d:РеквизитСоставной_Type>StandardODATA.Undefined</d:РеквизитСоставной_Type>
```

[Копировать в буфер обмена](#)

[Base64Data](#)

Данный суффикс используется при указании имени свойства, содержащего данные, расположенные в реквизите типа [ХранилищеЗначения](#), в виде строки [Base64](#). Так, если в объекте конфигурации есть реквизит [Файл](#), который имеет тип [ХранилищеЗначения](#), то в документе, который возвращает стандартный интерфейс OData, этому полю будет соответствовать два свойства:

- [Файл_Type](#) - содержит наименование типа данных, хранимых реквизитом;
- [Файл_Base64Data](#) - содержит строку [Base64](#), содержащую сами данные.

[Presentation](#)

Свойство [<Имя свойства>____Presentation](#) содержит представление реквизита (между именем свойства и [Presentation](#) находятся 4 символа «_»). Значение поля эквивалентно значению, которое является результатом работы функции языка запросов [Представление\(\)](#).

Поля представлений не включаются в результаты запроса с пустым параметром [\\$select](#) и при указании в параметре [\\$select](#) значений [*](#) и [**](#). Таким образом, если необходимо получить все реквизиты объекта и представления некоторых реквизитов, необходимо указать значение [*](#), а затем отдельно перечислить поля представлений требуемых реквизитов, разделяя их запятыми: [\\$select=*, РеквизитСсылка____Presentation](#).

Для получения представлений всех реквизитов объекта, следует в параметре [\\$select](#) указать значение [*____Presentation: \\$select=*____Presentation](#).

Для получения значений всех полей и представлений для всех полей, следует использовать следующее выражение: [\\$select=*, *____Presentation](#).

Для получения представления текущей сущности имеется возможность указывать в списке запрашиваемых полей поле [Presentation: \\$select=Ref_Key, Presentation](#). В этом случае будет получено значение ссылки и представление этой ссылки.

Поля представлений не содержатся в описании метаданных, которое возвращается в описании стандартного интерфейса OData (параметр [\\$metadata](#)). Получение представления не поддерживается для табличных частей и реквизитов типа [ХранилищеЗначения](#). Не поддерживается модификация значений для полей представлений.

17.4.4. Правила формирования имени ресурса

При обращении к какому-либо ресурсу, его идентификатор формируется по следующему принципу: **ПрефиксИмени_ИмяОбъектаКонфигурации_СуффиксИмени**. С помощью стандартного интерфейса OData можно получить доступ к следующим объектам (**ПрефиксИмени**):

Объект конфигурации	Префикс имени для указания в URL
Справочник	Catalog
Документ	Document
Журнал документов	DocumentJournal
Константа	Constant
План обмена	ExchangePlan
План счетов	ChartOfAccounts
План видов расчета	ChartOfCalculationTypes
План видов характеристик	ChartOfCharacteristicTypes
Регистр сведений	InformationRegister
Регистр накопления	AccumulationRegister
Регистр расчета	CalculationRegister
Регистр бухгалтерии	AccountingRegister
Бизнес-процесс	BusinessProcess
Задача	Task

ИмяОбъектаКонфигурации - свойство **Имя** объекта конфигурации, как оно задано при разработке прикладного решения в конфигураторе.

СуффиксИмени - предназначено для уточнения имени ресурса и является необязательной частью имени. В качестве суффикса имени могут выступать следующие выражения:

- Имя табличной части объекта;
- Имя реквизита табличной части или набора записей;
- Имя виртуальной таблицы регистра;
- **RowType**;
- **RecordType**.

Далее будут более подробно рассмотрены вышеописанные уточнения имени ресурса:

Имя табличной части объекта

Если объект обладает табличной частью, то для получения доступа ко всем записям этой табличной части необходимо добавить имя табличной части после имени самого объекта, например, для получения всех строк табличных частей **Товары** всех документов **РасходТовара** будет необходимо выполнить GET-запрос по следующему адресу: http://host/base/odata/standard.odata/Document_РасходТовара_Товары.

Имя реквизита табличной части или набора записей

Если объект обладает табличной частью, то имеется возможность указать, что требуется получение не всех реквизитов табличной части, а некоторого списка этих реквизитов. Для этого необходимо указать в параметре **\$select** список требуемых реквизитов в следующем виде: **<Имя табличной части>/<Имя поля>**. Аналогичная возможность предоставляется для наборов записей регистров, где в качестве имени табличной части выступает **RecordSet**: **RecordSet/<Имя поля>**.

Имя виртуальной таблицы регистра

В роли виртуальной таблицы регистра выступает функция, связанная с ресурсом, возвращающей набор сущностей регистра. Имя функции совпадает с английским вариантом имени используемой виртуальной таблицы языка запросов. Параметры функции соответствуют параметрам виртуальной таблицы. Так, для получения среза последних регистра сведений **КурсыВалют**, следует выполнить GET-запрос по следующему адресу: [http://localhost/demo/odata/standard.odata/InformationRegister_КурсыВалют/SliceLast\(\)](http://localhost/demo/odata/standard.odata/InformationRegister_КурсыВалют/SliceLast()).

RowType

Сущность с таким суффиксом описывает тип строки табличной части какого-либо объекта.

RecordType

Сущность с таким суффиксом описывает отдельную запись регистра.

17.4.5. Правила формирования условия отбора

17.4.5.1. Общая информация

В данном разделе приводится информация по различным способам формирования отбора получаемых данных, используемых в стандартном интерфейсе OData системы «1С:Предприятие».

Описание возможностей отбора в протоколе OData можно получить в документации по протоколу (на английском языке):

- <http://www.odata.org/documentation/odata-version-3-0/odata-version-3-0-core-protocol/> раздел 10.2.3.1;
- <http://www.odata.org/documentation/odata-version-3-0/odata-version-3-0-core-protocol/> раздел 5.1.2.

17.4.5.2. \$filter

При получении данных можно фильтровать данные. Для этого предназначен специальный язык, который позволяет описывать условия, каким должны соответствовать данные, которые возвращает стандартный интерфейс OData. Описание отбора начинается с ключевого слова **\$filter**, после которого следует собственно условие. Поддерживаются следующие операции:

- Логические операции:

Описание	Имя	Пример
Равно	eq	/Catalog_Города?\$filter=Description eq 'Главный'
Не равно	ne	/Catalog_Города?\$filter=Description ne 'Пермь'
Больше	gt	/Catalog_Товары?\$filter=Цена gt 10
Больше или равно	ge	/Catalog_Товары?\$filter=Цена ge 10
Меньше	lt	/Catalog_Товары?\$filter=Цена lt 10
Меньше или равно	le	/Catalog_Товары?\$filter=Цена le 10
Логическое Или	or	/Catalog_Товары?\$filter=Цена lt 10 or Цена gt 100
Логическое И	and	/Catalog_Товары?\$filter=Цена gt 10 and Цена lt 100
Отрицание	not	/Catalog_Товары?\$filter=not (Цена eq 10)

- Арифметические операции:

Описание	Имя	Пример
Сложение	add	/Catalog_Товары?\$filter=Цена add 5 gt 10
Вычитание	sub	/Catalog_Товары?\$filter=Цена sub 5 gt 10
Умножение	mul	/Catalog_Товары?\$filter=Цена mul 5 gt 1000
Деление	div	/Catalog_Товары?\$filter=Цена div 4 gt 2

- Группирующие операторы:

Описание	Имя	Пример
Приоритет операции	()	/Catalog_Товары?\$filter=(Цена add 5) gt 10

Пример отбора:

[http://host/odata/standard.odata/Catalog_Товары?\\$filter=Имя eq 'Молоко' and Цена lt 2500](http://host/odata/standard.odata/Catalog_Товары?$filter=Имя eq 'Молоко' and Цена lt 2500) [Копировать в буфер обмена](#)

При формировании условия отбора следует учитывать приоритет операций. В следующей таблице представлен список операций языка выражений отбора в порядке уменьшения приоритета. Операции с одинаковым приоритетом вычисляются слева направо:

Оператор	Описание
()	Повышение приоритета операции
/	Навигация
-	Арифметическое отрицание
Not	Логическое отрицание
Mul	Умножение
Div	Деление
Add	Сложение
Sub	Вычитание
Gt	Больше
Ge	Больше или равно

Lt	Меньше
Le	Меньше или равно
Eq	Равно
Ne	Не равно
And	Логическое «И»
Or	Логическое «ИЛИ»

При формировании условий запроса (параметр **filter**) или формировании реквизита, по которому выполняется упорядочивание (параметр **orderby**) могут применяться следующие функции:

- Строковые функции:

Функция	Описание	Пример
<code>concat(Str1, Str2)</code>	Возвращает строку, являющуюся результатом конкатенации <code>Str1</code> и <code>Str2</code> .	<code>/Catalog_Поставщик? \$filter=concat(concat(Город, ' '), Страна) eq 'Москва, Россия'</code>
<code>endswith(Str1, Str2)</code>	Возвращает <code>true</code> в том случае, если <code>Str1</code> заканчивается на <code>Str2</code> .	<code>/Catalog_Товары? \$filter=endswith(Производитель, 'ООО') eq true</code>
<code>like(Str, Template)</code>	Возвращает <code>true</code> , если значение <code>Str1</code> удовлетворяет шаблону <code>Template</code> . Синтаксис шаблона аналогичен функции <code>ПОДОБНО()</code> языка запросов (см. здесь).	<code>/Catalog_Товары? \$filter=like(Наименование, '[^к]%)'</code>
<code>startswith(Str1, Str2)</code>	Возвращает <code>true</code> в том случае, если <code>Str1</code> начинается на <code>Str2</code> .	<code>/Catalog_Товары? \$filter=startswith(Производитель, 'ООО') eq true</code>
<code>substring(Str, Int1)</code>	Возвращает подстроку из <code>Str1</code> . В варианте с двумя параметрами возвращается строка с позиции <code>Int</code> и до конца строки.	<code>/Catalog_Поставщики? \$filter=substring(ИНН, 1, 2) eq '77'</code>
<code>substring(Str, Int1, Int2)</code>	В варианте с тремя параметрами возвращается подстрока, начиная с позиции <code>Int1</code> и длиной <code>Int2</code> .	
<code>substringof(Str1, Str2)</code>	Возвращает <code>true</code> в том случае, если <code>Str1</code> является подстрокой <code>Str2</code> .	<code>/Catalog_Товары? \$filter=substringof('Красный Октябрь', Производитель) eq true</code>

- Функции работы с датами:

Функция	Описание	Пример
<code>dateadd(DateTime1, Type, Int1)</code>	Возвращает дату, полученную добавлением к значению <code>DateTime1</code> значения <code>Int1</code> , выраженное в единицах <code>Type</code> : <code>second</code> - секунды; <code>minute</code> - минуты; <code>hour</code> - часы; <code>day</code> - дни; <code>month</code> - месяцы; <code>quarter</code> - кварталы; <code>year</code> - года.	<code>/Catalog_Товары? \$filter=dateadd(Произведен, 'month', 1) eq ГоденДо</code>
<code>datedifference(DateTime1, DateTime2, Type)</code>	Возвращает разность дат <code>DateTime2</code> и <code>DateTime1</code> в единицах, указанных параметром <code>Type</code> : <code>second</code> - секунды; <code>minute</code> - минуты; <code>hour</code> - часы; <code>day</code> - дни; <code>month</code> - месяцы; <code>quarter</code> - кварталы; <code>year</code> - года.	<code>/Catalog_Товары? \$filter=datedifference(Произведен, ГоденДо, 'day') gt 10</code>
<code>day(DateTime)</code>	Возвращает день из значения типа <code>Edm.DateTime</code> или	<code>/Catalog_Товары?</code>

	<code>Edm.DateTimeOffset.</code>	<code>\$filter=day(Произведен) eq 1</code>
<code>dayofweek(DateTime)</code>	Возвращает день недели по значению типа <code>Edm.DateTime</code> .	<code>/Catalog_Товары?</code> <code>\$filter=dayofweek(ДатаПроизводства)</code> <code>eq 7</code>
<code>dayofyear(DateTime)</code>	Возвращает день года по значению <code>Edm.DateTime</code> .	<code>/Catalog_Товары?</code> <code>\$filter=dayofyear(ДатаПроизводства)</code> <code>eq 1</code>
<code>hour(DateTime)</code>	Возвращает значение часов из значения типа <code>Edm.DateTime</code> или <code>Edm.DateTimeOffset</code> .	<code>/Catalog_Товары?</code> <code>\$filter=hour(Произведен) eq 23</code>
<code>minute(DateTime)</code>	Возвращает значение минут из значения типа <code>Edm.DateTime</code> или <code>Edm.DateTimeOffset</code> .	<code>/Catalog_Товары?</code> <code>\$filter=minute(Произведен) eq 59</code>
<code>month(DateTime)</code>	Возвращает месяц из значения типа <code>Edm.DateTime</code> или <code>Edm.DateTimeOffset</code> .	<code>/Catalog_Товары?</code> <code>\$filter=month(Произведен) eq 12</code>
<code>quarter(DateTime)</code>	Номер квартала года, в котором находится указанное значение типа <code>Edm.DateTime</code> .	<code>/Catalog_Товары?</code> <code>\$filter=quarter(ДатаПроизводства) eq 1</code>
<code>second(DateTime)</code>	Возвращает значение секунд из значения типа <code>Edm.DateTime</code> или <code>Edm.DateTimeOffset</code> .	<code>/Catalog_Товары?</code> <code>\$filter=second(Произведен) eq 59</code>
<code>year(DateTime)</code>	Возвращает год из значения типа <code>Edm.DateTime</code> или <code>Edm.DateTimeOffset</code> .	<code>/Catalog_Товары?</code> <code>\$filter=year(Произведен) eq 2013</code>

• Прочие функции:

Функция	Описание	Пример
<code>cast(expr, type)</code>	Возвращает объект, на который указывает параметр <code>expr</code> , приведенный к типу, указанному параметром <code>type</code> . В качестве типа значения принимается строка, обозначающая имя данного типа, например: <code>String</code> , <code>Number</code> , <code>Boolean</code> , <code>Date</code> , <code>Catalog_Товары</code> и т. д.	<code>/Catalog_Товары?</code> <code>\$filter=</code> <code>cast(РеквизитСоставной,</code> <code>'Number') le 12</code>
<code>isof(expr, type)</code>	Возвращает <code>true</code> в том случае, когда объект, на который указывает параметр <code>expr</code> , имеет тип, на который ссылается <code>type</code> . В качестве типа значения принимается строка, обозначающая имя данного типа, например: <code>String</code> , <code>Number</code> , <code>Boolean</code> , <code>Date</code> , <code>Catalog_Товары</code> и т. д.	<code>/Catalog_Товары?</code> <code>\$filter=isof(Цена,</code> <code>'Number')</code>
<code>round(Number)</code>	Возвращает параметр, округленный до ближайшего целого числа.	<code>/Catalog_Товары?</code> <code>\$filter=round(Вес) gt 31</code>

ПРИМЕЧАНИЕ. Не поддерживаются операции сравнения с реквизитом типа `ХранилищеЗначения`.

Имеется возможность выполнять отбор сущностей при помощи проверки на равенство поля составного типа и ссылки. Для этого следует использовать функцию `cast()`.

Пример:

Копировать в буфер обмена

```
$filter=ДокументПрихода eq cast(guid'0d4a79cb-9843-4147-bcd9-80ac3ca2b9c7', 'Document_ПриходнаяНакладная')
```

В данном примере у используемой сущности имеется реквизит составного типа `ДокументПрихода`. Запрос будет отбирать все записи сущности, у которой данный реквизит заполнен ссылкой на документ `ПриходнаяНакладная` с указанным уникальным идентификатором (`0d4a79cb-9843-4147-bcd9-80ac3ca2b9c7`).

Если требуется выполнить отбор по реквизиту типа `УникальныйИдентификатор`, то это выполняется с помощью простой операции сравнения:

Копировать в буфер обмена

```
$filter=ИмяРеквизита eq guid'0d4a79cb-9843-4147-bcd9-80ac3ca2b9c7'
```

Для отбора по типу какого-либо реквизита составного типа следует использовать функцию `isof()`:

Копировать в буфер обмена

```
$filter=isof(ДокументыПрихода, 'Document_ПриходнаяНакладная')
```

Следующее выражение всегда вернет пустой результат запроса, вне зависимости от физического наличия используемого уникального идентификатора в соответствующей колонке сущности:

Копировать в буфер обмена

```
$filter=ДокументыПрихода eq guid'0d4a79cb-9843-4147-bcd9-80ac3ca2b9c7'
```

При необходимости выполнить отбор по элементу коллекции, необходимо использовать лямбда-функции. В качестве аргументов лямбда-функции выступает имя лямбда-переменной, за которой следует символ двоеточия и логическое выражение, которое использует лямбда-переменную для указания на свойства элемента коллекции. Система поддерживает использование следующих лямбда-функций:

• `any` - применяет логическое выражение к каждому элементу коллекции и возвращает значение `true`, если хоть один элемент коллекции удовлетворяет этому условию. Лямбда-функция `any` без аргументов возвращает `true`, если коллекция не

пуста.

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Document_Продажи?$filter=Товары/any(d: d/Цена gt 10000)`

В приведенном примере формируется список документов **Продажи**, у которых есть табличная часть **Товары**, в которой есть реквизит **Цена**. При этом **Цена** должна быть больше, чем **10000**. В результирующий список попадут документы, в состав которых входит хотя бы одна строка, удовлетворяющая условию.

- **all** - применяет логическое выражение к каждому элементу коллекции и возвращает значение **true**, если все элементы коллекции ему удовлетворяют.

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Document_Продажи?$filter=Товары/all(d: d/Цена lt 10000)`

В приведенном примере формируется список документов **Продажи**, у которых есть табличная часть **Товары**, в которой есть реквизит **Цена**. При этом **Цена** должна быть меньше, чем **10 000**. В результирующий список попадут документы, в составе которых все строки удовлетворяют заданному условию.

При необходимости указать условие на значение реквизита другого реквизита составного типа, содержащего ссылочные значения, таким образом, чтобы проверялось одновременно и значение проверяемого реквизита и тип составного реквизита, требуется использовать функцию **cast()**.

Например, имеется документ **МаршрутныйЛист**, который содержит реквизит составного типа **ОснованиеОтгрузки**. Данный реквизит может принимать значения типа **ДокументСсылка.Накладная** и **ДокументСсылка.ВнутреннееПеремещение**. При этом у документа **Накладная** существует реквизит **МестоОтгрузки**, который может принимать значения типа **СправочникСсылка.Склад** и **СправочникСсылка.АдресаОтгрузки**.

В том случае, если необходимо отобрать документы **МаршрутныйЛист** с условием по названию склада, указанного в накладной (т. е. по реквизиту **Название** объекта типа **СправочникСсылка.Склад**, записанного в реквизит **МестоОтгрузки** документа **Накладная**), то условие должно выглядеть следующим образом:

[Копировать в буфер обмена](#)

`cast(cast(ОснованиеОтгрузки, 'Document_Накладная')/МестоОтгрузки, 'Catalog_Склад')/Название eq 'Основной в Москве'`

Не поддерживаются следующие стандартные функции: **length**, **indexof**, **replace**, **tolower**, **toupper**, **trim**, **years**, **days**, **hours**, **seconds**, **floor**, **ceiling**;

17.4.5.3. \$top

Имеется возможность ограничить количество записей, возвращаемых при обращении к ресурсу. Для этого используется параметр **\$top**.

Пример:

[Копировать в буфер обмена](#)

`http://host/odata/standard.odata/Catalog_Товары?$filter=Цена lt 1000&$top=10`

17.4.5.4. allowedOnly

Если при выполнении запроса необходимо получить только те объекты данных, которые не попадают под ограничения доступа к данным, то в URL получения данных необходимо добавить параметр **allowedOnly**.

Пример:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Catalog_Товары?$allowedOnly=true`

Если параметр не указан или указан со значением **false**, то во время исполнения запроса к данным может возникнуть ошибка с кодом 401 (если результат выполнения запроса содержит данные, доступ к которым запрещен). Ошибка может не произойти в том случае, если были указаны дополнительные условия, которые ограничили выборку только разрешенными данными.

Например, для справочника **Данные** настроено ограничение доступа к данным, которое не позволяет получить элементы справочника, у которых реквизит **ЗначениеСекретности** равно 1, но позволяет получить элементы справочника, у которых реквизит **ЗначениеСекретности** равно 2. В этом случае следующий запрос не приведет к возникновению ошибки, т. к. явно накладывается условие, которое оставляет в выборе только разрешенные данные:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Catalog_Данные?allowedOnly=false&$filter=ЗначениеСекретности eq 2`

Параметр **allowedOnly** можно использовать только для GET-запросов к наборам сущностей.

17.4.5.5. \$skip

Позволяет исключить из результата запроса первые несколько записей. Если параметры **\$top** и **\$skip** указываются одновременно, то параметр **\$skip** будет применен раньше, чем параметр **\$top**. Приоритет применения параметров не зависит от порядка их указания в теле запроса.

Пример:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Catalog_Товары?$skip=2`

17.4.6. Параметры запроса

17.4.6.1. \$count

Данный параметр позволяет получить в качестве результата запроса не выборку, а ее (выборки) размер. При успешном выполнении, тело ответа должно содержать только число элементов коллекции, отформатированное как обычное число.

Пример:

```
// возвращает количество элементов справочника Товары,  
// для которых справедливо условие: значение реквизита Цена больше 500  
http://host/base/odata/standard.odata/Catalog_Товары/$count?$filter=Цена gt 500
```

[Копировать в буфер обмена](#)

17.4.6.2. \$inlinecount

Параметр позволяет указать, что система должна включить результат запроса не только полученные записи, но и количество этих записей. Для этого необходимо указывать значение параметра **\$inlinecount**, равное **allpages**. Использование параметра **\$inlinecount** со значением **none** подавляет возвращение количества записей вместе с результатом запроса. Если параметр **\$inlinecount** не указан в теле запроса, то количество записей вместе с результатом запроса не возвращается. Использование параметра **\$inlinecount** приводит к игнорированию параметров **\$skip** и **\$top**. Допускается совместное использование параметров **\$filter** и **\$inlinecount**.

Пример:

```
http://host/base/odata/standard.odata/Catalog_Товары?$inlinecount=allpages
```

[Копировать в буфер обмена](#)

Пример результата запроса, с параметром **\$inlinecount=allpages**:

Формат atom+xml:

```
<?xml version="1.0" encoding="UTF-8"?>  
<feed xmlns=http://www.w3.org/2005/Atom xmlns:at=http://purl.org/atompub/tombstones/1.0  
xmlns:d=http://schemas.microsoft.com/ado/2007/08/dataservices  
xmlns:m=http://schemas.microsoft.com/ado/2007/08/dataservices/metadata  
xml:base="http://localhost/ODataTests/odata/standard.odata/">  
  <entry>  
    ...  
  </entry>  
  ...  
  <entry>  
    ...  
  </entry>  
  <m:count>19</m:count>  
</feed>
```

[Копировать в буфер обмена](#)

Формат json:

```
{  
  "odata.metadata": "http://localhost/ODataTests/odata/standard.odata/$metadata#Document_ДемоДокумент",  
  "odata.count": "19",  
  "value": [{...}, ..., {...}]  
}
```

[Копировать в буфер обмена](#)

17.4.6.3. \$orderby

Данный параметр позволяет задать упорядочивание результата запроса. Значение параметра **\$orderby** содержит список реквизитов, разделенных запятыми. Значение реквизита может включать направление упорядочивания:

- **asc** - для упорядочивания по возрастанию;
- **desc** - для упорядочивания по убыванию.

Направление упорядочивания должно быть отделено от имени реквизита на 1 и более пробелов. Если **asc** или **desc** не указан, то считается, что требуется упорядочивание по возрастанию.

Пример:

```
http://host/base/odata/standard.odata/Catalog_Товары?$orderby=Название asc, Производитель desc
```

[Копировать в буфер обмена](#)

Допускается упорядочивание по свойствам дочерних реквизитов. В этом случае полное имя реквизита, по которому выполняется упорядочивание, формируется с использованием разделителя **"/**.

Пример:

```
http://host/base/odata/standard.odata/Document_Расход?$orderby=Контрагент/ИНН asc
```

[Копировать в буфер обмена](#)

В этом примере упорядочивание будет произведено по возрастанию (**asc**) значения реквизита **ИНН** у сущности, на которую ссылается реквизит **Контрагент** документа **Расход**.

17.4.6.4. \$expand

Данный параметр позволяет вместе с результатами основного запроса получать значения связанных сущностей, что позволит не запрашивать каждую сущность отдельно.

Значением параметра выступает список из навигационных ссылок или составных реквизитов (если составные реквизиты могут принимать ссылочное значение), разделенных запятыми. Для расширения реквизитов составных реквизитов, после имени составного реквизита должно идти название типа, на который может хранить ссылку составной реквизит, а уже после этого сам расширяемый реквизит. Тогда, если у сущности составной реквизит имеет ссылку на объект указанного типа, то в нем будет расширен указанный реквизит. Если же значение является ссылкой на объект иного типа или не является ссылкой, то расширение проводиться не будет.

Возможно использование опции для расширения сущностей из результатов функций `SliceLast()` и `SliceFirst()`.

Пример:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Сертификат](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Сертификат) [Копировать в буфер обмена](#)

Для каждого элемента из справочника **Клиенты**, в результат запроса должен быть помещен объект, на который ссылается реквизит **Сертификат**.

Пример:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Договор/Document_ДоговорНайма/Заявление](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Договор/Document_ДоговорНайма/Заявление) [Копировать в буфер обмена](#)

Для каждого элемента из справочника **Клиенты**, в результат запроса должен быть добавлен объект, на который ссылается реквизит составного типа **Договор** (если он содержит какое-либо ссылочное значение). Если реквизит **Договор** хранит ссылку на объект типа **ДокументСсылка.ДоговорНайма**, то объект, на который ссылается **Договор.Заявление** так же должен быть добавлен в результате запроса.

Значение ***** в конце пути параметра **\$expand** означает необходимость раскрыть все навигационные ссылки внутри сущности. Однако, нельзя раскрывать навигационные поля у составного реквизита без явного указания типа ссылки.

Пример:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Договор/Document_ДоговорНайма/*](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Договор/Document_ДоговорНайма/*) [Копировать в буфер обмена](#)

В данном примере **Договор** - это составной реквизит. Данный вариант указания ***** поддерживается.

Пример:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Договор/*](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Договор/*) [Копировать в буфер обмена](#)

В данном примере **Договор** - это составной реквизит. Данный вариант указания ***** не поддерживается.

Примеры:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=*](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=*) [Копировать в буфер обмена](#)

Для каждого элемента из справочника **Клиенты**, в результат запроса должны быть добавлены объекты, на которые ссылаются все навигационные ссылки объекта **Клиента**.

Пример:

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Договор/*](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Договор/*) [Копировать в буфер обмена](#)

Для каждого элемента из справочника **Клиенты**, в результат запроса должны быть добавлены объекты, на которые ссылаются все навигационные ссылки внутри объекта **Договор** и сам объект **Договор**. При этом реквизит **Договор** является реквизитом ссылочного типа.

Ограничения параметра **\$expand**:

- Не поддерживается расширение реквизитов табличных частей.
- Не поддерживается расширение при запросе одиночных сущностей (сущность, строки табличной части, запись регистра).
- Расширение ссылочных и составных типов виртуальных таблиц не соответствует протоколу OData версии 3.

Запрос, в котором будет выполняться расширение измерения виртуальной таблицы будет выглядеть следующим образом:

[http://host/base/odata/standard.odata/AccountingRegister_РегистрБухгалтерии/Balance?\\$expand=Организация](http://host/base/odata/standard.odata/AccountingRegister_РегистрБухгалтерии/Balance?$expand=Организация) [Копировать в буфер обмена](#)

В результате исполнения этого запроса будет получен следующий результат:

`<d:Организация_Key>УникальныйИдентификатор</d:Организация_Key>
<d:Организация>
...ДанныеОбъекта...
</d:Организация>` [Копировать в буфер обмена](#)

При расширении составного типа поведение аналогично, за исключением того, что данные объекта будут представлены в элементе с суффиксом `_Expanded`.

17.4.7. Способы получения описания стандартного интерфейса OData

Для получения упрощенного описания стандартного интерфейса OData (только список сущностей) необходимо выполнить GET-запрос с использованием URL <http://host/base/odata/standard.odata>.

Для получения описания стандартного интерфейса OData необходимо выполнить GET-запрос с использованием URL [http://host/base/odata/standard.odata/\\$metadata](http://host/base/odata/standard.odata/$metadata). В результате будет получен полный список доступных сущностей, их атрибутов и функций в виде XML-документа. Подробное описание документа можно получить по адресу <http://www.odata.org/documentation/odata-version-3-0/common-schema-definition-language-csdl/> (на английском языке).

В случае получения данных в формате **json**, не поддерживается запрос получения расширенного описания метаданных вида [http://host/base/odata/standard.odata/\\$metadata](http://host/base/odata/standard.odata/$metadata). Однако имеется возможность управлять детализацией информации о метаданных, которая возвращается при получении данных (как сущностей, так и списков сущностей). Для этого необходимо особым образом формировать параметр `$format` при выполнении запроса:

- `$format=application/json;odata=minimalmetadata` - в этом случае информация о метаданных передается в минимальном объеме. Это значение по умолчанию.
- `$format=application/json;odata=nometadata` - в этом случае информация о метаданных не передается вовсе.
- `$format=application/json;odata=fullmetadata` - в этом случае информация о метаданных передается в полном объеме.

17.4.8. Способы получения данных

С помощью стандартного интерфейса OData можно получать как списки сущностей, так и сами сущности. Эти способы получения данных отличаются URL, по которому происходит обращение к данным.

Для получения списка сущностей URL выглядит следующим образом: http://host/base/odata/standard.odata/Catalog_Товары. Это **URL набора сущностей**. Для получения сущности URL будет выглядеть следующим образом: [http://host/base/odata/standard.odata/Catalog_Товары\(guid'value'\)](http://host/base/odata/standard.odata/Catalog_Товары(guid'value')). В общем виде такой адрес будет называться **канонический URL экземпляра сущности**. Разница заключается в том, что в первом случае получается вся таблица справочника товаров, а вот втором выбирается один объект, который описывается набором ключевых значений (в примере ключевое значение единственное - ссылка на объект). В конкретном случае набор ключевых полей, описывающих сущность, зависит от получаемой сущности. Если ключевых полей более одного, то они должны быть указаны все. Ключевые значения указываются парами **Имя=Значение**, разделенными запятыми. Если ключевое значение одно - имя ключа можно не указывать. Например, для получения значения курса валюты на конкретную дату необходимо выполнить GET-запрос со следующим URL:

[Копировать в буфер обмена](#)

[http://host/base/odata/standard.odata/InformationRegister_КурсыВалют\(Period=datetime'2008-02-05T00:00:00', Валюта_Key=guid'9d5c4222-8c4c-11db-a9b0-00055d49b45e'\)](http://host/base/odata/standard.odata/InformationRegister_КурсыВалют(Period=datetime'2008-02-05T00:00:00', Валюта_Key=guid'9d5c4222-8c4c-11db-a9b0-00055d49b45e'))

При получении данных существует возможность указать, какие свойства сущности (или набора сущностей) необходимо получить с помощью стандартного интерфейса OData. Для этого существует ключевое слово `$select`, которое имеет несколько вариантов использования:

- Признак получения всех свойств.

В этом случае следует указать выражение вида `$select=*` или совсем не указывать выражение `$select`.

- Перечень конкретных свойств, которые необходимо получить.

В этом случае необходимо указать в выражении оператора `$select` перечень (через запятую) всех необходимых полей. При этом следует указывать суффиксы свойств, если это необходимо. Например, для получения значения ссылки на справочник и свойств **Код** и **Артикул**, для справочника **Товары**, необходимо использовать следующий URL:

[Копировать в буфер обмена](#)

[http://host/base/odata/standard.odata/Catalog_Товары?\\$select=Ref_Key, Код, Артикул](http://host/base/odata/standard.odata/Catalog_Товары?$select=Ref_Key, Код, Артикул)

Для получения этих же полей для конкретного экземпляра сущности (с известным идентификатором) будет использоваться следующий URL:

[Копировать в буфер обмена](#)

[http://host/base/odata/standard.odata/Catalog_Товары\(guid'value'\)?\\$select=Ref_Key, Код, Артикул](http://host/base/odata/standard.odata/Catalog_Товары(guid'value')?$select=Ref_Key, Код, Артикул)

- Признак получения всех свойств, кроме табличных частей.

Для этого в выражении оператора `$select` необходимо указать значение `**`. При использовании следующего URL будут получены все документы **РасходТовара** со всеми реквизитами, кроме табличных частей:

[Копировать в буфер обмена](#)

[http://host/base/odata/standard.odata/Document_РасходТовара?\\$select=**](http://host/base/odata/standard.odata/Document_РасходТовара?$select=**)

- Указание списка для получения связанных сущностей.

Допускается совместное использование параметров запроса `$select` и `$expand`. В этом случае с помощью параметра `$expand` указываются те свойства получаемой сущности, которые следует поместить в результат запроса. Конкретные значения получаемых (или разворачиваемых) свойств указывается с помощью параметра `$select`. Описание работа с параметром `$expand` более подробно см. [здесь](#). В параметре `$select`, при использовании совместно с параметром `$expand`, поддерживается использование символов `*` и `**`. Указание символа `*` означает, что необходимо развернуть все реквизиты получаемого объекта. Указание символа `**` означает, что необходимо развернуть все реквизиты получаемого объекта, кроме табличных частей.

[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Сертификат&\\$select=Сертификат/Дата](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Сертификат&$select=Сертификат/Дата)
[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Договор/Document_ДоговорНайма/Заявление&\\$select=Договор/Document_ДоговорНайма/Заявление/**](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Договор/Document_ДоговорНайма/Заявление&$select=Договор/Document_ДоговорНайма/Заявление/**)
[http://host/base/odata/standard.odata/InformationRegister_КурсыВалют?\\$expand=Валюта&\\$select=Период,Валюта/Код,Курс](http://host/base/odata/standard.odata/InformationRegister_КурсыВалют?$expand=Валюта&$select=Период,Валюта/Код,Курс)
[http://host/base/odata/standard.odata/Catalog_Клиенты?\\$expand=Сертификат&\\$select=Сертификат/*](http://host/base/odata/standard.odata/Catalog_Клиенты?$expand=Сертификат&$select=Сертификат/*)

Если в параметре **\$select** одновременно указаны поля вида **Объект/***, **Объект/****, **Объект/ИмяРеквизита**, то они обрабатываются в следующем порядке:

1. Если указан символ *****, то включаются все реквизиты;
2. Иначе, если указан символ ******, то включаются все реквизиты, кроме табличных частей, иные указания игнорируются. Таким образом, если указано выражение вида **\$select=Объект/**, Объект/ТабличнаяЧасть**, то табличная часть включена не будет.
3. Иначе, включаются все указанные реквизиты.

Имеется возможность получать данные, доступные «через точку». В этом случае к адресу, описывающему **конкретную** сущность, прибавляются имена свойств, идущие «через точку», но разделенные символом «/». При этом для стандартных реквизитов следует использовать только английские имена реквизитов. Например, для получения свойства **Наименование** реквизита **Валюта** документа **РасходТовара**, необходимо использовать GET-запрос со следующим URL:

[http://host/base/odata/standard.odata/Document_РасходТовара\(guid'value'\)/Валюта/Description](http://host/base/odata/standard.odata/Document_РасходТовара(guid'value')/Валюта/Description).

17.4.9. Выполнение функций и действий

С некоторыми сущностями и наборами сущностей могут быть связаны функции. Например, через функции выполняется работа с виртуальными таблицами регистров. В этом случае URL ресурса формируется следующим образом:

[http://host/base/odata/standard.odata/<ресурс>/<функция>\(<параметры>\)](http://host/base/odata/standard.odata/<ресурс>/<функция>(<параметры>)). Подробнее рассмотрим, из чего состоит адрес. Первая часть адреса (<http://host/base/odata/standard.odata>) представляет собой стандартный префикс адреса при обращении к стандартному интерфейсу OData. **<ресурс>** - имя ресурса, правила формирования которого см. [здесь](#). Имя функции соответствует англоязычному имени виртуальной таблицы языка запросов (см. [здесь](#)). **<параметры>** у функции задаются парами **Ключ=Значение** и разделяются запятыми. Если в качестве параметра функции используется отбор, то выражение, описывающее отбор, должно удовлетворять общим правилам описания отборов (см. [здесь](#)). Так, получение среза первых для регистра курсов валют (с параметрами), будет выполняться по следующему URL: [http://host/base/odata/standard.odata/InformationRegister_КурсыВалют/SliceFirst\(Period=datetime'2008-01-01T00:00:00',Condition='Валюта_Key eq guid'value'\)](http://host/base/odata/standard.odata/InformationRegister_КурсыВалют/SliceFirst(Period=datetime'2008-01-01T00:00:00',Condition='Валюта_Key eq guid'value')).

17.4.10. Ошибочные ситуации

В случае ошибочной ситуации возвращается ответ с HTTP-статусом **4xx** или **5xx**. Статус **4xx** информирует об ошибках на стороне клиентского приложения, статус **5xx** информирует об ошибке на стороне сервера.

В случае статуса **4xx** сервер пытается уточнить причину ошибки и может передать клиентскому приложению дополнительный внутренний код ошибки и информационное сообщение (в виде xml-документа) в теле ответа.

Пример:

```
<m:error xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
  <m:code>9</m:code>
  <m:message>Экземпляр сущности "НакладнаяОтгрузки" не найден по переданному ключу.</m:message>
</m:error>
```

Далее перечислены внутренние коды ошибок с описанием причины появления:

Код	Описание
0	Возможность не поддерживается
1	Не удалось разобрать строку
2	Неверный формат запроса
3	Запрошенный тип представления не поддерживается
4	Неверное значение свойства
5	Отсутствует обязательное значение свойства
6	Неверный URL
7	Не хватает элемента ключа сущности
8	Тип сущности не найден
9	Экземпляр сущности не найден
10	Запрошенное свойство не найдено
11	Метод не найден
12	Отсутствует обязательный аргумент метода
13	Создание строк табличных частей напрямую не поддерживается

14	Ошибка разбора опций запроса
15	Сущность с таким ключом уже существует
16	Не удалось присвоить свойство
17	Объект не поддерживает режим загрузки данных
18	Ошибка инициализации интерфейса OData: в объекте есть свойства с одинаковыми именами
19	Использованный HTTP-метод запрещен в данном контексте
20	Ошибка прав доступа. Может возникать: • Когда у пользователя нет прав на запрошенное действие над данным объектом; • Когда в выборку попадает объект, недоступный в связи с ограничением доступа к данным и параметр <code>allowedOnly</code> не используется.
21	Вызов нереализованной функции. Указание неверного количества аргументов функции. Попытка передачи аргумента неверного типа. Указание нереализованной лямбда-функции.

17.4.11. Установка значений разделителей

Значения разделителей можно устанавливать только следующими способами:

- С помощью фрагментов URL при обращении к стандартному интерфейсу OData;
- С помощью файла описания публикации `default.vrd`.

Указанием разделителей с помощью параметра командной строки `Z` не поддерживается.

17.4.12. Публикация стандартного интерфейса OData

Публикация стандартного интерфейса OData выполняется с помощью диалога публикации на веб-сервере ([Администрирование - Публикация на веб-сервере](#)) и описано [в книге 1С:Предприятие 8.3. "Руководство администратора"](#).

Для того чтобы объекты конфигурации стали доступны через стандартный интерфейс OData, необходимо разрешить это с помощью метода глобального контекста `УстановитьСоставСтандартногоИнтерфейсаOData()`. Если прикладное решение функционирует в режиме совместимости с версией 8.3.4 (и ниже), данный метод не используется. В этом случае с помощью стандартного интерфейса OData доступны все поддерживаемые объекты конфигурации.

С помощью данного метода можно ограничить перечень доступных объектов конфигурации только теми (из общего перечня поддерживаемых, см. [здесь](#)), доступ к которым действительно необходим для внешних приложений.

Механизм установки состава объектов, доступных с помощью стандартного интерфейса OData, можно выполнить в виде внешней обработки. Для этого не требуется модифицировать прикладное решение.

17.4.13. Выполнение типовых операций

В данном разделе будут приведены примеры выполнения некоторых типовых операций по работе с данными с помощью стандартного интерфейса OData.

ВНИМАНИЕ! Данные примеры не являются законченными. Они приведены только для иллюстрации применения различных конструкций.

17.4.13.1. Работа с одним объектом

17.4.13.1.1. Чтение данных

Для получения информации о сущности следует использовать канонический URL сущности. Чтение выполняется с помощью GET-запроса.

Пример чтения ссылочного объекта:

```
http://host/base/odata/standard.odata/Catalog_Товары(guid'value')
```

[Копировать в буфер обмена](#)

Пример чтения набора записей подчиненного регистра сведений:

```
http://host/base/odata/standard.odata/InformationRegister_ПриходныеЦены(Recorder_Key=guid'value')
```

[Копировать в буфер обмена](#)

В данном примере `guid'value'` идентифицирует документ, выполнивший движение по регистру сведений.

Пример чтения записи регистра сведений:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/InformationRegister_ПриходныеЦены_RecordType(Товар_Key=guid'value', ТипЦены="Приходная")`

Пример чтения строки конкретного документа:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/Document_РасходТовара_Товары(Ref_Key=guid'value', LineNumber=1)`

Чтение виртуальной таблицы `СрезПоследних` регистра сведений `КурсыВалют` (с отбором по измерению `ОсновнаяВалюта` типа `СправочникСсылка.Валюты`) будет выглядеть следующим образом:

Пример чтения независимого регистра сведений:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/InformationRegister_КурсыВалют/SliceLast?Condition=Валюта/ОсновнаяВалюта_Key eq guid'value'`

Пример чтения подчиненного регистра сведений (к имени регистра следует добавить `_RecordType`):

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/InformationRegister_КурсыВалют_RecordType/SliceLast?Condition=Валюта/ОсновнаяВалюта_Key eq guid'value'`

В данном примере `guid'value'` идентифицирует элемент справочника `Валюты`, по значению которого выполняется отбор.

Пример получения остатков на счете из регистра бухгалтерии с условием по конкретному контрагенту:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/AccountingRegister_Хозрасчетный/Balance(AccountCondition='Account_Key eq guid'value1',Condition='ExtDimension1 eq cast(guid'value2', 'Catalog_Контрагенты'),'ExtraDimensions='value3')?$format=json`

В данном примере:

- `guid'value1'` - уникальный идентификатор элемента плана счетов;
- `guid'value2'` - уникальный идентификатор элемента плана видов характеристик `ВидыСубконтоХозрасчетные` с типом значения `СправочникСсылка.Контрагенты`;
- `'value3'` - уникальный идентификатор элемента справочника `Контрагенты`.

Пример получения набора записей регистра бухгалтерии по регистратору:

[Копировать в буфер обмена](#)

`http://host/base/odata/standard.odata/AccountingRegister_РегистрБухгалтерииСКорреспонденцией(guid'value')?$format=json`

В данном примере:

- `guid'value'` - уникальный идентификатор регистратора, который выполнил запись набора записей регистра бухгалтерии `РегистрБухгалтерииСКорреспонденцией`.

В качестве результата будет получен json-документ, который содержит набор записей регистра бухгалтерии. Свойство `Recorder_Key` в данном документе будет соответствовать значению `guid'value'`, которое было передано в запросе.

17.4.13.1.2. Создание объекта

Для создания объекта следует воспользоваться POST-запросом с использованием URL набора сущностей, передав в теле запроса документ (в поддерживаемом формате), который содержит значения полей создаваемого объекта. Если передаваемый документ содержит свойства, отсутствующие у создаваемого объекта, то эти свойства игнорируются.

Далее приводится пример создания элемента справочника `Товары` и ответ стандартного интерфейса OData после успешного выполнения операции.

Пример POST-запроса (формат `atom`):

[Копировать в буфер обмена](#)

```
POST http://host/base/odata/standard.odata/Catalog_Товары HTTP/1.1
User Agent: Fiddler
Host: host
Content Length: 981
<entry>
  <category term="StandardODATA.Catalog_Товары"
  scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
    <title type="text"/>
    <updated>2014-02-14T12:05:55</updated>
    <author/>
    <summary/>
    <content type="application/xml">
    <m:properties xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices"
  xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
    <d:DeletionMark>false</d:DeletionMark>
    <d:Parent_Key>bbb079ae-8c51-11db-a9b0-00055d49b45e</d:Parent_Key>
    <d:IsFolder>false</d:IsFolder>
    <d:Code>000000800</d:Code>
```

```
<d:Description>Шлепанцы</d:Description>
<d:Артикул>SL56X</d:Артикул>
<d:Поставщик_Key>086715b0-f348-11db-a9c5-00055d49b45e</d:Поставщик_Key>
<d:Вид>Товар</d:Вид>
<d:Штрихкод/>
<d:Описание>&lt;html&gt;Шлепанцы пляжные&lt;/html&gt;</d:Описание>
</m:properties>
</content>
</entry>
```

Пример ответа стандартного интерфейса OData:

[Копировать в буфер обмена](#)

```
HTTP/1.1 201 Created
Content Length: 2705
Content Type: application/atom+xml;type=entry;charset=utf 8
Location: http://host/base/odata/standard.odata/Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008')
Server: Microsoft IIS/7.5
DataServiceVersion: 3.0
X Powered By: ASP.NET
Date: Fri, 14 Feb 2014 08:18:36 GMT
<?xml version="1.0" encoding="UTF-8"?>
<entry xmlns="http://www.w3.org/2005/Atom"
  xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices
  xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
  <id>http://host/base/odata/standard.odata/Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008')</id>
  <category term="StandardODATA.Catalog_Товары"
  scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
  <title type="text"/>
  <updated>2014-02-14T12:18:36</updated>
  <author/>
  <summary/>
  <link rel="http://schemas.microsoft.com/ado/2007/08/dataservices/related/ФайлКартинки"
  href="Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008')/ФайлКартинки"
  type="application/atom+xml;type=entry;charset=utf-8"
  title="ФайлКартинки"/>
  <link rel="http://schemas.microsoft.com/ado/2007/08/dataservices/related/Parent"
  href="Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008')/Parent"
  type="application/atom+xml;type=entry;charset=utf-8"
  title="Parent"/>
  <link rel="http://schemas.microsoft.com/ado/2007/08/dataservices/related/Поставщик"
  href="Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008')/Поставщик"
  type="application/atom+xml;type=entry;charset=utf-8"
  title="Поставщик"/>
  <link rel="edit"
  href="Catalog_Товары(guid'41aa6331-954f-11e3-814b-005056c00008') "
  title="edit link"/>
  <content type="application/xml">
  <m:properties xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices"
  xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
  <d:Ref_Key>41aa6331-954f-11e3-814b-005056c00008</d:Ref_Key>
  <d:DataVersion m:null="true"/>
  <d:Description>Шлепанцы</d:Description>
  <d:Code>000000800</d:Code>
  <d:Parent_Key>bbb079ae-8c51-11db-a9b0-00055d49b45e</d:Parent_Key>
  <d:IsFolder>false</d:IsFolder>
  <d:DeletionMark>false</d:DeletionMark>
  <d:Артикул>SL56X</d:Артикул>
  <d:Поставщик_Key>086715b0-f348-11db-a9c5-00055d49b45e</d:Поставщик_Key>
  <d:ФайлКартинки_Key>00000000-0000-0000-0000-000000000000</d:ФайлКартинки_Key>
  <d:Вид>Товар</d:Вид>
  <d:Штрихкод/>
  <d:Описание>&lt;html&gt;Шлепанцы пляжные&lt;/html&gt;</d:Описание>
  <d:ФайлКартинки_Key>00000000-0000-0000-0000-000000000000</d:ФайлКартинки_Key>
  <d:Parent_Key>bbb079ae-8c51-11db-a9b0-00055d49b45e</d:Parent_Key>
  <d:Поставщик_Key>086715b0-f348-11db-a9c5-00055d49b45e</d:Поставщик_Key>
  </m:properties>
  </content>
</entry>
```

Пример POST-запроса (формат json):

[Копировать в буфер обмена](#)

```
POST http://host/base/odata/standard.odata/Catalog_Товары HTTP/1.1
Accept: application/json
Accept Charset: UTF 8
User Agent: Fiddler
Content Type: application/json
Content Length: 2426
{
  "DeletionMark":false,
  "Parent_Key":"bbb079ae-8c51-11db-a9b0-00055d49b45e",
  "IsFolder":false
  "Code":"000000800",
  "Description":"Шлепанцы",
  "Артикул":"SL56X",
```

```
"Поставщик_Key": "086715b0-f348-11db-a9c5-00055d49b45e",
"Вид": "Товар",
"Штрихкод": null,
"Описание": "Шлепанцы пляжные"
}
```

17.4.13.1.3. Обновление объекта

Для обновления объекта необходимо выполнить PUT-/PATCH-запрос с использованием канонического URL сущности (аналогично запросу GET для получения сущности), передав в теле запроса XML-документ (в формате atom) или JSON-документ (в формате json), который содержит значения свойств сущности. Если передаваемый документ содержит свойства, отсутствующие у создаваемого объекта, то эти свойства игнорируются.

В случае PATCH-запроса пропущенные свойства сущности будут проигнорированы, т. е. будут изменены только те свойства, которые переданы в запросе на изменение. Для PUT-запроса необходимо указывать значения всех свойств обновляемой сущности.

При модификации сущности через PUT-запрос, при записи значений ссылочных реквизитов, следует использовать поле `ИмяСсылочногоРеквизита@odata.bind` куда следует поместить URI-сущности (можно в укороченном варианте - только фрагмент пути после `standard.odata`).

Например:

[Копировать в буфер обмена](#)

```
PUT http://host /base/odata/standard.odata/Document_Документ(guid'19961ec3-3c5f-11e7-8785 50465da19fe4')?
$format=json
{
  "Date": "2015-01-01T12:01:07",
  "Организация@odata.bind": "Catalog_Организации(guid'298584b5-92e4-11e3-8bc3-0050568b1678')",
  "Склад@odata.bind": http://host /base/odata/standard.odata/Catalog_Склады(guid'081bc346-2abc-13e4-a1bd
0050568b1688')
  ...
}
```

Изменение реквизита справочника

В следующем примере рассматривается изменение реквизита **Наименование** и состава табличной части **ТорговыеЗалы** справочника **Магазины**. Остальные реквизиты справочника не используются в рамках данного примера. В примере используется формат данных json.

Пример PATCH-запроса:

[Копировать в буфер обмена](#)

```
PATCH http://host/base/odata/standard.odata/Catalog_Магазины(guid'value')?$format=json HTTP/1.1
Host: host
Connection: keep alive
Accept: application/json
Content Length: 638
{
  "odata.metadata": "http://host/base/odata/standard.odata/$metadata#Catalog_Магазины/@Element",
  "Description": "Новое описание магазина",
  "ТорговыеЗалы@odata.type" : "Collection(StandardODATA.Catalog_Магазины_ТорговыеЗалы_RowType)",
  "ТорговыеЗалы": [
    {
      "LineNumber": "1",
      "Название": "Синий зал",
      "Площадь": 56,
      "ДатаОткрытия": "2015-01-01T00:00:00"
    },
    {
      "LineNumber": "2",
      "Название": "Красный зал",
      "Площадь": 56,
      "ДатаОткрытия": "2015-06-13T11:45:41"
    }
  ]
}
```

ВНИМАНИЕ! Табличную часть следует передавать полностью (все строки) даже в том случае, если требуется изменить данные только в одной строке этой табличной части.

В данном примере `guid'value'` идентифицирует изменяемый элемент справочника **Магазины**.

Изменение набора записей регистра бухгалтерии

Изменение набора записей регистра бухгалтерии выполняется с помощью следующей последовательности действий:

1. Выполняется чтение набора записей, который планируется изменить. Используется GET-запрос.
2. Выполняется модификация данных, который вернул предыдущий запрос. Под «модификацией» понимается изменение данных в существующих записях набора, добавление новых записей и удаление существующих.
3. Выполняется обновление набора записей в информационной базе. Для этого используется PATCH-запрос.

Рассмотрим последовательность действий более подробно.

Для получения набора записей необходимо выполнить GET-запрос следующего вида:

[Копировать в буфер обмена](#)

```
http://host/base/odata/standard.odata/AccountingRegister_РегистрБухгалтерииСКорреспонденцией(guid'value')?$format=json
```

В данном примере выполняется запрос к регистру бухгалтерии **РегистрБухгалтерииСКорреспонденцией**, а **guid'value'** - это ссылка на регистратор, который создал набор записей в регистре.

В результате GET-запроса будет получена информация следующего вида:

[Копировать в буфер обмена](#)

```
Content-Type: application/json;charset=utf-8
Content-Length: X
{
  "odata.metadata":
"http://host/base/odata/standard.odata/$metadata#AccountingRegister_РегистрБухгалтерииСКорреспонденцией/@Element",
  "Recorder_Key": "value",
  "RecordSet": [
    {
      "Recorder_Key": "value",
      ...
    }
  ]
}
```

В приведенном фрагменте результата **value** - это значение ссылки на регистратор, который был передан в GET-запросе.

При модификации полученных данных следует сформировать «заголовок» записываемых данных следующим образом:

[Копировать в буфер обмена](#)

```
{
  "odata.type": "StandardODATA.AccountingRegister_РегистрБухгалтерииСКорреспонденцией_RowType",
  "Recorder_Key": "value",
  "RecordSet@odata.type":
"Collection(StandardODATA.AccountingRegister_РегистрБухгалтерииСКорреспонденцией_RowType)",
  "RecordSet": [
    ... // содержимое набора записей
  ]
}
```

В этом наборе данных:

- **value** - это значение ссылки на регистратор.
- **odata.type** - это описание типа данных, которые будут помещены в информационную базу.
- **RecordSet@odata.type** - уточнение типа набора записей (свойство **RecordSet**).

В наборе записей можно изменять как свойства существующих записей, так и выполнять добавление новых и удаление существующих записей.

После того, как модификация набора записей завершена, следует выполнить обновление информации в информационной базе. Для этого предназначен PATCH-запрос следующего вида:

[Копировать в буфер обмена](#)

```
http://host/base/odata/standard.odata/Constant_ИспользоватьИнтеграцию(guid'00000000-0000-0000-0000-000000000000')?$format=json
```

В качестве тела запроса будет выступать json-документ, который подготовлен ранее, на основании результата GET-запроса.

Изменение константы

В следующем примере рассматривается изменение константы **ИспользоватьИнтеграцию** типа **Булево**. В примере используется формат данных json.

Пример PATCH-запроса:

[Копировать в буфер обмена](#)

```
PATCH http://host/base/odata/standard.odata/Constant_ИспользоватьИнтеграцию(guid'00000000-0000-0000-0000-000000000000')?$format=json
{
  "Value": true,
}
```

17.4.13.1.4. Удаление объекта

Для удаления объекта следует воспользоваться DELETE-запросом с использованием канонического URL сущности.

ВНИМАНИЕ! Пометка на удаление не выполняется, объект удаляется непосредственно.

17.4.13.1.5. Оптимистическая блокировка данных

Для оптимистической блокировки данных (т. е. для проверки, что данные не изменились с момента считывания) нужно использовать заголовок `If-Match` HTTP-запроса, связанного с модификацией (PATCH) или удалением (DELETE) данных. В качестве значения заголовка должно выступать значение свойства `DataVersion`, которое получено при предварительном чтении сущности.

Механизм работает следующим образом:

- При запросе экземпляра сущности среди прочих возвращается свойство `DataVersion`. При запросе набора сущностей с каждой из сущностей набора возвращается свойство `DataVersion`.
- Клиентское приложение должно сохранить у себя это значение версии объекта и затем использовать его в запросах PATCH и DELETE, передавая его в заголовке `If-Match`.
- Если значение свойства `DataVersion` в момент исполнения запроса совпадает со значением заголовка `If-Match`, то происходит запрошенное действие. В противном случае действие не выполняется, а клиенту возвращается HTTP статус 412.

17.4.13.1.6. Запись объекта в режиме загрузки данных

Если при записи объекта необходимо эмулировать запись, выполняемую во время работы механизма обмена данными (свойство `ОбменДанными.Загрузка = Истина`) следует использовать HTTP-заголовок `1C_OData-DataLoadMode` со значением `true` при выполнении соответствующей операции записи.

17.4.13.1.7. Проведение и отмена проведения документа

Для проведения документа необходимо выполнить POST-запрос с использованием канонического URL сущности (аналогично запросу GET для получения сущности), указав особым образом сформированный суффикс URL. Суффикс в этом случае будет состоять из команды `Post` и параметра, указывающего режим проведения документа:
[http://host/base/odata/standard.odata/Document_РасходТоваров\(guid'value'\)/Post?PostingModeOperational=false](http://host/base/odata/standard.odata/Document_РасходТоваров(guid'value')/Post?PostingModeOperational=false).

Для отмены проведения документа суффикс состоит из команды `Unpost` без параметров.

17.4.13.2. Работа с коллекцией объектов

Для получения набора сущностей какого-либо вида, следует выполнить GET-запрос с использованием URL следующего вида:
http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными.

Если необходимо установить отбор на получаемый список, его можно выполнить с помощью параметра `$filter` (см. [здесь](#)), который указывается в URL:

[http://host/base/standard.odata/Document_Накладная?\\$filter=Приоритет eq 1](http://host/base/standard.odata/Document_Накладная?$filter=Приоритет eq 1) Копировать в буфер обмена

Обращение к виртуальной таблице регистра выглядит следующим образом (на примере виртуальной таблицы `ОстаткиИОбороты` регистра накопления `ТоварныеЗапасы`):

[http://host/base/odata/standard.odata/AccumulationRegister_ТоварныеЗапасы/](http://host/base/odata/standard.odata/AccumulationRegister_ТоварныеЗапасы/BalanceAndTurnovers(StartPeriod=datetime'2014 01 01', EndPeriod=datetime'2014 02 01', Condition='Товар_Key eq guid'value'))
`BalanceAndTurnovers(StartPeriod=datetime'2014 01 01', EndPeriod=datetime'2014 02 01', Condition='Товар_Key eq guid'value')` Копировать в буфер обмена

Также для ограничения набора сущностей можно использовать параметры `$top` и `$select` (см. [здесь](#)).

17.4.13.3. Работа с планами обмена

17.4.13.3.1. Формирование сообщения обмена

Для формирования сообщения обмена необходимо выполнить POST-запрос с использованием URL следующего вида:
<http://host/base/odata/standard.odata/SelectChanges?<params>>. Где в качестве параметров необходимо указать следующее:

- Параметр `DataExchangePoint` - должен содержать канонический URL сущности требуемого элемента плана обмена;
- Параметр `MessageNo` - должен содержать номер сообщения обмена данными, который будет сформирован в результате данного вызова.

В результате, полный URL для формирования сообщения обмена, будет выглядеть следующим образом:

[http://host/base/odata/standard.odata/SelectChanges?](http://host/base/odata/standard.odata/SelectChanges?DataExchangePoint='http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными(guid'value')&MessageNo=34)
[DataExchangePoint='http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными\(guid'value'\)&MessageNo=34](http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными(guid'value')&MessageNo=34).

В результате будет получен список изменений, которые необходимо передать в другой узел, в виде потока atom-feed. Каждый элемент будет представлен в формате atom-entry, а удаленные элементы будут представлены в формате atom-deleted-entry (RFC 6721, <https://datatracker.ietf.org/doc/html/rfc6721>, на английском языке).

17.4.13.3.2. Уведомление о получении изменений

Для уведомления сервера о том, что сообщение обмена успешно получено, необходимо выполнить POST-запрос с использованием URL следующего вида: <http://host/base/odata/standard.odata/NotifyChangesReceived?<params>>. Где в качестве параметров необходимо указать следующее:

- Параметр `DataExchangePoint` - должен содержать канонический URL сущности требуемого элемента плана обмена;

- Параметр [MessageNo](#) - должен содержать номер сообщения обмена данными, подтверждение получения которого необходимо зафиксировать.

В результате, полный URL для подтверждения получения сообщения обмена, будет выглядеть следующим образом:

[http://host/base/odata/standard.odata/NotifyChangesReceived?](http://host/base/odata/standard.odata/NotifyChangesReceived?DataExchangePoint='http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными(guid'value')&MessageNo=34)

[DataExchangePoint='http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными\(guid'value'\)&MessageNo=34](http://host/base/odata/standard.odata/ExchangePlan_ОбменДанными(guid'value')&MessageNo=34).

17.5. WebSocket

17.5.1. Общая информация

Протокол WebSocket (RFC 6455, <https://datatracker.ietf.org/doc/html/rfc6455>) обеспечивает возможность обмена данными между сервером и клиентом посредством постоянного соединения. Обмен данными выполняется с помощью «пакетов». Данные передаются в обе стороны (дуплексный обмен), без разрыва и дополнительных HTTP-запросов. Обмен данными может выполняться как через незащищенный канал (используется схема `ws://`), так и через защищенный канал (используется схема `wss://`).

В определении выше под терминами «клиент» и «сервер» понимается следующее:

- Сервер - это некоторая удаленная программная система, которая хранит/обрабатывает какие-либо данные и может передавать их клиенту.
- Клиент - это локальная программная система, которая может получать данные от сервера. Клиент выступает инициатором соединения с сервером. В рамках данного раздела под термином «клиент» будет пониматься прикладное решение системы «1С:Предприятие» (в любом варианте использования), которое подключается к любой удаленной программной системе, выступающей в роли WebSocket-сервера.

Также, важным является тот факт, что обмен по протоколу WebSocket выполняется в дуплексном режиме, т. е. передавать данные могут и клиент, и сервер и они могут это делать одновременно.

С точки зрения собственно протокола, WebSocket является достаточно простым. Клиент может инициировать выполнение следующих действий: открыть соединение, закрыть соединение и отправить данные. В то же время, у WebSocket есть 4 события, которые мы должны обработать: соединение открыто, соединение закрыто, получены данные и произошла ошибка. Объектная модель «1С:Предприятие» предоставляет доступ и к выполнению действий, и к обработке перечисленных событий. Дополнительно объектная модель системы «1С:Предприятие» позволяет получить текущее состояние соединения.

Система «1С:Предприятие» может выступать только в качестве клиента.

17.5.2. WebSocket-клиент

17.5.2.1. Общая информация

Когда какой-либо программный продукт выступает в роли клиента, это означает, что этот программный продукт может подключаться к серверу, но не может принимать подключения других клиентов. В основе любого взаимодействия с помощью WebSocket лежит «соединение». Соединением будем называть представление физического соединения между клиентом и сервером в объектной модели встроенного языка системы «1С:Предприятие». Соединение может использоваться различным образом:

- Полностью из встроенного языка. Используется объект [WebSocketКлиентСоединение](#). Доступно и на стороне клиентского приложения, и на стороне кластера серверов (и это будут разные списки соединений). В рамках документации будем называть такое использование «динамическим».
- Через объект метаданных [WebSocket-клиент](#) (доступен в дереве конфигурации в ветви [Общие - WebSocket-клиенты](#)). Объекты, созданные в дереве конфигурации, доступны только на стороне кластера серверов «1С:Предприятия». В рамках документации будем называть такое использование «WebSocket-клиент».

Если говорить о динамическом использовании WebSocket, то логично предположить, что такое использование нужно применять в том случае, когда адрес WebSocket-сервера не известен разработчику прикладного решения в момент разработки, но что такое взаимодействие будет использоваться - разработчик точно знает. В этом случае можно реализовать инструмент хранения параметров подключения к WebSocket-серверам, и реализацию протокола обмена данными, в конфигурации, а настройку выполнять уже во время внедрения прикладного решения. Еще одним вариантом динамического использования WebSocket может выступать ситуация, когда соединение нужно использовать именно на клиентском компьютере, например, на серверном компьютере запрещен доступ к требуемому URL (например, по соображениям безопасности).

Применение WebSocket-клиентов выглядит логичным тогда, когда разработчик конфигурации точно знает, что ему будет нужно взаимодействовать с WebSocket-сервером, расположенным по известному адресу. Тогда можно реализовать объект конфигурации (и связанный с ним код на встроенном языке) в момент разработки конфигурации. Во время внедрения прикладного решения никаких дополнительных действий выполнять не придется.

Еще раз обратим внимание, что в системе существует три списка соединений, которые, в общем случае, не связаны друг с другом:

- Динамические соединения на стороне клиентского приложения.
- Динамические соединения на стороне кластера серверов.
- WebSocket-клиенты на стороне кластера серверов.

С точки зрения объектной модели, общая схема работы с WebSocket выглядит следующим образом:

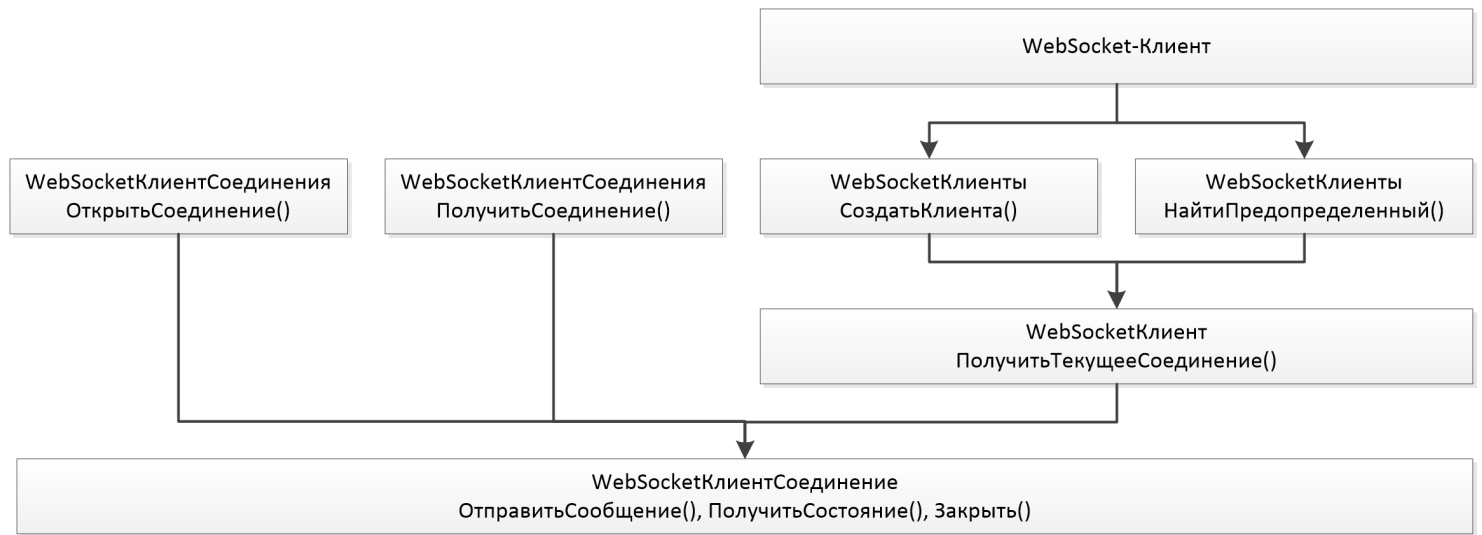


Рис. 510. WebSocket. Краткая схема объектов

Более подробно объекты и методы, участвующие в схеме, будут рассмотрены далее в этом разделе.

17.5.2.2. Динамическое использование соединения

В общем случае, работа с WebSocket выглядит следующим образом:

- Создается соединение. Этому соединению передаются имена обработчиков различных событий WebSocket.
- Обработчики событий используются для реализации реакции на стандартные события WebSocket: открытие/закрытие соединения, получение сообщения от сервера и возникновение ошибки.
- Данные передаются по соединению с помощью вызова специального метода соединения.
- После того, как соединение становится не нужным, его закрывают.

Теперь рассмотрим эту последовательность более подробно.

Создание соединения выполняется с помощью менеджера соединений WebSocket-клиентов. Этот менеджер доступен через свойство глобального контекста `WebSocketКлиентСоединения` (на клиенте и на сервере «1С:Предприятие»). Создание соединения выполняется с помощью метода `ОткрытьСоединение()` этого менеджера. Первым параметром метода (**Ключ**) выступает идентификатор создаваемого соединения. Система «1С:Предприятие» поддерживает одновременное открытие нескольких соединений к одному WebSocket-серверу, поэтому значение параметра **Ключ** будет использоваться далее для получения объекта `WebSocketКлиентСоединение` (который необходим для осуществления взаимодействия с сервером). Отсюда вытекает, что значение параметра **Ключ** должно быть уникально там, где создается соединение (на стороне клиента или сервера «1С:Предприятие»). Параметр **URLСервера** служит для указания URL WebSocket-сервера, с которым мы будем работать через создаваемое соединение. URL может использовать как схему **ws** (открытое соединение, аналог протокола HTTP), так и схему **wss** (защищенное соединение, аналог протокола HTTPS). Следующий параметр (**Обработчики**) позволяет указать методы встроенного языка, которые будут обрабатывать события соединения. Обработчики событий могут быть как в модуле формы, так и в общем модуле. Очевидно, что указывать обработчики формы не следует в том случае, если WebSocket будет использоваться после того, как форма, в которой создано соединение, будет закрыта. В этом случае рекомендуется использовать общий модуль. Параметр **Параметры** предназначен для указания вспомогательных параметров соединения. Параметр **ИмяПользователяИнформационнойБазы** позволяет указать пользователя, от чьего имени будут вызываться методы обработчиков событий соединения. Этот параметр имеет смысл только при создании соединения на стороне серверной части системы «1С:Предприятие». Таким образом, создание соединения в клиентской части формы клиентского приложения будет выглядеть следующим образом:

[Копировать в буфер обмена](#)

```

ОбработчикиСокета = Новый ОбработчикиWebSocketКлиентСоединения;
ОбработчикиСокета.Модуль = ЭтотОбъект;
ОбработчикиСокета.ОбработчикОткрытияСоединения = "ВебСокет_ОбработчикОткрытия";
ОбработчикиСокета.ОбработчикЗакрытияСоединения = "ВебСокет_ОбработчикЗакрытия";
ОбработчикиСокета.ОбработчикПолученияСообщения = "ВебСокет_ОбработчикПолученияСообщения";
ОбработчикиСокета.ОбработчикОшибки = "ВебСокет_ОбработчикОшибки";
WebSocketКлиентСоединения.ОткрытьСоединение("test-ws-client", АдресВебСокета, ОбработчикиСокета);
  
```

В примере создается соединение, которое будет иметь идентификатор `test-ws-client`, а URL сервера передается в переменной `АдресВебСокета`. Очевидно, что для промышленного применения следует каждому соединению назначать ключ, который имеет функциональный смысл (для облегчения использования) и будет уникальным. Также нужно понимать, что после создания соединения, оно не будет закрыто при уничтожении объекта `WebSocketКлиентСоединение`. Для закрытия соединения требуется одно из следующих действий:

- Явный вызов метода `WebSocketКлиентСоединение.Закрыть()`.
- Разрыв соединения по внешним причинам (Интернет, WebSocket-сервер и т. д.).
- Завершение работы:
 - Клиентского приложения (для соединения, созданного на стороне клиента).

- Менеджера кластера (для соединения, созданного на стороне кластера серверов).

Если теперь в другом месте прикладного решения нам потребуется что-то передать в созданное соединение или узнать статус этого соединения, то будет необходимо воспользоваться методом `ПолучитьСоединение()` менеджера клиентских WebSocket-соединений. В качестве параметра метода необходимо передать тот самый уникальный ключ, который использовался при создании соединения (значение параметра `Ключ метода ОткрытьСоединение()`):

[Копировать в буфер обмена](#)

```
ВебСокет = WebSocketКлиентСоединения.ПолучитьСоединение("test-ws-client");
```

Если требуется получить все созданные WebSocket-соединения (в виде массива), то следует использовать метод `ПолучитьСоединения()` менеджера клиентских WebSocket-соединений. В качестве результата будет получен массив объектов типа `WebSocketКлиентСоединение`. Свойства данного объекта повторяют параметры метода создания соединения и повторяются мы не будем.

В то же время, у объекта `WebSocketКлиентСоединение` есть методы, которые реализуют стандартный интерфейс клиента:

- `ОтправитьСообщение()` - метод выполняет отправку данных (в виде сообщения) серверу, с которым выполнено соединение.
- `ПолучитьСостояние()` - метод возвращает состояние соединения в данный момент времени. Состояние описывается системным перечислением `СостояниеWebSocketСоединения`.
- `Закрыть()` - закрывает соединение с сервером. Пользоваться этим сокетом больше нельзя.

Стандарт RFC 6455 явно указывает, что в сообщениях может передаваться или текст, или двоичные данные. Других ограничений самим стандартом не накладывается. Однако отмечается, что если при работе по WebSocket используется какое-либо расширение, то это расширение может указывать формат передаваемых данных. С точки зрения программной работы с WebSocket, важно понимать, что по соединению могут быть отправлены любые данные. В том числе оформленные в виде документов JSON или XML. Главное, чтобы эти данные могли быть представлены в виде значения типа `Строка` или `ДвоичныеДанные`. Отправка данных, в простейшем случае, будет выглядеть следующим образом:

[Копировать в буфер обмена](#)

```
ВебСокет.ОтправитьСообщение("строка данных");
```

В данном примере `ВебСокет` - это переменная, которая содержит объект типа `WebSocketКлиентСоединение`.

В процессе работы с сокетом может возникать необходимость проверить, что соединение до сих пор открыто. Для этого можно использовать метод `ПолучитьСостояние()` объекта, который описывает соединение с WebSocket-ом. Метод может вернуть устойчивое состояние: WebSocket открыт или закрыт, а может вернуть информацию о том, что WebSocket находится в промежуточном состоянии: открывается или закрывается. Это происходит потому, что работа с WebSocket является асинхронной. Поэтому при открытии WebSocket-соединения, платформе передаются имена обработчиков событий. Еще одной причиной использования обработчиков обратного вызова является то, что получение данных от сервера - это в принципе асинхронная операция. Клиент не знает, когда сервер решит передать ему какую-то информацию.

Еще одним способом проверить статус соединения является метод `ПолучитьСостояниеСоединения()` объекта `WebSocketКлиент`. Метод возвращает состояние аналогично методу WebSocket-соединения, но не требует получения этого соединения и анализа, что соединение существует.

Обработчики событий WebSocket получают в качестве параметров всю необходимую информацию: объект `WebSocketКлиентСоединение`, для которого вызван обработчик и событие-зависимая вспомогательная информация (полученные данные, код ошибки и т. д.).

Теперь рассмотрим еще один объект, который используется при работе с WebSocket. Это объект `ПараметрыWebSocketКлиентСоединения`, который описывает различные параметры, которые могут потребоваться не каждый раз или не с каждым сервером. Объект данного типа может быть передан в метод `ОткрытьСоединение()` и потом будет доступен через свойства `WebSocketКлиентСоединение.Параметры` (в том числе и в обработчиках событий WebSocket). Объект обладает следующими свойствами:

Свойство	Описание
<code>ДополнительныеПараметры</code>	Тип: произвольный. Данное свойство предназначено для передачи в прикладной код, использующий объект <code>WebSocketКлиентСоединение</code> данных, которые нужны для корректной обработки событий. Такими местами могут быть обработчики событий или какие-то другие участки кода, где происходит получение объекта <code>WebSocketКлиентСоединение</code> по ключу и работа с соединением.
<code>Заголовки</code>	Тип: <code>Соответствие</code> . Данное свойство содержит заголовки, которые платформа добавит к заголовкам HTTP-запроса <code>GET</code> (с заголовком <code>Upgrade: websocket</code>). Этим запросом начинается попытка установки соединения с WebSocket-сервером.
<code>ИспользоватьАутентификациюОС</code>	Тип: <code>Булево</code> . Указывает необходимость использовать аутентификацию ОС на WebSocket-сервере. Есть несколько вариантов указания пользователя аутентификации ОС: • Без указания конкретных имен и паролей. В это случае служба аутентификации операционной системы, используемые на WebSocket-сервере, будут использовать для этой цели того пользователя, от имени которого работает сервис, обслуживающий

	WebSocket-соединение. Свойства Пользователь и Пароль. Это будут имя пользователя и пароль пользователя, который будет принят службой аутентификации той операционной системы, в которой работает WebSocket-сервер. В любом случае, говоря о использовании аутентификации ОС, следует понимать, что это будет пользователь того компьютера/сети (той ОС), где размещен WebSocket-сервер.
ИспользоватьПроксиОС	Тип: Булево. Указывает, что при установке соединения в WebSocket-сервером будут использоваться (значение Ложь) настройки прокси-сервера, указанные в свойстве Прокси. Если свойство установлено в значение Истина, то при установке соединения будут использоваться настройки прокси-сервера, указанные в настройках операционной системы того компьютера, с которого выполняется соединение.
Пароль	Тип: Строка. Указывает пароль пользователя, который используется для стандартной аутентификации или аутентификации ОС на WebSocket-сервере.
Пользователь	Тип: Строка. Указывает имя пользователя, который используется для стандартной аутентификации или аутентификации ОС на WebSocket-сервере.
Прокси	Тип: ИнтернетПрокси. Описывает настройки прокси-сервера, которые будут использоваться при установке соединения, если свойство ИспользоватьПроксиОС установлено в значение Ложь. Это свойство необходимо заполнять в том случае, например, когда прокси-сервер требует авторизации.
Таймаут	Тип: Число. Таймаут установки соединения с WebSocket-сервером, в секундах.

Таким образом, простейший пример открытия WebSocket-соединения и отправки по нему данных может выглядеть следующим образом:

[Копировать в буфер обмена](#)

```
&НаКлиенте
Процедура ОткрытьСокет (Команда)
    ОбработчикиСокета = Новый ОбработчикиWebSocketКлиентСоединения;
    ОбработчикиСокета.Модуль = ЭтотОбъект;
    ОбработчикиСокета.ОбработчикОткрытияСоединения = "ВебСокет_ОбработчикОткрытия";
    ОбработчикиСокета.ОбработчикЗакрытияСоединения = "ВебСокет_ОбработчикЗакрытия";
    ОбработчикиСокета.ОбработчикПолученияСообщения = "ВебСокет_ОбработчикПолученияСообщения";
    ОбработчикиСокета.ОбработчикОшибки = "ВебСокет_ОбработчикОшибки";
    WebSocketКлиентСоединения.ОткрытьСоединение ("websocket-test", "wss://echo.websocket.org",
ОбработчикиСокета);
КонiecПроцедуры

// Обработчики событий веб-сокета
&НаКлиенте
Процедура ВебСокет_ОбработчикОткрытия (сокетСоединение) Экспорт
КонiecПроцедуры

&НаКлиенте
Процедура ВебСокет_ОбработчикПолученияСообщения (сокетСоединение, сокетСообщение) Экспорт
    Сообщить ("Получено сообщение: " + сокетСообщение);
КонiecПроцедуры

&НаКлиенте
Процедура ВебСокет_ОбработчикОшибки (сокетСоединение, КодОшибки, Описание) Экспорт
    Сообщить ("websocket error: " + КодОшибки + ", " + Описание);
КонiecПроцедуры

&НаКлиенте
Процедура ВебСокет_ОбработчикЗакрытия (сокетСоединение, КодЗакрытия) Экспорт
КонiecПроцедуры

// отправка данных в WebSocket
ВебСокет = WebSocketКлиентСоединения.ПолучитьСоединение ("websocket-test");
ВебСокет.ОтправитьСообщение ("Привет, WebSocket!");
```

Немного поясним пример:

- Пример предполагает размещение в модуле формы.

- Код отправки данных следует разместить в каком-либо клиентском обработчике модуля формы.
- Ключ создаваемого соединения: `websocket-test`. Очевидно, что, если необходимо создавать несколько соединений к одному (или разным) серверам - нужно создавать ключи, которые позволят однозначно идентифицировать соединения.
- Адрес тестового WebSocket: <ws://echo.websocket.org>. Доступно и незащищенное соединение: <ws://echo.websocket.org>. Сервера предоставляются сайтом <https://websocket.org/>. WebSocket-сервер выполняет единственную функцию - возвращает данные, которые ему прислал клиент (echo-сервер). Это означает, что после каждой отправки сообщения через соединение, клиент будет получать во входящем сообщении отправленные данные (в случае примера это будет строка `Привет, WebSocket!`).
- Коды состояния (параметры `КодОшибки` или `КодЗакрытия`) при работе с WebSocket можно получить в разделе 7.4 RFC 6455 (<https://datatracker.ietf.org/doc/html/rfc6455#section-7.4>).

17.5.2.3. Использование WebSocket-клиента

В том случае, когда адрес WebSocket-сервера заранее известен и необходимо, чтобы прикладное решение «слушало» сообщения от этого сервера постоянно, можно использовать возможность создания объекта конфигурации `WebSocket-клиенты`. Создание и редактирование объекта выполняется стандартным образом.

Среди свойств объекта выделим следующие свойства:

- **Предопределенный** - такие WebSocket-клиенты нельзя явно создавать и удалять из встроенного языка, а в остальном они не отличаются от WebSocket-клиентов, созданных из встроенного языка. Предопределенный WebSocket-клиент всегда существует в единственном экземпляре, который создается в информационной базе при обновлении конфигурации после того, как в метаданных установлен признак предопределенного WebSocket-клиента. Когда признак предопределенности WebSocket-клиента снимается, в информационной базе (при обновлении конфигурации) удаляется связанный WebSocket-клиент. Начальные значения свойств предопределенного WebSocket-клиента устанавливаются в метаданных.
- **Подключать автоматически** - данное свойство указывает, что подключение к серверу надо выполнять автоматически, при запуске системы «1С:Предприятие» и переустанавливать соединение при разрыве связи. В противном случае выполнять соединение с сервером будет нужно из встроенного языка, с помощью метода `WebSocketКлиент.Подключить()`.
- **URL сервера** - адрес WebSocket-сервера, с которым будет осуществляться работы данным WebSocket-клиентом.

Остальные параметры аналогичны свойствам объекта `ПараметрыWebSocketКлиентСоединения` и имеют аналогичный смысл.

Во время исполнения, объект дерева метаданных `WebSocket-клиент` будет олицетворяться объектом `WebSocketКлиент`. Данный объект доступен только на стороне сервера системы «1С:Предприятие». Так же на стороне сервера доступны и все операции с этим объектом. Способ получения объекта `WebSocketКлиент` будет разным для предопределенного элемента метаданных `WebSocket-клиент` и для не предопределенного элемента.

Если WebSocket-клиент указан как предопределенный и с автоматическим подключением, то никаких отдельных действий для запуска такого клиента предпринимать не требуется. Чтобы получить объект `WebSocketКлиент`, связанный с предопределенным WebSocket-клиентом, необходимо использовать метод `НайтиПредопределенный()` менеджера `WebSocket-клиентов`. Этот менеджер доступен через свойство глобального контекста `WebSocketКлиенты`. Если такого предопределенного WebSocket-клиента нет - метод вернет значение `Неопределено`.

Копировать в буфер обмена

```
сокетКлиент = WebSocketКлиенты.НайтиПредопределенный(Метаданные.WebSocketКлиенты.ПолучениеСобытий);
```

Если WebSocket-клиент указан как не предопределенный, то для получения объекта `WebSocketКлиент` следует использовать метод `СоздатьКлиента()` менеджера `WebSocket-клиентов`. Для метода следует указать объект метаданных, которые описывает создаваемого клиента и ключ создаваемого объекта.

Копировать в буфер обмена

```
сокетКлиент = WebSocketКлиенты.СоздатьКлиента(Метаданные.WebSocketКлиенты.ПолучениеСобытий, "websocket-test");
```

И здесь мы увидим различие между предопределенным и не предопределенным WebSocket-клиентом: для объекта `WebSocketКлиент`, описывающего объект метаданных с признаком предопределенный, значение свойства `Ключ` будет автоматически установлено системой. Во втором случае ключ будет передан во время создания клиента.

Стоит также отметить еще несколько моментов:

- Если нужно создать еще один экземпляр предопределенного WebSocket-клиента, то это можно сделать с помощью метода `СоздатьКлиента()`. Но это будет другой объект относительно того, который создан платформой во время обновления конфигурации информационной базы.
- Фактическое создание объекта `WebSocketКлиент` выполняется после записи этого объекта методом `Записать()`. В этом смысле поведение ничем не отличается от остальных объектов конфигурации.

Если надо получить объект `WebSocketКлиент` с каким-либо ключом - следует использовать метод `WebSocketКлиенты.НайтиПоКлючу()`. Если клиент с таким ключом не создавался - метод вернет значение `Неопределено`. Если надо получить список всех WebSocket-клиентов - нужно использовать метод `WebSocketКлиенты.ПолучитьКлиентов()`.

Теперь, когда у нас так или иначе есть объект, описывающий WebSocket-клиента, мы можем выполнять с этим объектом различные действия, аналогичные тем, что мы выполняли при динамическом создании клиента (см. [здесь](#)). Рассмотрим свойства и методы объекта `WebSocketКлиент`. Начнем со свойств.

Свойство	Описание
<code>URLСервера</code>	Тип: <code>Строка</code> .

	Содержит адрес WebSocket-сервера.
ИмяПользователяИнформационнойБазы	Тип: Строка. Имя пользователя, от имени которого будет выполняться сеанс, в котором будут исполняться обработчик событий. Если имя пользователя не задано, сеанс будет выполняться от имени пользователя по умолчанию.
Ключ	Тип: Строка. Идентификатор объекта WebSocketКлиент, который позволяет однозначно найти этот объект среди подобных. Для клиентов, связанных с предопределенными объектами из метаданных, это свойство заполняется системой «1С:Предприятие».
Метаданные	Тип: ОбъектМетаданных. Описывает объект метаданных, на основании которого создан данный объект WebSocketКлиент.
ПараметрыСоединения	Тип: ПараметрыWebSocketКлиентСоединения. Дополнительные параметры WebSocket-соединения.
ПодключатьАвтоматически	Тип: Булево. Указывает на необходимость автоматического подключения к серверу и восстановления подключения после разрыва соединения.
Предопределенный	Тип: Булево. Указывает на то (если установлено в значение Истина), что данный объект создан на основании предопределенного объекта конфигурации. Иначе - установлено в значение Ложь.

Теперь рассмотрим методы этого объекта:

Метод	Описание
Записать ()	Записывает объект WebSocketКлиент, если в него были внесены какие-то изменения. Позволяет переопределить некоторые данные, описывающие WebSocket-клиента относительно тех, которые заданы в метаданных.
Отключить ()	Выполняет закрытие соединения с сервером. После выполнения метода использовать данное соединение уже невозможно.
Подключить ()	Выполняет подключение к серверу, указанному в свойствах объекта WebSocketКлиент и возвращает объект WebSocketКлиентСоединение или значение Неопределено в тех случаях, когда соединение отсутствует.
ПолучитьСостояниеСоединения ()	Позволяет получить состояние соединения WebSocket-клиента без необходимости получать сам объект WebSocketКлиентСоединение.
ПолучитьТекущееСоединение ()	Получает объект WebSocketКлиентСоединение, когда это необходимо. Если в момент вызова метода WebSocket-соединение еще не установлено, то метод вернет значение Неопределено. В этом случае можно установить соединение с помощью метода WebSocketКлиент.Подключить () .
Удалить ()	Удаляет данные о WebSocket-клиенте из информационной базы.

И последнее, что нам осталось рассмотреть в части WebSocket-клиента, указанного в метаданных, это те события, которые могут быть обработаны с помощью встроенного языка. Обработчики событий находятся в модуле WebSocket-клиента:

Событие	Параметры и описание
ПередПодключением	Клиент: тип WebSocketКлиент Событие вызывается перед тем, как будет выполнена попытка установить соединение. Через параметр можно выполнить дополнительные настройки, такие как формирование заголовков, изменение адреса WebSocket-сервера, получение токена авторизации и т. д.
ПриЗакрытииСоединения	Соединение: тип WebSocketКлиентСоединение КодЗакрытия: тип Строка Обработчик события вызывается после закрытия соединения.
ПриОткрытииСоединения	Соединение: тип WebSocketКлиентСоединение Событие вызывается после того, как установлено соединение к сервером.
ПриОшибке	Соединение: тип WebSocketКлиентСоединение КодОшибки: тип Строка

	Описание: тип Строка Обработчик вызывается в том случае, если при работе соединения произошла какая-либо аварийная ситуация.
ПриПолученииСообщения	Соединение: тип WebSocketКлиентСоединение Сообщение: тип Строка, ДвоичныеДанные Возникает в том случае, когда сервер прислал сообщение. Соединение описывает соединение, через который пришло сообщение. Сообщение - это, собственно, необработанное тело сообщения.

Из всего вышеперечисленного становится ясно, что для создания минимально работоспособного WebSocket-клиента, необходимо:

- Создать объект метаданных [WebSocket-клиент](#).
- Указать для этого объекта свойства [URL сервера](#), [Предопределенный](#) и [Подключать автоматически](#).
- Реализовать обработчик события [ПриПолученииСообщения](#).

Если в процессе работы прикладного решения необходимо отправить какие-либо данные через WebSocket-клиент, описанный в метаданных, то будет необходимо выполнить следующий набор действий на стороне сервера «1С:Предприятие»:

- Получить объект [WebSocketКлиент](#) на основании объекта метаданных каким-либо методом менеджера WebSocket-клиентов.
- Получить у этого клиента его соединение (объект типа [WebSocketКлиентСоединения](#)) с помощью метода [ПолучитьТекущееСоединение\(\)](#).
- Отправить данные через полученный объект с помощью метода [ОтправитьСообщение\(\)](#).

17.6. Внешние источники данных

17.6.1. Общая информация

При эксплуатации информационной системы, построенной на базе «1С:Предприятия», могут возникать задачи, связанные с получением информации из внешних баз данных (как реляционных, так и аналитических) и использованием этой информации в «1С:Предприятии» различным образом, например, в виде отчетов или для каких-либо расчетов. Для решения такого рода задач в «1С:Предприятии» существует объект конфигурации [ВнешниеИсточникиДанных](#). В состав внешнего источника данных могут входить таблицы реляционного источника данных и кубы аналитического источника данных. В прикладном решении может существовать произвольное количество подключенных внешних источников данных. Далее будет более подробно рассмотрена работа с каждым видом внешнего источника данных.

17.6.2. Работа с реляционными внешними источниками данных

17.6.2.1. Общая информация

Объект, описывающий внешний источник данных, подключенный к реляционной базе данных, состоит из таблиц, а каждая таблица состоит из полей. Данные в таблицах могут ссылаться на другие таблицы - например, значение в поле таблицы может являться идентификатором записи в другой таблице. Объект, описывающий внешний источник данных, может быть использован следующим образом:

- в качестве источника данных для запросов;
- в качестве источника данных в системе компоновки данных;
- в качестве источника для динамических списков;
- входить в состав общих реквизитов (см. [здесь](#));
- записи таблиц могут отображаться в управляемых формах «1С:Предприятия» (не поддерживается использование обычных форм для объектов внешних источников данных);
- таблицы внешнего источника данных могут выступать в качестве типов реквизитов информационной базы;
- к таблицам (и полям) внешних источников данных можно применять права доступа и накладывать ограничения доступа к данным;
- доступ к таблицам и полям возможен из встроенного языка;
- таблица внешнего источника данных может входить в состав подсистем (см. [здесь](#));
- таблица внешнего источника данных может входить в состав функциональных опций (см. [здесь](#));
- для таблицы внешнего источника данных можно создавать характеристики (см. [здесь](#)).

Для получения доступа к внешним источникам данных используется механизм ODBC. Данные внешних источников данных доступны как для **чтения**, так и для **записи**.

ВНИМАНИЕ! Механизм внешних источников данных не должен использоваться для доступа к базам данных «1С:Предприятия», так как модель данных «1С:Предприятия» не рассчитана на работу с данными на уровне физических

структур хранения в системах управления базами данных (СУБД). Возможность записи во внешние источники данных не должна использоваться для замены штатного механизма хранения данных прикладного решения.

Возможности системы «1С:Предприятие» максимально учитываются при использовании в качестве источника внешних данных следующих СУБД (свойство [Тип СУБД](#) параметров соединения с источником данных):

- Microsoft SQL Server,
- IBM Db2,
- PostgreSQL,
- Oracle Database,
- MySQL.

ПРИМЕЧАНИЕ. При работе с внешним источником данных могут возникать ошибки в том случае, если свойство [Тип СУБД](#) параметров соединения с внешним источником данных содержит значение, которое не соответствует реально используемой системе.

При использовании других СУБД возможности работы с внешними источниками данных зависят от самой СУБД. Для таких источников данных, следующие функции и выражения языка запросов: [ПОДСТРОКА](#), [ГОД](#), [КВАРТАЛ](#), [МЕСЯЦ](#), [ДЕНЬГОДА](#), [ДЕНЬ](#), [НЕДЕЛЯ](#), [ДЕНЬНЕДЕЛИ](#), [ЧАС](#), [МИНУТА](#), [СЕКУНДА](#), [РАЗНОСТЬДАТ](#), [ЕСТЬNULL](#), [ВЫРАЗИТЬ](#), [СПЕЦСИМВОЛ](#), [СОЕДИНЕНИЕ](#), [ЛЕВОЕ СОЕДИНЕНИЕ](#), [ПРАВОЕ СОЕДИНЕНИЕ](#), [ПОЛНОЕ СОЕДИНЕНИЕ](#), преобразуются в запрос к СУБД с использованием управляющих последовательностей ODBC (ODBC Escape Sequences, [http://msdn.microsoft.com/en-us/library/windows/desktop/ms715364\(v=vs.85\).aspx](http://msdn.microsoft.com/en-us/library/windows/desktop/ms715364(v=vs.85).aspx), на английском языке).

В случае использования внешнего источника данных в качестве источника для динамического списка, следует помнить, что упорядочивание динамического списка выполняется с использованием порядка сортировки, который установлен для базы данных, выступающей в качестве внешнего источника данных. Порядок сортировки может не совпадать с тем порядком сортировки, который используется в том случае, если база данных используется собственно «1С:Предприятием». В связи с этим возможны ситуации, когда результат упорядочивания в информационной базе и во внешнем источнике данных будет отличаться (при одинаковых упорядочиваемых данных). Такое поведение является нормальным.

Для осуществления подключения к внешнему источнику данных следует сформировать строку подключения, которая может содержать в себе либо все параметры подключения, необходимые для выбранного драйвера ODBC, либо указание на сформированное описание источника данных DSN (Data Source Name). Подробнее о строке подключения см. [здесь](#).

Ниже приводятся примеры строк подключения.

Строка соединения с указанием логина и пароля доступа:

```
DRIVER={SQL Server};SERVER=(local);UID=user;PWD=password;DATABASE=AdventureWorksLT2008
```

[Копировать в буфер обмена](#)

Логин и пароль доступа требуется указывать дополнительно:

```
DRIVER={SQL Server};SERVER=CASH_SERVER;DATABASE=CashReceipts
```

[Копировать в буфер обмена](#)

Указание предварительно настроенного источника данных (DSN):

```
DSN=MyDB
```

[Копировать в буфер обмена](#)

Также следует помнить, что соединение с внешним источником данных следует указывать не только в конфигураторе (если используется механизм импорта структуры таблиц из внешнего источника данных), но и в режиме [1С:Предприятие](#) для получения собственно данных.

17.6.2.2. Общая схема использования

Для того чтобы использовать информацию из внешних источников данных в системе на базе «1С:Предприятия», следует придерживаться следующей общей схемы:

- Изучить структуру внешнего источника данных и понять, какая информация (таблицы и поля) необходима для работы алгоритмов или отчетов в «1С:Предприятии».
- Создать объект [Внешние источники данных](#) в конфигураторе и для него создать подчиненные таблицы и поля. Это можно сделать с помощью специального помощника.
- Реализовать использование созданных объектов в прикладном решении.
- Выполнить настройку параметров подключения к внешнему источнику данных в той сети, где будет использоваться прикладное решение. Эти параметры могут отличаться от тех, которые использовались при загрузке структуры внешнего источника данных.

ПРИМЕЧАНИЕ. Параметры доступа к внешнему источнику данных, которые были заданы в конфигураторе, не будут использованы системой в режиме 1С:Предприятие.

17.6.2.3. Редактирование структуры внешнего источника данных

17.6.2.3.1. Общая информация

Для работы с таблицами внешних источников данных предназначена ветвь с именем соответствующего внешнего источника данных ветви **Внешние источники данных** - **<Имя внешнего источника данных>** - **Таблицы** дерева конфигурации. Определение внешнего источника данных состоит из следующих шагов:

1. Определение собственно внешнего источника данных;
2. Определение таблиц источника данных;
3. Определение полей для каждой таблицы внешнего источника данных.

При этом в конфигурации может быть описано меньшее количество таблиц и полей в них, чем есть в реальной базе данных, но нельзя задать таблицу или поле, которое отсутствует в реальной базе данных.

Создание структуры внешнего источника данных возможно как в ручном режиме, так и при загрузке этой структуры с помощью специального конструктора.

17.6.2.3.2. Внешний источник данных

При создании внешнего источника данных следует указать его имя. Этот объект предназначен для идентификации группы таблиц при обращении к данным, которые в нем содержатся. Источник данных состоит из одной или нескольких таблиц, которые, в свою очередь, состоят из полей.

Если во внешний источник данных предполагается запись, то следует установить нужное значение в свойстве **Режим управления блокировкой данных**. Если для внешнего источника данных режим управления блокировок установлен в значение **Автоматический и управляемый**, то для каждой таблицы режим блокировок определяется значением соответствующего свойства таблицы. Если для внешнего источника данных выбран определенный конкретный режим управления блокировками, то одноименное свойство таблицы внешнего источника данных игнорируется.

17.6.2.3.3. Таблица внешнего источника данных

При создании новой таблицы открывается окно редактирования объекта (см. [здесь](#)).

Свойство таблицы **Имя** служит для идентификации таблицы внутри прикладного решения. Для того чтобы система знала, какой физической таблице внешнего источника данных соответствует объект конфигурации, следует заполнить свойство **Имя в источнике данных**. Значение в этом свойстве в точности должно соответствовать имени таблицы внешнего источника данных. Не поддерживается размещение в одной таблице прикладного решения данных из нескольких физических таблиц внешнего источника данных.

Свойство **Тип данных таблицы** определяет, какие сущности в этой таблице хранятся: объектные или неobjектные. Если в таблице можно выделить одно поле, которое однозначно определяет запись в таблице, значит, таблица может хранить объектные данные. Ближайшим аналогом таких таблиц может служить справочник. Если запись в таблице идентифицируется несколькими ключевыми полями, то такая таблица содержит неobjектные данные. Ближайшим аналогом таких таблиц служит регистр сведений. Ключевые поля таблицы необходимо указать в свойстве **Поля ключа**.

ПРИМЕЧАНИЕ. Не рекомендуется использовать в качестве ключевых полей объектной таблицы внешнего источника данных поля, которые содержат значения **NULL**.

Для таблиц, содержащих объектные данные, можно указать поле, которое будет выступать в качестве представления объекта. Это можно сделать с помощью свойства **Поле представления**.

В зависимости от того, какой объект СУБД описывает создаваемый объект в «1С:Предприятие», следует корректно установить свойство **Вид таблицы**. Если свойство установлено в значение **Таблица** - то объект конфигурации будет выступать аналогом реальной таблицы базы данных или представления (view). В остальных случаях свойство **Вид таблицы** следует установить в значение **Выражение**. Если таблица внешнего источника данных имеет вид **Таблица**, то в свойство **Имя в источнике данных** необходимо указать имя реальной таблицы или не параметризованного представления, например **dbo.ErrorLog**. Для таблицы вида **Выражение**, следует указать свойство **Выражение в источнике данных**. В простейшем случае это будет вызов функции, возвращающей табличные данные, например **dbo.ufnGetContactInformation(&1)**. Если в это поле записывается выражение, то рекомендуется указать его в скобках, для минимизации различных конфликтов при реальном выполнении запроса к СУБД.

Если таблица хранит иерархические данные, то системе можно сообщить об этом с помощью свойств **Поле родителя**, **Признак незаполненного родителя** и **Значение незаполненного родителя**. Эти свойства доступны только для таблиц, в которых хранятся объектные данные. Свойство **Поле родителя** служит для указания поля таблицы базы данных, которое хранит ссылку на родителя для данной записи. Поля, которые могут быть выбраны в данном свойстве, должны иметь тип ссылки на используемую таблицу. Для таблицы **products** из примера (см. [здесь](#)) поле родителя должно иметь тип **ВнешнийИсточникДанныхТаблицаСсылка.DBF.products**. Свойства **Признак незаполненного родителя** и **Значение незаполненного родителя** определяют, какое значение будет определять запись, у которой отсутствует родительская запись. После указания данных свойств:

- система компоновки данных использует их при работе с иерархическими группировками и при проверке условий иерархии;
- язык запросов использует их для работы конструкции **В ИЕРАРХИИ**;
- эти свойства используются при отображении таблиц, хранящих иерархические данные (возможности отображения аналогичны возможностям, которые используются при отображении иерархических справочников).

Свойство **Ввод по строке** (см. [здесь](#)) позволяет указать имена полей, по которым будет выполняться поиск в поле, тип данных которого указывает на таблицу внешнего источника данных.

При ручном создании таблиц свойства **Поле ключа**, **Поле представления** и **Ввод по строке** можно указать только после создания полей таблицы внешнего источника данных.

За возможность записи в таблицу внешнего источника данных отвечает флажок **Только чтение**. Если флажок сброшен - в указанную таблицу допускается запись информации. Флажок устанавливается в значение **Истина** автоматически для таблиц, основанных на представлениях (view) и функциях (см. [здесь](#)). Для таблицы, в которую допустима запись информации, следует обратить внимание на следующие свойства:

- **Уровень изоляции транзакций** - определяет уровень изоляции транзакций, которые будут устанавливаться при неявных транзакциях записи в эту таблицу (см. [здесь](#)).
- **Поле версии данных** - в данном свойстве указывается поле внешнего источника данных, которое увеличивает свое значение автоматически, при каждой записи в данную таблицу. Если поле указано, то система при интерактивном изменении объекта опирается на значение этого поля. При записи выполняется сравнение значения этого поля в базе данных и в форме: если значения различаются, значит, объект изменен.

Если поле не указано, то при интерактивном открытии формы происходит считывание объекта в память, а при записи происходит повторное чтение объекта и сравнение с копией в памяти. Если две копии различаются, значит, объект изменен.

- **Поля блокировки данных** - указывает поля, по которым можно выполнять блокировку данных. Подробнее см. [здесь](#).
- **Вводится на основании** - указываются объекты конфигурации, на основании которых могут создаваться записи в таблице.
- **Режим управления блокировкой данных** - указывает, какой режим управления блокировками будет применяться при записи в данную таблицу (см. [здесь](#)).

17.6.2.3.4. Поле таблицы внешнего источника данных

Поля таблицы описывают, какие данные физической таблицы будут доступны из прикладного решения. Свойство поля **Имя** служит для идентификации поля таблицы в прикладном решении. Для указания соответствия между полем прикладного решения и полем физической таблицы служит свойство **Имя в источнике данных**. Значение в этом свойстве должно в точности соответствовать имени колонки таблицы, указанному в свойстве **Имя в источнике данных** родительского объекта данного поля. Не поддерживается объединение в одном поле таблицы данных из нескольких колонок одной или нескольких таблиц внешнего источника данных.

Если значение, находящееся в свойстве **Имя в источнике данных** заключено в одинарные кавычки, то в SQL-запрос к базе данных это значение попадает без преобразований, вне зависимости от состава символов. Если значение в поле не заключено в одинарные кавычки, то в SQL-запрос к базе данных это значение попадет заключенное в двойные кавычки, если в имени содержатся спецсимволы.

Поле **Тип** позволяет указать тип данного поля. Для выбора доступен ограниченный набор типов. Типами поля могут быть:

- **Число**;
- **Строка**;
- **Дата**;
- **Булево**;
- **УникальныйИдентификатор**;
- **ДвоичныеДанные**;
- типы, которые определяются таблицами внешних источников данных.

Если необходимо указать для поля составной тип, то в таком составном типе могут участвовать только типы **Число**, **Строка**, **Дата**, **Булево**.

При получении данных из внешних источников система автоматически преобразует эти данные к тому типу, который указан у соответствующего поля в прикладном решении. Описание правил преобразования см. [здесь](#).

Свойство **Только чтение** указывает, что данное поле в таблице не может быть записано. Это свойство следует устанавливать в значение **Истина** для полей с автоматическим изменением, автоматически формируемые ключевые поля, вычисляемых полей и т. д.

Однако при работе с внешним источником данных возникает необходимость выполнять запись полей, которые в обычной работе предназначены только для чтения, или наоборот. При этом такая необходимость является временной, например, это необходимо для выполнения какой-либо однократной или регламентной операции. Для изменения списка записываемых полей следует использовать методы **ПолучитьИзменяемыеПоля()** и **УстановитьИзменяемыеПоля()**. Таким образом, состояние свойства **Только чтение** описывает поведение поля по умолчанию, а с помощью указанных методов можно в редких случаях изменять поведение поля.

Свойство **Разрешить Null** указывает, можно или нет в поле таблицы записать **NULL**. Если в поле таблицы записано **NULL**, то в форме это значение будет отображаться как **Не заполнено**.

17.6.2.3.5. Прочие свойства

Форма, используемая для отображения записи и списка таблицы внешнего источника данных, может быть создана системой автоматически или прикладным разработчиком. При этом следует помнить, что если таблицы является необъектной и не заданы ключевые поля, то:

- отображение формы записи невозможно;

- невозможен доступ к объекту [ВнешнийИсточникДанныхТаблицаНаборЗаписей](#).

Это связано с тем, что невозможно однозначно идентифицировать необходимый набор записей в таблице.

17.6.2.3.6. Работа с транзакциями

При записи во внешние источники данных могут использоваться как автоматические, так и управляемые блокировки (см. [здесь](#)).

Для работы с транзакциями предназначены методы [НачатьТранзакцию\(\)](#), [ЗафиксироватьТранзакцию\(\)](#), [ОтменитьТранзакцию\(\)](#) и [ТранзакцияАктивна\(\)](#) менеджера внешнего источника данных. В частности, это означает, что недоступны транзакции для нескольких источников данных (распределенные транзакции).

Управление блокировками осуществляется с помощью следующих свойств и объектов:

- Свойство [Режим управления блокировкой данных](#) внешнего источника данных;
- Свойство [Режим управления блокировкой данных](#) таблицы внешнего источника данных;
- Полями, указанными в свойстве [Поля блокировки данных](#) таблицы внешнего источника данных;
- Свойством [Уровень изоляции транзакций](#) таблицы внешнего источника данных;
- Параметрами метода [НачатьТранзакцию\(\)](#) менеджера внешних источников данных.

Свойство [Режим управления блокировкой данных](#) внешнего источника данных предназначено для установки по умолчанию режима блокировок таблиц внешнего источника данных (аналогично свойству конфигурации [Режим управления блокировкой данных](#), см. [здесь](#)). Данное свойство может принимать следующие значения:

- [Автоматический](#) - режим автоматических блокировок для всех таблиц внешнего источника данных. В качестве уровня изоляции транзакций используется [Упорядочиваемость](#). Такой режим позволяет использовать транзакции к базе данных внешнего источника одновременно из платформы и из других клиентов. В данном режиме одноименное свойство у таблицы внешнего источника данных игнорируется.
- [Управляемый](#) - режим управляемых блокировок для всех таблиц внешнего источника данных. В качестве уровня изоляции транзакций используется [Чтение зафиксированных](#). Такой режим можно использовать в том случае, когда к базе данных внешнего источника имеет только система на базе «1С:Предприятие». В данном режиме игнорируется одноименное свойство у таблицы внешнего источника данных.
- [Автоматический и управляемый](#) - режим блокировок выбирается для каждой таблицы в зависимости от одноименного свойства таблицы. Уровень изоляции транзакций выбирается в зависимости от свойства [Уровень изоляции транзакций](#) таблицы внешнего источника данных.

Если свойство [Уровень изоляции транзакций](#) таблицы внешнего источника данных установлено в значение [Авто](#), то фактический режим изоляции транзакций устанавливается в зависимости от свойства [Режим управления блокировкой данных](#) (описано выше). Если указано конкретное свойство - будет использоваться именно оно. Если СУБД внешнего источника данных не поддерживает указанный в этом свойстве уровень изоляции транзакций, то уровень будет автоматически изменяться по следующей цепочке (до первого поддерживаемого): [Чтение незафиксированных](#) à [Чтение зафиксированных](#) à [Повторяемое чтение](#) à [Упорядочиваемость](#).

При работе в режиме управляемых блокировок, блокировки на уровне базы данных накладываются установкой соответствующего уровня изоляции транзакций и указанием выражения [для ИЗМЕНЕНИЯ](#) в запросах к базе данных.

Метод [ТранзакцияАктивна\(\)](#) позволяет определить текущее состояние транзакции используемого внешнего источника данных.

17.6.2.3.7. Особенности поведения системы при записи во внешние источники данных

При не интерактивной работе с внешним источником данных следует учитывать следующие особенности поведения:

- При создании записи в таблице внешнего источника данных:
 - Изменить реквизиты записываемого объекта, у которых свойство [ТолькоЧтение](#) установлено в значение [Истина](#), невозможно.
 - Если у реквизита, входящего в состав полей ключа, свойство [ТолькоЧтение](#) установлено в значение [Истина](#), то такой реквизит может быть изменен только в результате вызова метода [УстановитьСсылкуНового\(\)](#). Если у такого реквизита свойство [ТолькоЧтение](#) установлено в значение [Ложь](#), то его значение можно изменять явным образом (с помощью операции присваивания).
 - Система не занимается автоматическим заполнением значений реквизитов, участвующих в формировании ключа объекта (или набора записей). Это означает, что запись будет завершена с ошибкой, если для записываемого объекта не установлены значения ключевых реквизитов.
- При изменении записи в таблице внешнего источника данных:
 - Изменяются только те реквизиты, для которых свойство [ТолькоЧтение](#) установлено в значение [Ложь](#).
 - При изменении ключевых полей изменяется значение ссылки.
 - Если заполнен реквизит [ВерсияДанных](#), то это значение используется в выражении [WHERE](#) для запроса обновления данных. После успешного выполнения запроса значение реквизита повторно считывается из базы данных.

- При удалении записи из таблицы внешнего источника данных:
 - Выполняется только непосредственное удаление. Пометка удаления недоступна и не поддерживается.
 - При удалении не выполняется контроль ссылочной целостности.

При использовании в качестве внешнего источника данных СУБД PostgreSQL, не поддерживается запись полей типа **УникальныйИдентификатор**.

При записи уникального идентификатора в поле таблицы внешнего источника данных платформа «1С:Предприятие» выполняет перестановку данных уникального идентификатора. Таким образом, данные во внешнюю базу данных будут помещаться в измененном виде. При чтении уникального идентификатора данные переставляются обратно. Поэтому, при записи/чтении уникальный идентификатор сохраняет свое исходное значение.

Хранение значение типа **УникальныйИдентификатор** в памяти компьютера можно представить следующим образом (порядок байт «младший-старший»):

- значение **Data1** длиной 4 байта;
- значение **Data2** длиной 2 байта;
- значение **Data3** длиной 2 байта;
- значение **Data4** длиной 8 байта.

Запись в базу данных значения типа **УникальныйИдентификатор** для такого хранения будет выполняться в следующем порядке:

- целиком значение **Data4**;
- 1-й байт значения **Data3**;
- 0-й байт значения **Data3**;
- 1-й байт значения **Data2**;
- 0-й байт значения **Data2**;
- 3-й байт значения **Data1**;
- 2-й байт значения **Data1**;
- 1-й байт значения **Data1**;
- 0-й байт значения **Data1**.

17.6.2.3.8. Загрузка структуры таблиц из внешнего источника данных

Существует возможность загрузки структуры таблиц из внешнего источника данных, который доступен для разработчика (например, копии реальной базы данных).

Для выполнения этой операции следует при создании таблицы указать пункт **Выбрать из списка таблиц внешнего источника данных** в конструкторе таблиц внешнего источника данных. Затем нужно указать строку соединения с внешней базой данных, воспользовавшись окном **Подключение к источнику данных**. При формировании строки подключения можно воспользоваться конструктором строки соединения с внешним источником данных. Для этого следует нажать кнопку «...» справа от поля **Строка соединения**.

После того как подключение к внешней базе успешно выполнено, будет открыт список с перечнем таблиц и полей подключенного источника данных. Затем следует выбрать те таблицы и поля, которые будут использованы конфигуратором для создания структуры объектов, описывающих текущий источник данных.

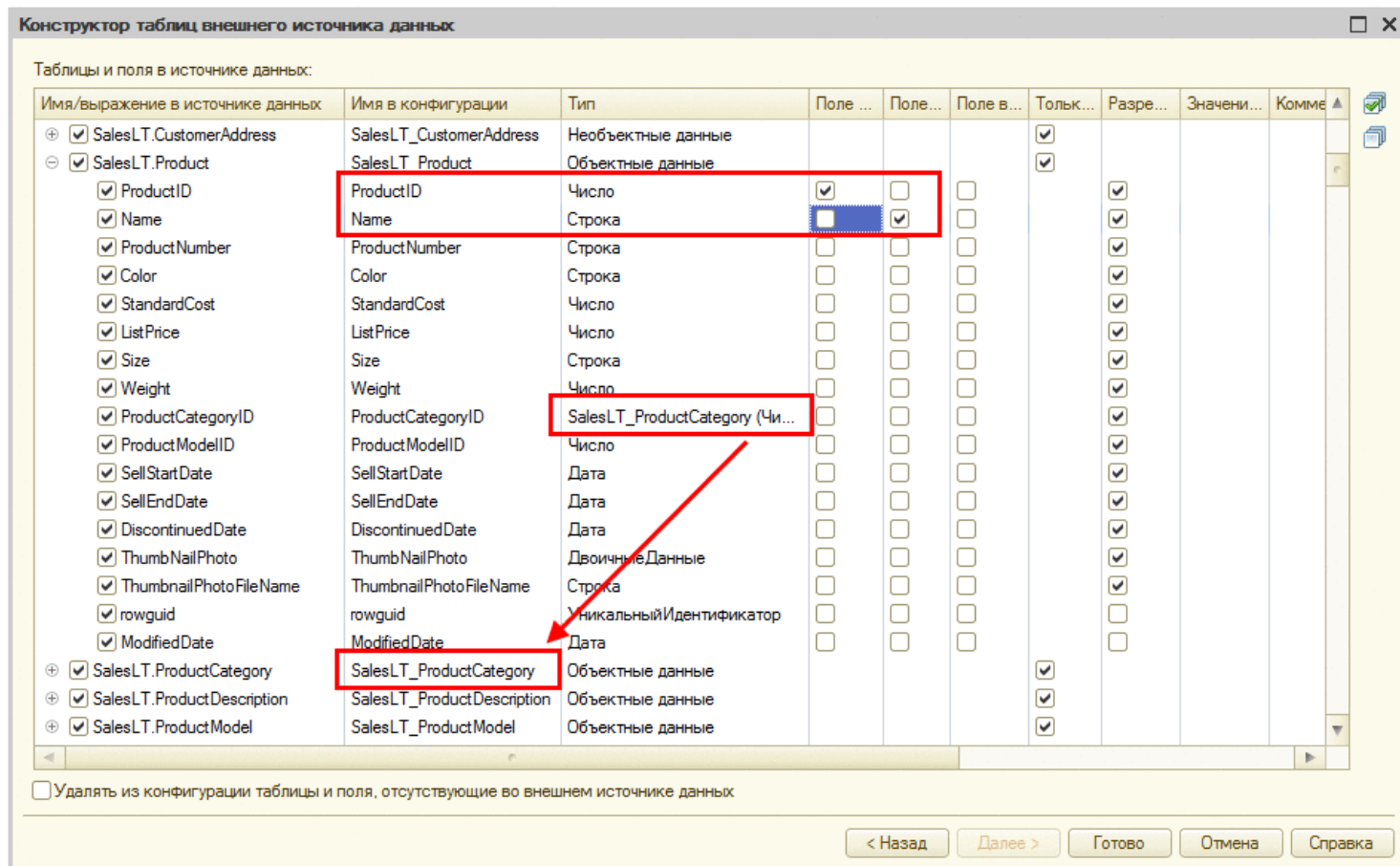


Рис. 511. Конструктор таблиц внешнего источника данных

При получении структуры внешнего источника данных система выполняет следующие действия:

- Выполняет попытку определить тип таблицы с данными: объектные или необъектные данные расположены в таблице. Таблица будет считаться объектной, если для нее указано только одно ключевое поле, и необъектной в противном случае. Если система ошиблась с указанием ключевых полей, можно вручную изменить состав полей, которые образуют ключ таблицы. Если таблица определена как объектная, у нее можно указать поле, которое формирует представление данных такого типа. Поле представления необходимо указать вручную.
- Преобразует типы колонок источника данных в типы «1С:Предприятия», которые будут использоваться для указания типов реквизитов таблиц. Все типы из внешнего источника данных преобразуются к следующим типам «1С:Предприятия»: число, строка, дата, булево, уникальный идентификатор, двоичные данные и типы, связанные с объектными таблицами источников данных.
- Выполняет попытку определить типы для полей таблиц. В этом случае система пытается определить, какого типа данные хранятся в колонке таблицы, и в том случае, если это можно считать ссылкой на данные другой таблицы, указывает в колонке соответствующий тип. Если система неправильно выбрала тип колонки таблицы, его можно изменить вручную.
- Во время настройки загрузки, имеется возможность настроить следующие параметры загружаемой структуры данных:
 - Имя объекта в конфигурации - колонка [Имя в конфигурации](#);
 - Тип загружаемого реквизита - колонка [Тип](#);
 - Указать поля, входящие в состав ключа (для объектных данных) - колонка [Поле ключа](#);
 - Указать поле, хранящее представление объекта - колонка [Поле представления](#);
 - Указать поле, хранящее версию объекта (для осуществления оптимистической блокировки данных) - колонка [Поле версии](#);
 - Указать, что таблица или реквизит доступны только на чтение - колонка [Только чтение](#);
 - Указать возможность указывать значение `NULL` в реквизите - колонка [Разрешить NULL](#);
 - Задать значение заполнения для реквизита - колонка [Значение заполнения](#).

Затем следует отметить флажками таблицы и поля, которые будут перенесены в метаданные прикладного решения.

Если флажок **Удалять из конфигурации таблицы и поля, отсутствующие во внешнем источнике данных** установлен, то при завершении работы конструктора таблиц из конфигурации будут удалены те таблицы и поля, которые отсутствуют во внешнем источнике (например, во внешнем источнике таблицы или поля были удалены) данных.

После нажатия кнопки **Готово** произойдет загрузка структуры внешнего источника данных.

17.6.2.4. Пример создания внешнего источника данных

Допустим, у нас существует база данных формата dbf, доступ к которой необходимо получить из прикладного решения.

База данных состоит из трех таблиц (файлов):

- Таблица `products` (хранится в файле `products.dbf`), которая содержит информацию о товарах и состоит из следующих полей:
 - `id` - идентификатор товара (ключевое поле). Тип: `Число`.
 - `code` - код товара. Тип: `Строка`.
 - `name` - наименование товара. Тип: `Строка`.
 - `article` - артикул товара. Тип: `Строка`.
- Таблица `price` (хранится в файле `price.dbf`), которая содержит текущие цены на товары и состоит из следующих полей:
 - `product` - идентификатор товара, для которого хранится цена. Тип: `Число`.
 - `price` - цена товара. Тип: `Число`.
- Таблица `sales` (хранится в файле `sales.dbf`), которая содержит данные о каждой продаже товара с указанием цены, количества и суммы. Таблица состоит из следующих полей:
 - `product` - идентификатор продаваемого товара. Тип: `Число`.
 - `price` - отпускная цена товара. Тип: `Число`.
 - `qty` - количество проданного товара. Тип: `Число`.
 - `summa` - отпускная стоимость проданного товара. Тип: `Число`.

Создадим необходимые объекты конфигурации, чтобы иметь доступ ко всем данным этих таблиц.

Вначале создадим собственно внешний источник данных. Назовем его `DBF`. Затем вручную создадим таблицы источника данных. При этом следует помнить, что реальное имя физической таблицы внешнего источника данных указывается в свойстве `Имя в источнике данных` создаваемого объекта. При создании таблиц назовем объекты конфигурации следующим образом:

- таблица `products` будет отображаться в объект `Товары`;
- таблица `price` будет отображаться в объект `Цены`;
- таблица `sales` будет отображаться в объект `Продажи`.



Рис. 512. Таблицы внешнего источника данных

Теперь следует создать поля для каждой таблицы, указать типы полей и задать поля ключей и представлений (если они есть).

В примере будет одна объектная таблица (таблица `Товары`). Остальные таблицы будут неobjектные. Соответствие полей физических таблиц и полей объекта конфигурации, а также типы полей можно посмотреть на [рис. 513](#).

Имя в источнике д...	Имя в конфигу...	Тип	Поле ...	По...	Комментар
price	Цены	Необъектные данные			Цены това...
Product	Товар	ВнешнийИсточникДанныхТаблицаСсылка.DBF.Товар...	☒		
Price	Цена	Число	☐		
products	Товары	Объектные данные			Список тов
id	Идентификатор	Число	☒	☐	
Code	Код	Строка	☐	☐	
Name	Наименование	Строка	☐	☒	
Article	Артикул	Строка	☐	☐	
sales	Продажи	Необъектные данные			Продажи тс
Product	Товар	ВнешнийИсточникДанныхТаблицаСсылка.DBF.Товар...	☒		
Price	Цена	Число	☐		
Qty	Количество	Число	☐		
Summa	Сумма	Число	☐		

Рис. 513. Структура таблиц в прикладном решении

Теперь следует указать, в какую подсистему входят созданные таблицы и другие параметры объектов конфигурации (при необходимости).

Если затем запустить систему в режиме 1С:Предприятие и корректно указать параметры соединения с внешним источником данных, то в панели навигации будут присутствовать созданные таблицы.

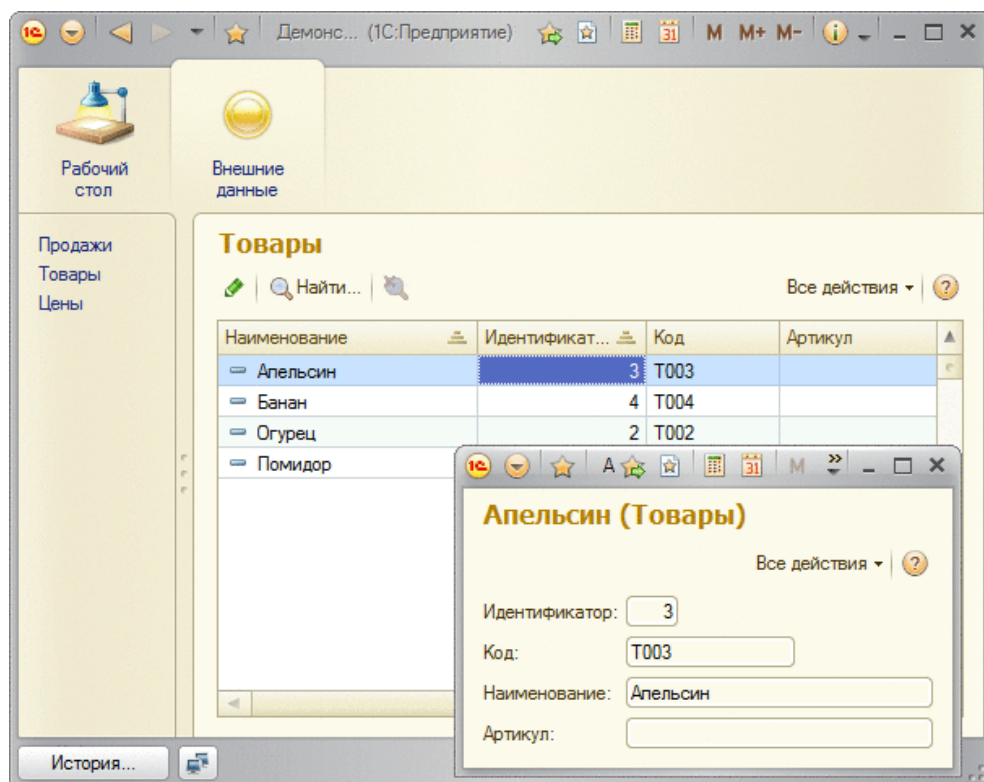


Рис. 514. Динамический список внешнего источника данных

17.6.2.5. Строка соединения ODBC

17.6.2.5.1. Общая информация

Для осуществления подключения к внешнему источнику данных следует сформировать строку соединения, которая может содержать в себе либо все параметры подключения, необходимые для выбранного драйвера ODBC, либо указание на сформированное описание источника данных DSN (Data Source Name).

17.6.2.5.2. Полная строка соединения

Описание строки соединения можно получить по следующему адресу (на английском языке): <http://msdn.microsoft.com/en-us/library/ms722656.aspx>. Из этого описания следует выделить следующее:

- Параметры строки соединения записываются парами **КлючевоеСлово=Значение** и разделяются символом ";".
- Если в строке соединения встречаются несколько параметров с одинаковым ключевым словом, то использоваться будет последнее указание параметра в строке соединения. Поэтому, если такие параметры как **Имя пользователя** или **Пароль** указаны и в строке соединения и в диалоге **Параметры соединения** (см. [здесь](#)), то будут использоваться значения из диалога **Параметры соединения**, т. к. эти параметры будут добавлены с правой стороны к указанной строке соединения.

Для упрощения формирования строки соединения можно воспользоваться сайтом <http://www.connectionstrings.com/>.

17.6.2.5.3. Описание источника данных

Вместо указания полной строки соединения, можно воспользоваться специальными утилитами, которые могут сформировать описание источника данных (DSN) и указывая в параметрах соединения специальную конструкцию вида `DSN=ИмяDSN`. Описание источника данных могут быть пользовательскими и системными. Пользовательские описания источников данных доступны на данном компьютере и только тому пользователю, который создавал этого описание. Системное описание создается системным администратором и доступно всем пользователям данного компьютера (при наличии соответствующих разрешений).

17.6.2.5.4. Утилита администрирования источников данных ODBC

Для создания описания источника данных следует воспользоваться утилитой администрирования источников данных ODBC. В ОС Windows эта утилита расположена следующим образом: [Панель управления - Администрирование - Источники данных \(ODBC\)](#). При этом из панели управления Windows открывается утилита администрирования, соответствующая версии операционной системы. В ОС Linux эта утилита называется [ODBCConfig](#) и доступна при установке соответствующего пакета (зависит от версии ОС Linux).

Следует помнить, что в 64-разрядной версии ОС Windows драйвера и источники данных различаются для 32-разрядного и 64-разрядного варианта ODBC. Поэтому для создания описания источника данных следует использовать утилиту администрирования той разрядности, которая соответствует разрядности системы «1С:Предприятия», которая будет исполнять запросы к внешнему источнику данных. Так, например, если обращение к ODBC выполняется из 64-разрядной версии сервера «1С:Предприятия», то надо использовать 64-разрядную версию утилиты администрирования. Если используется 32-разрядная версия «1С:Предприятия» на 64-разрядной ОС Windows, то следует использовать 32-разрядную версию утилиты администрирования.

На 64-разрядной версии ОС Windows утилиты администрирования расположены следующим образом:

- 64-разрядная версия: `%SYSTEMROOT%\System32\odbcad32.exe`.
- 32-разрядная версия: `%SYSTEMROOT%\SysWOW64\odbcad32.exe`.

17.6.3. Работа с внешним источником данных OLAP

17.6.3.1. Общая информация

OLAP - это технология обработки данных, заключающаяся в подготовке агрегированной информации на основе больших массивов данных, структурированных по многомерному принципу. Данные в аналитических базах данных (OLAP-системах) формируются на основании данных систем транзакционной обработки данных (также называемых OLTP-системами). Информация в OLAP-системах представлена в виде пространства (называемого **кубом**), оси которого представляют собой **измерения**, а в узлах этого пространства располагаются некоторые **меры** (measures). Каждое измерение куба характеризуется определенными **членами** (members) измерения.

Ближайшим (но не точным!) аналогом хранения данных в OLAP-системах можно считать регистр накопления, где сам регистр подобен кубу, измерение типа [СправочникСсылка](#) подобно измерению куба, сам справочник содержит члены измерения, а ресурсы регистра подобны мерам куба.

В системе «1С:Предприятие» куб OLAP-системы представлен в виде модели, где:

- Куб представлен кубом в конфигурации;
- Измерения и члены измерений представлены в виде двух объектов:
 - Измерения являются аналогом измерений,
 - Таблицы измерений служат для описания коллекции членов измерений.
- Меры представлены ресурсами.

Таким образом, в состав внешнего источника, который подключается к OLAP-системе, входят кубы, каждый куб состоит из таблиц измерений, собственно измерений и ресурсов. Таблицы измерений, в свою очередь, состоят из полей. Ресурсы кубов могут быть числовыми или строковыми.

Объект, описывающий внешний источник данных, может быть использован следующим образом:

- в качестве источника данных для запросов;
- в качестве источника данных в системе компоновки данных;
- в качестве источника для динамических списков;
- входить в состав общих реквизитов (см. [здесь](#));
- записи таблиц измерений и кубов могут отображаться в управляемых формах «1С:Предприятия» (не поддерживается использование обычных форм для объектов внешних источников данных);
- таблицы измерений внешнего источника данных могут выступать в качестве типов реквизитов информационной базы;
- к кубам, таблицам измерений, полям таблицы измерений, измерениям и ресурсам внешних источников данных можно применять права доступа;
- доступ к кубам, таблицам измерений, полям таблицы измерений, измерениям и ресурсам возможен из встроенного языка;
- кубы и таблицы измерений внешнего источника данных могут входить в состав подсистем (см. [здесь](#));

- кубы, таблицы измерений и измерения внешнего источника данных может входить в состав функциональных опций (см. [здесь](#));
- для кубов внешнего источника данных можно создавать характеристики (см. [здесь](#)).

Для работы с многомерными внешними источниками данных используется механизм XML для аналитики (XML for Analysis, XMLA). Платформа получает доступ к данным с помощью HTTP-запросов к веб-серверу, который может быть внешним (по отношению к OLAP-системе) или встроенным в OLAP-сервер. Вопросы настройки доступа к данным OLAP-сервера следует искать в документации к используемой системе.

Строка соединения с аналитическим внешним источником данных представляет собой комбинацию URL поставщика XMLA (для данной OLAP-системы) и параметров, которые используются «1С:Предприятием» для работы с источником. Данные внешних источников данных доступны только для чтения.

Возможности системы «1С:Предприятие» максимально учитываются при использовании в качестве внешнего источника данных следующих OLAP-систем (свойство [Тип СУБД](#) параметров соединения с источником данных):

- IBM Infosphere Warehouse,
- Microsoft SQL Server Analysis Services,
- Oracle Essbase.

ПРИМЕЧАНИЕ. При работе с внешним источником данных могут возникать ошибки в том случае, если свойство [Тип СУБД](#) параметров соединения с внешним источником данных содержит значение, которое не соответствует реально используемой системе.

При использовании других СУБД возможности работы с внешними источниками данных зависят от самой СУБД. Для осуществления подключения к внешнему источнику данных следует сформировать строку подключения. Подробнее о строке подключения см. [здесь](#).

Также следует помнить, что соединение с внешним источником данных следует указывать не только в конфигураторе (если используется механизм импорта структуры таблиц из внешнего источника данных), но и в режиме 1С:Предприятие для получения собственно данных.

Документация по OLAP-системам:

- IBM Infosphere Warehouse:
 - http://publib.boulder.ibm.com/infocenter/db2luw/v9r5/topic/com.ibm.dwe.navigate.doc/welcome_db2warehouse.html (на английском языке).
- Microsoft SQL Server Analysis Services:
 - Версия 2008: [http://technet.microsoft.com/ru-ru/library/bb522607\(v=sql.100\).aspx](http://technet.microsoft.com/ru-ru/library/bb522607(v=sql.100).aspx);
 - Версия 2008 R2: [http://technet.microsoft.com/ru-ru/library/bb522607\(v=sql.105\).aspx](http://technet.microsoft.com/ru-ru/library/bb522607(v=sql.105).aspx);
 - Версия 2012: [http://technet.microsoft.com/ru-ru/library/bb522607\(v=sql.110\).aspx](http://technet.microsoft.com/ru-ru/library/bb522607(v=sql.110).aspx).
- Oracle Essbase:
 - http://docs.oracle.com/cd/E17236_01/nav/portal_3.htm (на английском языке).

17.6.3.2. Общая схема использования

Для того чтобы использовать информацию из внешних источников данных в системе на базе «1С:Предприятия», следует придерживаться следующей общей схемы:

- Изучить структуру внешнего источника данных и понять, какая информация (кубы, таблицы измерений, измерения и ресурсы) необходима для работы алгоритмов или отчетов в «1С:Предприятии».
- Создать объект [Внешние источники данных](#) в конфигураторе и для него создать требуемую структуру. Это можно сделать с помощью специального помощника.
- Реализовать использование созданных объектов в прикладном решении.
- Выполнить настройку параметров подключения к внешнему источнику данных в той сети, где будет использоваться прикладное решение. Эти параметры могут отличаться от тех, которые использовались при загрузке структуры внешнего источника данных.

ПРИМЕЧАНИЕ. Параметры доступа к внешнему источнику данных, которые были заданы в конфигураторе, не будут использованы системой в режиме 1С:Предприятие.

17.6.3.3. Редактирование структуры внешнего источника данных

17.6.3.3.1. Общая информация

Для работы с таблицами внешних источников данных предназначена ветвь с именем соответствующего внешнего источника данных ветви [Внешние источники данных](#) - [<Имя внешнего источника данных>](#) - [Кубы](#) дерева конфигурации. Определение внешнего источника данных состоит из нескольких шагов:

1. Определение собственно внешнего источника данных.

2. Определение кубов;
3. Определение таблиц измерений и полей для каждой таблицы измерений;
4. Определение измерений для куба;
5. Определение ресурсов для куба.

При этом в конфигурации может быть описано меньшее количество объектов, чем есть в реальной базе данных, но нельзя создать объект, которого отсутствует в реальной базе данных.

Создание структуры внешнего источника данных возможно как в ручном режиме, так и при загрузке этой структуры с помощью специального конструктора.

17.6.3.3.2. Внешний источник данных

При создании внешнего источника данных следует указать его имя. Этот объект предназначен для идентификации группы кубов при обращении к данным, которые в нем содержатся. Источник данных состоит из одного или нескольких кубов, которые, в свою очередь, состоят из других объектов (таблицы измерений, измерения, ресурсы).

17.6.3.3.3. Куб внешнего источника данных

При создании нового куба открывается окно редактирования объекта (см. [здесь](#)).

Свойство таблицы **Имя** служит для идентификации куба внутри прикладного решения. Для того чтобы система знала, какой физический куб внешнего источника данных соответствует объект конфигурации, следует заполнить свойство **Имя в источнике данных**. Значение в этом свойстве в точности должно соответствовать имени куба внешнего источника данных. Не поддерживается размещение в одном кубе прикладного решения данных из нескольких кубов внешнего источника данных.

17.6.3.3.4. Таблица измерения

Таблица измерения описывает члены (members) измерения куба OLAP-системы. Свойство **Имя в источнике данных** содержит точное имя измерения или уровня иерархии в источнике данных. Свойство **Поле представления** содержит имя свойства измерения или уровня иерархии, которое система «1С:Предприятие» использует для формирования представления элемента таблицы измерения.

Если измерение является иерархическим, то для такого измерения может быть использована иерархическая таблица измерения. Иерархическая таблица измерения описывается с помощью следующих свойств:

- **Иерархическая** - содержит признак того, что таблица измерения описывает иерархию в источнике данных.
- **Имя иерархии в источнике данных** - указывает имя иерархии, к которой относится таблица измерения, описывающая данный уровень.
- **Номер уровня** - в том случае, если таблица описывает какой-либо уровень иерархии, данное свойство содержит номер этого уровня. Для собственно иерархической таблицы значение этого свойства равно 0, а свойство **Имя иерархии в источнике данных** совпадает со значением свойства **Имя в источнике данных**.

Свойства **Признак незаполненного родителя** и **Значение незаполненного родителя** позволяют указать системе «1С:Предприятие», каким образом идентифицировать значения самого верхнего уровня иерархии.

17.6.3.3.5. Поле таблицы измерений

Поля таблицы измерений описывают, какие свойства измерений будут доступны из прикладного решения. Свойство поля **Имя** служит для идентификации свойства (поля таблицы измерения) в прикладном решении. Для указания соответствия между полем прикладного решения и атрибутом измерения служит свойство **Имя в источнике данных**. Значение в этом свойстве должно в точности соответствовать имени атрибута измерения, указанного в свойстве **Имя в источнике данных** родительского объекта данного поля. Не поддерживается объединение в одном поле таблицы измерений данных из нескольких атрибутов одного или нескольких измерений внешнего источника данных.

Поле **Тип** позволяет указать тип данного поля. Для выбора доступен ограниченный набор типов. Типами поля могут быть:

- **Число**;
- **Строка**;
- **Дата**;
- **Булево**;
- **УникальныйИдентификатор**;
- **ДвоичныеДанные**;
- типы, которые определяются таблицами измерений внешних источников данных.

Если необходимо указать для поля составной тип, то в таком составном типе могут участвовать только типы **Число**, **Строка**, **Дата**, **Булево**.

При получении данных из внешних источников система автоматически преобразует эти данные к тому типу, который указан у

соответствующего поля в прикладном решении. Описание правил преобразования см. [здесь](#).

17.6.3.3.6. Измерения

Измерение внешнего источника данных описывает измерение куба OLAP-системы. Типом измерения может быть только ссылка на соответствующую таблицу измерения. Фактически, таблицы измерений и сами измерения, соотносятся один к одному: одно измерение описывается одной таблицей измерения.

Не поддерживается объединение в одном поле измерения данных из нескольких измерений внешнего источника данных.

17.6.3.3.7. Ресурсы

Ресурс внешнего источника данных описывает факт куба OLAP-системы. Ресурс куба может быть числовым или строковым. Именно значения ресурсов рассчитываются по значениям измерений куба. Свойство [Имя в источнике данных](#) должно совпадать с именем факта (ресурса) куба внешнего источника данных.

Не поддерживается объединение в одном ресурсе данных из нескольких фактов (ресурсов) внешнего источника данных.

17.6.3.3.8. Загрузка структуры куба из внешнего источника данных

Существует возможность загрузки структуры куба из внешнего источника данных, который доступен для разработчика (например, копии реальной базы данных).

Для выполнения этой операции следует при создании куба указать пункт [Выбрать из списка кубов внешнего источника данных](#) в конструкторе кубов внешнего источника данных. Затем нужно указать строку соединения с внешней базой данных, воспользовавшись окном [Подключение к источнику данных](#). При формировании строки подключения можно воспользоваться конструктором строки соединения с внешним источником данных. Для этого следует нажать кнопку «...» справа от поля [Строка соединения](#).

После того как подключение к внешней базе успешно выполнено, будет открыт список с перечнем кубов, таблиц измерений, полей таблиц измерений и ресурсов подключенного источника данных. Затем следует выбрать требуемый куб, а также таблицы измерений, поля и ресурсы, которые будут использованы конфигуратором для создания структуры объектов, описывающих текущий источник данных.

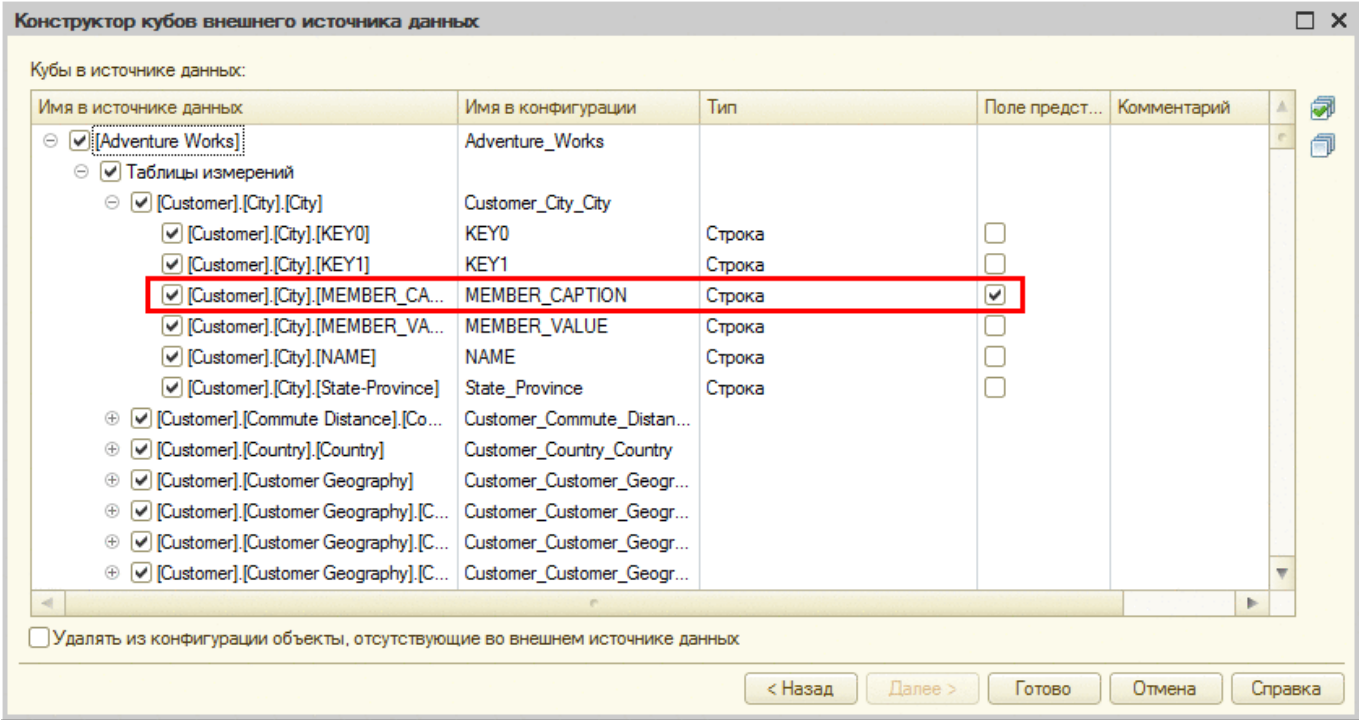


Рис. 515. Конструктор кубов внешнего источника данных

При получении структуры внешнего источника данных система выполняет следующие действия:

- Выполняет попытку определить поля, формирующие представления для таблиц измерений. Если система ошиблась, поле представления можно указать вручную.
- Преобразует типы колонок источника данных в типы «1С:Предприятия», которые будут использоваться для указания типов реквизитов объектов. Все типы из внешнего источника данных преобразуются к следующим типам «1С:Предприятия»: число, строка, дата, булево, уникальный идентификатор, двоичные данные и типы, связанные с таблицами измерений источника данных.
- Выполняет попытку определить типы для полей таблиц измерений. В этом случае система пытается определить, какого типа данные хранятся в колонке таблицы измерения, и в том случае, если это можно считать ссылкой на данные другой таблицы измерения, указывает в колонке соответствующий тип. Если система неправильно выбрала тип колонки таблицы, его можно изменить вручную.

Затем следует отметить флажками объекты, которые будут перенесены в метаданные прикладного решения.

Если флажок **Удалять из конфигурации объекты, отсутствующие во внешнем источнике данных** установлен, то при завершении работы конструктора кубов из конфигурации будут удалены те объекты (кубы, таблицы измерений, поля таблицы измерений, измерения, ресурсы), которые отсутствуют во внешнем источнике (например, во внешнем источнике куб или ресурс были удалены) данных.

После нажатия кнопки **Готово** произойдет загрузка структуры внешнего источника данных.

17.6.3.4. Ограничения языка запросов при использовании аналитического внешнего источника данных

При работе с кубами и таблицами измерений в языке запросов, следует учитывать следующие ограничения:

Выражение	Microsoft SQL Server Analysis Services	Oracle Essbase	IBM Infosphere Warehouse
РАЗРЕШЕННЫЕ	–	–	–
РАЗЛИЧНЫЕ	–	–	–
СТРУППИРОВАТЬ	–	–	–
ИМЕЮЩИЕ	–	–	–
СОЕДИНЕНИЕ	–	–	–
ОВЪЕДИНИТЬ	–	–	–
Подзапросы	–	–	–
ДЛЯ ИЗМЕНЕНИЯ	–	–	–
ПУСТАЯТАБЛИЦА	–	–	–
ГОД	Да	–	–
КВАРТАЛ	Да	–	–
МЕСЯЦ	Да	–	–
ДЕНЬГОДА	Да	–	–
ДЕНЬ	Да	–	–
НЕДЕЛЯ	Да	–	–
ДЕНЬНЕДЕЛИ	Да	–	–
ЧАС	Да	–	–
МИНУТА	Да	–	–
СЕКУНДА	Да	–	–
НАЧАЛОПЕРИОДА	Да	–	–
КОНЕЦПЕРИОДА	Да	–	–
ДОБАВИТЬКДАТЕ	Да	–	–
РАЗНОСТЬДАТ	Да	–	–

Где,

- - указанное выражение не поддерживается при работе с соответствующей OLAP-системой;
- Да - указанное выражение поддерживается при работе с соответствующей OLAP-системой.

17.6.3.5. Строка соединения с OLAP-сервером

Строка соединения с OLAP-сервером имеет вид:

http://<Адрес хоста OLAP>:<порт>/<источник>?<параметры>

Копировать в буфер обмена

Где:

- адрес хоста OLAP, порт, источник - адрес доступа к OLAP-системе, сформированный по правилам, описанным в документации конкретной OLAP-системе.
- параметры - параметры, используемые «1С:Предприятием» для доступа к данным OLAP-системы. Параметры задаются в виде Параметр=Значение. Параметры разделяются символом &. Используются следующие параметры:
 - ProviderName - имя сервиса XMLA OLAP-источника;
 - DataSourceName - имя OLAP-источника;
 - Catalog - имя каталога, или базы данных, OLAP-источника.

Примеры строк соединения приведены ниже:

Для Microsoft SQL Server Analysis Services:

```
http://localhost:80/msolap/msmdpump.dll?ProviderName=Microsoft Analysis Services&
DataSourceName=host&Catalog=Adventure Works DW
```

[Копировать в буфер обмена](#)

Для Oracle Essbase:

```
http://localhost:13080/aps/XMLA?ProviderName=Essbase XML for Analysis&DataSourceName=host&Catalog=Sample
```

[Копировать в буфер обмена](#)

17.6.4. Работа с процедурами и функциями внешних источников данных

17.6.4.1. Общая информация

Состав языка запросов (см. [здесь](#)) и системы компоновки данных (см. [здесь](#)) предоставляет широкие возможности по обработке данных, но не включает в себя специфические возможности различных СУБД, к которым можно отнести функции, последовательности, хранимые процедуры и различные специфические конструкции СУБД. В то же время, применение специфических возможностей СУБД может оказаться критически важным при работе с внешними источниками данных. В качестве примера можно привести объектную таблицу СУБД, ключ которой получается из последовательности. Функции и процедуры могут ничего не возвращать (процедура), а также возвращать одиночное значение или таблицу.

Для работы с такими объектами СУБД в «1С:Предприятии» существует понятие **функции внешнего источника данных**. Функции в «1С:Предприятии» могут быть следующих видов:

1. Таблица вида выражение в реляционном источнике данных;
2. Функция, которая не возвращает значения (процедура);
3. Функция, которая возвращает какое-либо значение.

Любая из вышеописанных функций может принимать параметры, количество параметров может быть как фиксированным, так и переменным. Функция внешнего источника данных может выступать как прослойкой между реальной функцией (или сохраненной процедурой) СУБД, так и явным образом описывать некоторую конструкцию, которая будет подставлена в выполняемый запрос.

Функции могут использоваться во встроенном языке, языке запросов и системе компоновки данных. Для функций можно указывать права доступа, регламентирующие использование этих функций.

17.6.4.2. Общая схема использования

Для того чтобы использовать функции внешних источников данных в системе на базе «1С:Предприятия», следует придерживаться следующей общей схемы:

- Изучить структуру внешнего источника данных и понять, какие функции, процедуры или конструкции на языке СУБД необходимы для работы алгоритмов или отчетов в «1С:Предприятии».
- Создать объект [Внешние источники данных](#) в конфигураторе и для него создать подчиненные функции. Это можно сделать с помощью специального помощника.
- Реализовать использование созданных объектов в прикладном решении.
- Выполнить настройку параметров подключения к внешнему источнику данных в той сети, где будет использоваться прикладное решение. Эти параметры могут отличаться от тех, которые использовались при загрузке структуры внешнего источника данных.

ПРИМЕЧАНИЕ. Параметры доступа к внешнему источнику данных, которые были заданы в конфигураторе, не будут использованы системой в режиме 1С:Предприятие.

17.6.4.3. Редактирование функций

17.6.4.3.1. Общая информация

Для работы с функциями внешних источников данных предназначена ветвь с именем соответствующего внешнего источника данных ветви [Внешние источники данных](#) - [<Имя внешнего источника данных>](#) - [Функции](#) дерева конфигурации. Определение внешнего источника данных состоит из следующих шагов:

1. Определение собственно внешнего источника данных;
2. Определение функций источника данных;
3. Если во внешнем источнике данных существуют процедуры, возвращающие таблицы, то необходимо также определить таблицы внешнего источника данных вида выражение.

При этом в конфигурации может быть описано меньшее количество функций и процедур, чем есть в реальной базе данных. Также можно написать некоторую функцию (на языке используемой СУБД), которая отсутствует в реальной базе данных. При этом надо учитывать следующие особенности:

- Если для функции (как объекта конфигурации) указано, что выражение возвращает значение, то:

- При использовании во встроенном языке - выражение подставляется в конструкцию `select <выражение>`.
- При использовании в языке запросов (и системе компоновки данных) - происходит замена вызова функции на собственно выражение.
- Если для функции (как объекта конфигурации) указано, что она не возвращает значение, то:
 - При использовании во встроенном языке - в зависимости от СУБД, может использоваться либо вызов с использованием конструкции `select <выражение>`, либо будет использована управляющая последовательность ODBC `call`. В силу этого, не рекомендуется использование произвольного выражения для функции, не возвращающей значение, с целью максимальной совместимости с различными СУБД.
 - При использовании в языке запросов (или системе компоновки данных) - не поддерживается.

Создание структуры внешнего источника данных возможно как в ручном режиме, так и при загрузке этой структуры с помощью специального конструктора.

17.6.4.3.2. Внешний источник данных

При создании внешнего источника данных следует указать его имя. Этот объект предназначен для идентификации функций при обращении. Источник данных состоит из одной или нескольких функций, также возможна ситуация, когда во внешнем источнике данных располагаются таблицы-выражения.

Если объект внешнего источника данных служит исключительно для обращений к процедурам и функциям, то значение свойства **Режим управления блокировкой данных** не имеет смысла. Если для внешнего источника данных режим управления блокировок установлен в значение **Автоматический и управляемый**, то для каждой таблицы режим блокировок определяется значением соответствующего свойства таблицы. Если для внешнего источника данных выбран определенный конкретный режим управления блокировками, то одноименное свойство таблицы внешнего источника данных игнорируется.

17.6.4.3.3. Функция внешнего источника данных

При создании новой функции открывается окно конструктора функции внешнего источника данных. Если выбран ручной режим создания - будет открыта палитра свойств созданной функции внешнего источника данных (см. [здесь](#)).

Свойство функции **Имя** служит для идентификации функции при обращении к ней. Свойство **Выражение в источнике данных** будет содержать в себе выражение, которое будет использоваться в реальном запросе в базе данных при использовании функции. В данном свойстве можно указать не только обращение к реальной функции или сохраненной процедуры базы данных, но и написать выражение на языке СУБД внешнего источника данных.

Свойство **Возвращает значение** фактически указывает, что описывается в объекте конфигурации:

- **Истина** - описывается функция или выражение, возвращающее значение. В этом случае также необходимо указать, какого типа значение будет возвращаться данным выражением.
- **Ложь** - процедура или выражение, не возвращающее значение.

Процедура или функция может принимать параметры. Подробнее про параметры см. [здесь](#).

Функцией может выступать выражение на языке СУБД, например функция, которое получает следующее значение последовательности (`SimpleSequence`), будет иметь следующий вид:

```
NEXT VALUE FOR dbo.SimpleSequence
```

[Копировать в буфер обмена](#)

Если нужно получить следующее значение последовательности из встроенного языка, то это будет выглядеть следующим образом:

```
ЗначениеКлюча = ВнешниеИсточникиДанных.ВнешниеДанные.СледующееЗначениеКлюча ();
```

[Копировать в буфер обмена](#)

В данном примере **ВнешниеДанные** - имя внешнего источника данных, в котором описана функция **СледующееЗначениеКлюча ()**. Функция **СледующееЗначениеКлюча ()** описана выше.

Также функция может быть написана на языке используемой СУБД. При этом рекомендуется выражение заключать в круглые скобки, чтобы избежать различных побочных эффектов во время исполнения реального запроса. Следует помнить, что при трансформации запроса из языка запросов платформы «1С:Предприятие», в диалект SQL конкретной СУБД, функции подставляются непосредственно в текст получающегося запроса (с автоматической подстановкой параметров функций).

17.6.4.3.4. Таблица вида выражение внешнего источника данных

Функции могут возвращать не только значения, но и таблицы. В этом случае функцию необходимо создавать как **таблицу вида выражение**. Действия, которые необходимо выполнить при создании таблицы внешнего источника данных, см. [здесь](#). Также необходимо описать поля возвращаемых данных, аналогично обычным полям таблицы внешнего источника данных (см. [здесь](#)).

17.6.4.3.5. Загрузка функций из внешнего источника данных

Существует возможность загрузки структуры таблиц из внешнего источника данных, который доступен для разработчика (например, копии реальной базы данных).

Для выполнения этой операции следует при создании таблицы указать пункт [Выбрать из списка функций внешнего источника данных](#) в конструкторе функции внешнего источника данных. Затем нужно указать строку соединения с внешней базой данных, воспользовавшись окном [Подключение к источнику данных](#). При формировании строки подключения можно воспользоваться конструктором строки соединения с внешним источником данных. Для этого следует нажать кнопку «...» справа от поля [Строка соединения](#).

После того как подключение к внешней базе успешно выполнено, будет открыт список с перечнем процедур, функций и последовательностей подключенного источника данных. Затем следует выбрать те объекты, которые будут использованы конфигуратором для создания структуры объектов, описывающих текущий источник данных.

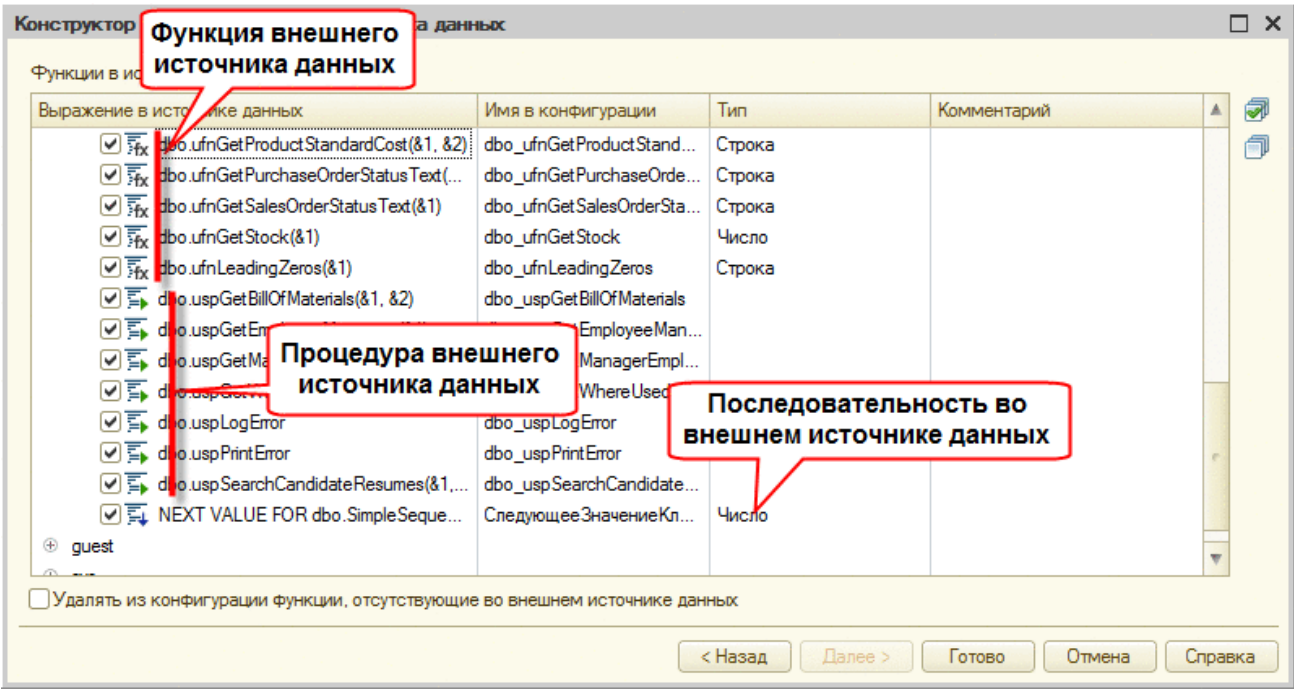


Рис. 516. Конструктор функций внешнего источника данных

Имеется возможность указать, под каким именем функция будет создана в конфигурации «1С:Предприятия».

Если флажок [Удалять из конфигурации функции, отсутствующие во внешнем источнике данных](#) установлен, то при завершении работы конструктора функций из конфигурации будут удалены те функции, которые отсутствуют во внешнем источнике (например, во внешнем источнике функции были удалены) данных.

После нажатия кнопки [Готово](#) произойдет загрузка структуры функций внешнего источника данных.

17.6.4.3.6. Параметры функций внешнего источника данных

Формальные параметры функций внешнего источника данных описываются как `&n`, где `n` - номер параметра. Нумерация параметров начинается с 1. Если параметры являются необязательными, то параметры заключаются в фигурные скобки `{ }`. Наличие необязательных параметров определяется каждый раз при фактическом вызове процедуры или функции. На место отсутствующих в необязательном блоке фактических параметров при вызове подставляется `NULL` (для каждого необязательного параметра).

```
// функция с двумя параметрами
dbo.uspGetWhereUsedProductID(&1, &2)
// функция в два необязательными параметрами
dbo.uspGetWhereUsedProductID(&1{, &2, &3})
```

[Копировать в буфер обмена](#)

Если процедура или функция может принимать переменное число параметров, то такая конструкция описывается как `&n[ ]`. В этом случае фактические параметры будут подставлены в выражение на место конструкции `&n[ ]` через запятую. Указание на возможность передачи переменного числа параметров может быть только последним в списке формальных параметров.

```
// функция, куда может быть передано от 2 до количества параметров,
// ограниченных используемой СУБД
dbo.uspGetWhereUsedProductID(&1, &2[ ])
```

[Копировать в буфер обмена](#)

17.6.5. Управление внешними источниками данных

Перед тем как начать использование данных из внешних источников в прикладном решении, следует настроить параметры подключения используемых внешних источников. Для этого служит стандартная функция [Управление внешними источниками данных](#), которая вызывается из окна [Функции для технического специалиста - Стандартные функции](#).

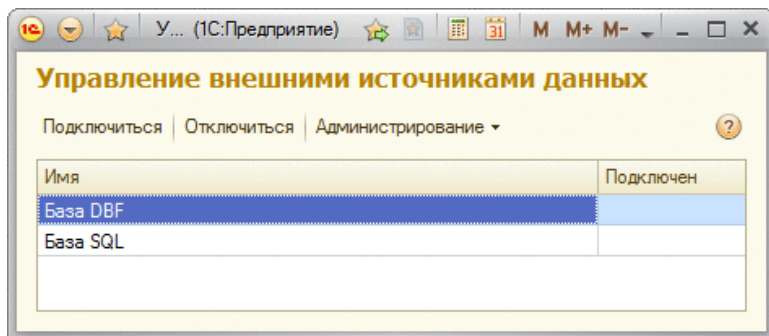


Рис. 517. Управление внешними источниками данных

В данной форме перечислены все внешние источники данных, созданные в конфигураторе и доступные пользователю, который открыл форму.

Колонка **Подключен** указывает состояние подключения к источнику данных в этом сеансе.

С помощью меню **Администрирование** можно указать общие параметры подключения (команда **Изменить общие параметры...**) и параметры подключения конкретных пользователей в том случае, если эти параметры отличаются от общих параметров (команда **Изменить параметры пользователей...**).

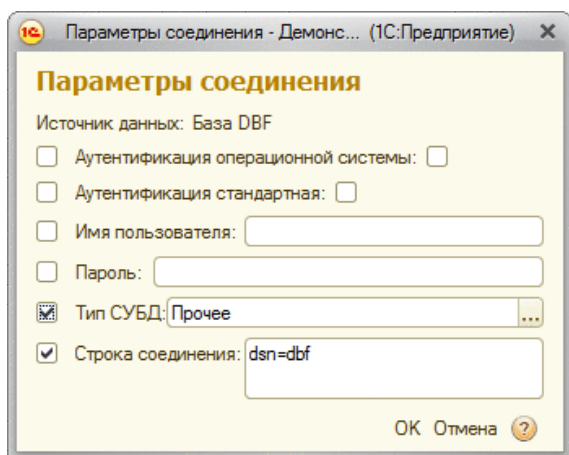


Рис. 518. Параметры соединения

Флажок в левой части формы (перед именами параметров) означает, что данный параметр используется в данном наборе параметров соединения. При этом параметры анализируются в следующем порядке:

- параметры, установленные для сеанса с помощью метода `УстановитьПараметрыСоединенияСеанса()` менеджера внешнего источника данных;
- параметры, указанные для пользователя при интерактивной настройке или с помощью метода `УстановитьПараметрыСоединенияПользователя()` менеджера внешнего источника данных;
- общие параметры, заданные интерактивно или с помощью вызова метода `УстановитьОбщиеПараметрыСоединения()` менеджера внешнего источника данных.

Результирующий набор параметров будет использован для соединения с внешним источником данных. Если какой-то параметр указан в нескольких настройках, будет использован то значение параметра, которое встретится в вышеописанной цепочке раньше.

Например, можно следующим образом комбинировать параметры подключения:

- в параметрах соединения сеанса указывается пользователь и его пароль;
- в общих параметрах указывается строка соединения с базой данных внешнего источника;
- в параметр конкретного пользователя указывается другая строка соединения, которая может использоваться для тестирования прикладного решения.

При указании строки соединения следует помнить, что доступ к внешним данным будет выполняться не с компьютера, где происходит настройка, а с другого компьютера (описание того, с какого компьютера выполняется запрос к внешнему источнику данных, см. [здесь](#)). Подробнее о строке соединения см. [здесь](#).

Нажатие кнопки **Подключиться** открывает диалог, в котором можно задать (или уточнить) параметры подключения к выбранному внешнему источнику данных.

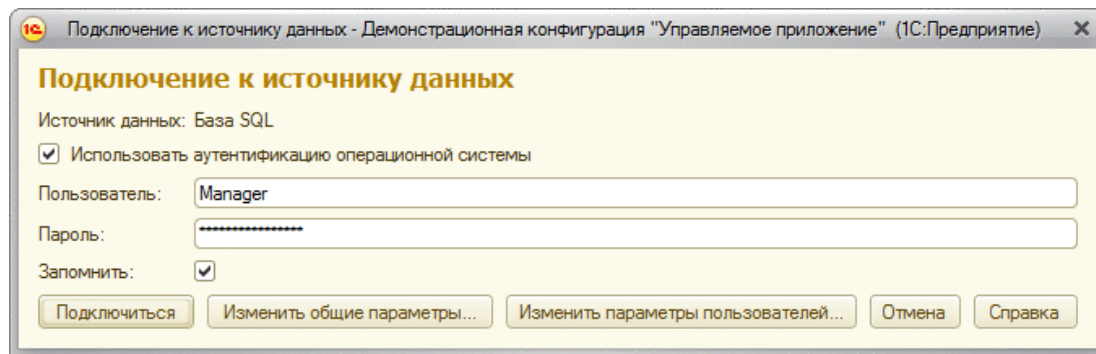


Рис. 519. Параметры подключения

Необходимость указания пользователя и пароля в данном диалоге зависит от драйвера ODBC и содержания строки соединения. Возможны ситуации, когда ввод логина и пароля не требуется. Если пользователь не является администратором внешнего источника данных, то кнопки [Общие параметры...](#) и [Параметры пользователей...](#) для него недоступны.

Флажок [Использовать аутентификацию операционной системы](#) доступен только в том случае, если у пользователя имеется право [ИзменениеАутентификацииОСсеанса](#).

После нажатия кнопки [Подключиться](#) происходит подключение к внешнему источнику данных. В случае успешного подключения изменяется содержимое колонки [Подключен](#) списка внешних источников данных.

Нажатие кнопки [Отключиться](#) приведет к отключению «1С:Предприятия» от выбранного внешнего источника данных.

17.6.6. Подключение к внешнему источнику данных в режиме «1С:Предприятие»

При выполнении любой операции, в ходе которой необходимо получать данные из внешнего источника данных, происходит попытка подключения, если подключение к этому источнику данных еще не было выполнено. Если подключение выполнено успешно, то выполняемые действия продолжают. Если подключение не выполнено, вызывается исключение.

В случае если в клиентском приложении возникает исключение, связанное с подключением к внешнему источнику данных, пользователю предлагается диалог, в котором можно уточнить параметры подключения (если пользователь имеет права на изменение параметров) и повторить подключение. Если подключение выполнено успешно, то пользователю будет предложено повторить действие, во время которого произошла ошибка подключения.

ПРИМЕЧАНИЕ. В режиме совместимости с версией 8.2.13, при работе с динамическим списком, в некоторых случаях диалог подключения к внешним данным может не показываться.

Также имеется возможность выполнить подключение вручную. Для этого предназначена стандартная функция [Управление внешними источниками данных](#).

17.6.7. Особенности использования внешних источников данных

17.6.7.1. Место исполнения запросов к внешним источникам данных

Исполнение запросов к внешним источникам данных выполняется:

- в файловом варианте - на компьютере с клиентским приложением;
- в файловом варианте с доступом через веб-сервер - на компьютере, где выполняется расширение веб-сервера;
- в клиент-серверном варианте - на компьютере, где выполняется:
 - сервис работы с внешними источниками данных через ODBC - для доступа к реляционным источникам данных;
 - сервис работы с внешними источниками данных через XMLA - для доступа к аналитическим источникам данных (OLAP-системы).

17.6.7.2. Использование внешних источников данных

В том случае, если один внешний источник данных содержит в своем составе и данные реляционных источников данных и данные аналитических источников данных, их одновременное использование не поддерживается. Это связано с тем, что один внешний источник данных может быть в один момент времени подключен или к реляционной или к аналитической СУБД.

17.6.7.2.1. В запросах

К реляционным внешним источникам

Таблицы внешних источников данных могут выступать в качестве источников запросов. При этом таблица внешнего источника данных описывается следующим образом:

ВнешнийИсточникДанных.<Имя источника>.Таблица.<Имя таблицы>

[Копировать в буфер обмена](#)

Пример:

ВЫБРАТЬ

Товары.Код,
Товары.Наименование,
Товары.Артикул

ИЗ

ВнешнийИсточникДанных.DBF.Таблица.Товары **КАК** Товары

В данном примере происходит выборка полей **Код**, **Наименование** и **Артикул** из таблицы **Товары** внешнего источника данных **DBF**.

В запросе к внешнему источнику данных возможно использование временных таблиц. При этом производится попытка создания временной таблицы непосредственно в базе данных, связанной с внешним источником данных. Если СУБД не поддерживает создание временных таблиц - будет вызвано исключение. Имя временной таблицы, при обращении к ней, формируется следующим образом:

[Копировать в буфер обмена](#)

ВнешнийИсточникДанных.<Имя внешнего источника данных>.ВременнаяТаблица.<Имя временной таблицы>

Пример:[Копировать в буфер обмена](#)

ВЫБРАТЬ Name, ProductID
ПОМЕСТИТЬ ВнешнийИсточникДанных.AdventureWorks.ВременнаяТаблица.Остатки
ИЗ &ТаблицаЗначений

ВЫБРАТЬ Name, ProductID
ПОМЕСТИТЬ ВнешнийИсточникДанных.AdventureWorks.ВременнаяТаблица.Остатки
ИЗ ВнешнийИсточникДанных.AdventureWorks.Таблица.Production_Balance

При использовании в запросах внешних источников данных нужно помнить следующие ограничения:

- В одном запросе можно использовать один источник данных. Не поддерживается одновременное использование, например, внешнего источника данных и данных информационной базы «1С:Предприятия».

К аналитическим внешним источникам

Объекты аналитических внешних источников данных могут выступать в качестве источников запросов. При этом куб внешнего источника данных описывается следующим образом:

[Копировать в буфер обмена](#)

ВнешнийИсточникДанных.<Имя источника>.Куб.<Имя куба>

Пример:[Копировать в буфер обмена](#)

ВЫБРАТЬ ПЕРВЫЕ 10
AdventureWorks.InternetSalesAmount
ИЗ
ВнешнийИсточникДанных.OLAP.Куб.AdventureWorks **КАК** AdventureWorks

В данном примере выполняется получение 10 первых ресурсов **InternetSalesAmount** из куба **AdventureWorks** внешнего источника данных **OLAP**.

ВНИМАНИЕ! Не рекомендуется получать **все** записи из куба внешнего источника данных. Количество записей очень велико и может привести к перегрузке системы.

Таблица измерений куба внешнего источника данных описывается следующим образом:

[Копировать в буфер обмена](#)

ВнешнийИсточникДанных.<Имя источника>.Куб.<Имя куба>.ТаблицаИзмерений.<Имя таблицы измерений>

Пример:[Копировать в буфер обмена](#)

ВЫБРАТЬ
AdventureWorksТаблицаИзмеренияProduct.Ссылка
ИЗ
ВнешнийИсточникДанных.OLAP.Куб.AdventureWorks.ТаблицаИзмерения.Product **КАК** AdventureWorksТаблицаИзмеренияProduct

В данном примере происходит получения списка ссылок на членов таблицы измерения **Product** из куба **AdventureWorks** внешнего источника данных **OLAP**.

В запросе к внешнему источнику данных возможно использование временных таблиц. При этом производится попытка создания временной таблицы непосредственно в базе данных, связанной с внешним источником данных. Если СУБД не поддерживает создание временных таблиц - будет вызвано исключение. Имя временной таблицы, при обращении к ней, формируется следующим образом:

[Копировать в буфер обмена](#)

ВнешнийИсточникДанных.<Имя внешнего источника данных>.ВременнаяТаблица.<Имя временной таблицы>

При использовании в запросах внешних источников данных нужно помнить следующие ограничения:

- В одном запросе можно использовать один источник данных. Не поддерживается одновременное использование, например, внешнего источника данных и данных информационной базы «1С:Предприятия».

17.6.7.2.2. В системе компоновки данных

В системе компоновки данных возможно использование наборов данных, получающих данные из разных однотипных источников данных. Так, можно создать набор данных, описывающий список товаров. Данные в этом наборе будут получены из информационной базы «1С:Предприятия». Другой набор данных может описывать данные, полученные из внешнего источника данных, например таблица продаж товаров из внешней базы данных.

Затем можно настроить связи наборов данных в конструкторе схемы компоновки данных и получать в отчет информацию одновременно из «1С:Предприятия» и внешнего источника данных.

17.6.7.3. Правила преобразования значений

При получении данных из внешних источников система автоматически преобразует эти данные к тому типу, который указан у соответствующего поля в прикладном решении. Преобразование выполняется по следующим правилам:

- Тип **Строка**:
 - Строковое значение преобразуется в строковое значение.
 - Числовое значение преобразуется в строковое значение (для преобразования используются региональные настройки операционной системы на том компьютере, где происходит выполнение запроса к внешнему источнику данных).
 - Значение даты преобразуется в строковое значение (для преобразования используются региональные настройки операционной системы на том компьютере, где происходит выполнение запроса к внешнему источнику данных).
 - Значение уникального идентификатора преобразуется в строковое представление уникального идентификатора.
 - Двоичные данные (и данные других типов) преобразуются в строку, которая является текстовым представлением преобразуемых данных в шестнадцатеричном виде.
- Тип **Число**:
 - Числовое значение преобразуется в число.
 - Для строкового значения выполняется попытка преобразования строки в число в соответствии с региональными настройками операционной системы компьютера, на котором выполняется запрос к внешнему источнику данных.
 - Для полей, у которых точность в базе источника превышает точность типов платформы, рекомендуется либо явно использовать **ВЫРАЗИТЬ** в запросах, либо создавать представление (view) в базе, так, чтобы точность полей соответствовала точности в конфигураторе.
 - Для остальных типов возникает исключение.
- Тип **Булево**:
 - Числовое значение преобразуется в значение **Ложь**, если его значение равно 0 и в значение **Истина** - для любого другого значения во внешнем источнике.
 - Двоичные данные преобразуются по значению первого байта данных. Если первый байт равен значению 0, то результатом преобразования является значение **Ложь** и значение **Истина** в случае любого другого значения.
 - Для остальных типов возникает исключение.
- Тип **ДвоичныеДанные**:
 - В двоичные данные преобразуются значения любых типов.
- Тип **Дата**:
 - Значение даты преобразуется в значение типа **Дата**.
 - Для остальных типов возникает исключение.

17.6.7.4. Внешний источник данных, входящий в состав разделителя

Если внешний источник данных входит в состав общего реквизита, то следует учитывать следующие особенности:

- Изменение структуры таблиц внешнего источника данных не производится, поэтому собственно данные не разделяются.
- Параметры соединения с внешним источником данных хранятся в разрезе значений разделителей, в состав которых входит внешний источник данных.
- Неиспользуемый разделитель является отдельным значением, в разрезе которого хранятся параметры соединения с внешним источником данных.

Таким образом, включая внешний источник данных в состав разделителя, появляется возможность реализовать один из следующих сценариев использования:

- Данные, доступные для пользователей различных областей данных, расположены в разных базах данных, которые

совпадают по структуре таблиц. Т. к. параметры соединения хранятся в разрезе значений разделителя, имеется возможность для каждой области данных настроить работу с необходимой базой данных.

- Данные, доступные для пользователей различных областей расположены в одной базе данных, но доступ к данным регулируется средствами СУБД (имя и пароль пользователя). В этом случае имеется возможность для каждой области данных указать свои имя и пароль пользователя, от имени которого выполняется доступ к данным.

Приведенные сценарии не являются единственными. Они приведены как пример использования внешних источников данных, входящих в состав разделителей.

17.7. Повторное использование сеансов интернет-сервисов

17.7.1. Общая информация

Устройство системы «1С:Предприятие» таково, что вызов интернет-сервиса (Web-сервис, HTTP-сервис, стандартный интерфейс OData) и запуск клиентского приложения выглядит примерно одинаково - система создает сеанс работы с информационной базой (подробнее см. [здесь](#)), в котором и выполняются все необходимые действия. При завершении вызова интернет-сервиса (и при завершении работы клиентского приложения) сеанс завершается, а при повторном вызове процесс повторяется заново. Очевидным недостатком такого поведения является существенное время, которое тратит система на создание сеанса (включая выполнение всех обработчиков событий и загрузку собственно конфигурации). Особенно это заметно на «больших» прикладных решениях, которые содержат много различных объектов конфигурации. Для клиентского приложения время создания сеанса может быть несущественным (на фоне общего времени непрерывной работы клиентского приложения), а для интернет-сервисов это время может привносить существенные задержки в общее время вызова сервиса. Система предоставляет возможность управлять необходимостью создания сеанса при каждом вызове интернет-сервиса:

- Не использовать повторное использование сеансов. При каждом обращении к интернет-сервису будет создаваться новый сеанс.
- Включить повторное использование сеансов. В этом случае создание и завершение сеансов регулируется вызывающей стороной с помощью заголовков HTTP-запроса.
- Включить автоматическое повторное использование сеансов. В этом случае системой организуется пул сеансов. Каждый сеанс в пуле характеризуется некоторым набором параметров. Если сеанс в пуле свободен и характеризуется тем же набором параметров, что и запрос интернет-сервиса, то запросу сразу будет предоставлен существующий сеанс.

Возможность повторного использования сеансов настраивается в нескольких местах:

- в свойствах объектов конфигурации Web-сервис и HTTP-сервис (вид повторного использования и время жизни сеанса);
- в файле публикации [default.vrd](#) (параметры пула сеансов).

Следует помнить, что значения, указанные в файле [default.vrd](#), имеют приоритет над значениями, которые заданы в свойствах объектов конфигурации.

17.7.2. Виды повторного использования сеансов

Не использовать

В этом случае каждый запрос интернет-сервиса приведет к созданию нового сеанса.

Использовать

В этом случае временем жизни и характером повторного использования сеанса управляет непосредственно клиент интернет-сервиса. Для управления созданием нового сеанса используются специальные заголовки HTTP-запроса к интернет-сервису.

Использовать автоматически

Пул сеансов ведется в разрезе следующих параметров:

- тип сервиса;
- имя сервиса;
- пользователь;
- пароль;
- набор разделителей;
- безопасный режим.

При обнаружении входящего запроса к интернет-сервису, система анализирует текущий пул сеансов и ищет сеанс с параметрами, в точности соответствующими входящему запросу. Если такой сеанс обнаруживается и он не используется запросом от другого клиента интернет-сервиса, входящий запрос будет обслуживаться найденным сеансом. Если свободного сеанса не найдено - выполняется попытка создать новый сеанс. Если при попытке создания нового сеанса будет превышен размер пула сеансов - входящий запрос будет ожидать некоторое время (таймаут пула). Если по истечению таймаута пула свободный сеанс не обнаруживается - интернет-запрос будет завершен с ошибкой **406 Not Acceptable**. Свободный сеанс будет уничтожен системой автоматически после истечения времени жизни свободного сеанса.

Настройки автоматического пула сеансов действуют в рамках публикации. Таким образом, если для некоторой информационной базы созданы несколько публикаций, то при вызове интернет-сервиса используется настройка пула той публикации, через которую выполняется вызов.

Рекомендуется подбирать такой размер пула сеансов, чтобы при максимальной загрузке в пуле оставалось несколько свободных сеансов (т. е. выбирать размер пула с небольшим запасом). В противном случае при пиковой нагрузке некоторые вызовы будут завершаться с ошибками. Система «1С:Предприятие» не поддерживает автоматическое определение необходимого размера пула сеансов.

17.7.3. Настройки повторного использования сеансов

17.7.3.1. Свойства интернет-сервисов

В палитре свойств интернет-сервисов (Web-сервис, HTTP-сервис), включая собственные интернет-сервисы расширений конфигурации, присутствуют два свойства, описывающие повторное использование сеансов:

- Свойство **Повторное использование сеансов** - указывает вид повторного использования сеансов для настраиваемого сервиса (подробнее см. [здесь](#)).
- Свойство **Время жизни сеанса**:
 - описывает, через какой промежуток времени «свободный» сеанс будет удален из пула сеансов (в случае автоматического повторного использования сеансов);
 - описывает, через какое время бездействия сеанс будет принудительно завершён системой при ручном повторном использовании сеансов.

Если данное свойство установлено в значение 0, то это означает, что сеансы повторно не используются. Такая установка эквивалентна установке свойства **Повторное использование сеансов** в значение **Не использовать**.

Параметры пула сеансов для автоматического повторного использования сеансов настраиваются с помощью файла [default.vrd](#) (подробнее см. [здесь](#)).

17.7.3.2. На стороне клиента

Если свойство **Повторное использование сеансов** интернет-сервиса установлено в значение **Использовать**, то в этом случае управление временем жизни сеанса выполняет клиент интернет-сервиса. В качестве управляющего элемента выступает заголовок HTTP-запроса **IBSession**. Этот заголовок может принимать два значения:

- **start** - в этом случае система «1С:Предприятие» создаёт новый сеанс, выполняет аутентификацию, устанавливает разделители, выполняются все необходимые обработки событий. Если система не может создать новый сеанс, то клиент получит ошибку **406 Not Acceptable**. Если создание сеанса выполнено успешно, в HTTP-ответ помещается директива установить cookie **ibsession** с идентификатором созданного сеанса: **Set-Cookie: ibsession=<ID сеанса>**.

При необходимости использовать ранее созданный сеанс, необходимо в HTTP-запросе к системе «1С:Предприятие» указать идентификатор ранее созданного сеанса: **Cookie: ibsession=<ID сеанса>**. Если в запросе указывается идентификатор сеанса, который ранее не создавался или был завершён, клиент получает ошибку **400 Bad Request**.

Если в HTTP-запрос не содержится заголовка **IBSession**, то сеанс создаётся и завершается при каждом вызове интернет-сервиса.

Если в процессе использования сеанса в HTTP-запросе изменяются значения разделителей или безопасный режим сеанса, то новые параметры игнорируются и сеанс будет использовать те значения, которые были указаны при старте сеанса.

- **finish** - в этом случае система «1С:Предприятие» завершает сеанс, который указан в запросе, одновременно с командой завершения сеанса: **Cookie: ibsession=<ID сеанса>**. Завершение сеанса произойдёт автоматически, если в этом сеансе не выполнялось никаких действий за время жизни сеанса.

Пул сеансов в этом случае не используется.

17.7.3.3. В файле default.vrd

Описание параметров файла [default.vrd](#), которые управляют повторным использованием сеансов, см. [здесь](#).

17.7.3.4. Для стандартного интерфейса OData

Режим и параметры повторного использования сеансов для стандартного интерфейса OData устанавливаются системой «1С:Предприятие» при публикации стандартного интерфейса OData на веб-сервере. При этом по умолчанию используются следующие значения:

- Повторное использование сеансов - включено автоматическое использование.
- Время жизни сеанса - 20 секунд.

17.8. Сервисы интеграции

17.8.1. Общая информация

Для взаимодействия с системами класса сервисная шина предприятия, в платформе «1С:Предприятие» предназначен механизм сервисов интеграции. При рассмотрении механизма будут использоваться следующие термины:

- **Внешний сервис интеграции** - программный продукт «1С:Шина», к которому выполняется подключение системы «1С:Предприятие». «1С:Шина» взаимодействует с различными системами обмена сообщениями. Адаптером для внешнего сервиса интеграции в объектной модели «1С:Предприятие» является объект **Сервис интеграции**.

• **Канал** - логический адрес в системе обмена сообщениями. Канал предназначен для передачи сообщений определенного типа. Канал может быть:

- Входящим - используется для получения сообщений от внешнего сервиса интеграции.
- Исходящим - используется для отправки сообщений внешнему сервису интеграции.
- Канал не может быть двунаправленным (одновременно и входящим и исходящим).

• **Сообщение** - это минимальный и неделимый элемент обмена. Сообщение включает в себя собственно данные (в любом поддерживаемом формате) и заголовок сообщения, который используется внешним сервисом интеграции. Сообщение имеет определенное время жизни, которое используется сервисом интеграции во время доставки сообщения.

При работе с сообщениями следует помнить о следующих особенностях:

- Сообщения помещаются в канал отправителем последовательно.
- Сообщения получаются из канала получателем последовательно. Последовательность получения сообщений из канала совпадает с последовательностью помещения сообщений в канал.
- Сообщения разных каналов обрабатываются и доставляются параллельно.
- Любые два сообщения, полученные из разных каналов в определенной последовательности, не обязательно будут обработаны в этой же последовательности, так как обработка сообщений из разных каналов может идти с разной скоростью.

Механизм сервисов интеграции «1С:Предприятия» не является альтернативной механизмам планов обмена, так как отвечает только за транспортировку сообщений, а не за формирование исходящих и интерпретацию входящих сообщений. В то же время, механизм планов обмена может выступать в качестве источника данных для сообщений, отправляемых с помощью сервиса интеграции.

Взаимодействие с внешним сервисом интеграции выполняется с гарантированной доставкой сообщения, что означает:

- Отправляемое сообщение сохраняется в информационной базе до тех пор, пока от внешнего сервиса интеграции не будет получено подтверждение того, что сообщение им получено.
- Система «1С:Предприятие» будет выполнять попытки доставить сообщения внешнему сервису интеграции до тех пор, пока не будет получено подтверждение получения сообщения или сообщение не устареет.
- При получении сообщения от внешнего сервиса интеграции это сообщение сохраняется в информационной базе, и только после этого внешний сервис интеграции получает подтверждение о получении сообщения.

17.8.2. Редактирование сервиса интеграции

Сервисы интеграции расположены в дереве метаданных конфигурации в ветки **Общие**. Создание и редактирование объекта выполняется стандартным образом. Свойство объекта **Адрес внешнего сервиса интеграции** позволяет указать адрес используемого внешнего сервиса интеграции. После указания адреса внешнего сервиса интеграции необходимо указать каналы сервиса интеграции, которые будут использоваться из системы «1С:Предприятие». Для указания каналов доступны два способа:

1. Указать каналы вручную. Для этого предназначена закладка **Каналы** окна редактирования сервиса интеграции. При создании канала необходимо указать:

- **Имя** - это идентификатор канала в рамках системы «1С:Предприятие».
- **Имя канала внешнего сервиса интеграции** - это идентификатор канала в рамках внешнего сервиса интеграции. С помощью этого идентификатора будет выполняться однозначное сопоставление объекта в системе «1С:Предприятие» и объекта во внешнем сервисе интеграции.
- **Направление сообщения** - указывает, что канал используется для приема или отправки сообщений. Если канал используется для приема сообщений, то становится возможным указать обработчик для обработки получаемых сообщений (свойство **Обработчик получения сообщения**). Обработчик будет размещаться в модуле сервиса интеграции. Как и все модули объектов, данный модуль исполняется на сервере системы «1С:Предприятие».
- **В транзакции** - указывает, что обработчик полученного сообщения будет выполняться в рамках транзакции. В зависимости от состояния этого флажка, обработка сообщений выполняется по-разному:

- Флажок установлен. Перед вызовом обработчика получения сообщения начинается транзакция. Затем выполняется вызов обработчика.

Если обработчик отказался от обработки сообщения (параметр **Отказ** установлен в значение **Истина** или в обработчике вызвано исключение), то выполняется отмена транзакции и следующее сообщение не обрабатывается до тех пор, пока не будет успешно обработано текущее сообщение.

Если обработчик успешно завершился, то сообщение помечается как обработанное, а транзакция фиксируется.

- Флажок сброшен. Перед вызовом обработчика получения сообщения транзакция не начинается.

Если обработчик отказался от обработки сообщения (параметр **Отказ** установлен в значение **Истина** или в обработчике вызвано исключение), то следующее сообщение не обрабатывается до тех пор, пока не будет успешно обработано текущее сообщение.

Если обработчик успешно завершился, то сообщение помечается как обработанное.

Следует помнить одну важную особенность: в том случае, если во время установки для сообщения отметки обработанности произойдет сбой, то после восстановления работоспособности система повторно вызовет обработчик получения сообщения несмотря на то, что в предыдущий раз обработчик был выполнен успешно. Другими словами, если для канала сервиса интеграции отключается работа в транзакции, то обработчик получения сообщений должен быть написан исходя из предположения, что одно и тоже сообщение может получаться более одного раза.

2. Выбрать каналы, которые будут использоваться в системе «1С:Предприятие» из списка доступных каналов указанного внешнего сервиса интеграции. Для этого предназначен диалог [Загрузка каналов внешнего сервиса интеграции](#), доступ к которому предоставляется после выполнения команды [Загрузить каналы](#) контекстного меню конкретного сервиса интеграции. В данном диалоге выводятся все каналы выбранного внешнего сервиса интеграции.

17.8.3. Схема использования сервиса интеграции

Для использования внешнего сервиса интеграции необходимо:

1. Предварительные требования:

- Узнать адрес внешнего сервиса интеграции (одного или нескольких), который планируется использовать. Указать адреса внешних сервисов интеграции можно при внедрении прикладного решения.
- Узнать имена каналов (в каждом сервисе интеграции), которые будут использоваться в системе «1С:Предприятие».

2. Порядок действий при разработке:

- Создать в конфигурации нужное количество объектов [Сервис интеграции](#). Один объект соответствует одному внешнему сервису интеграции.
- Для каждого созданного сервиса интеграции создать список каналов, который будет использоваться конфигурацией. Корректно указать имена каналов и направления их (каналов) использования.
- Для исходящих каналов реализовать программный код, который позволит отправлять сообщения в нужный канал требуемого внешнего сервиса интеграции. В результате прикладное решение сможет отправлять данные в СШП.
- Для каждого входящего канала реализовать обработчик, который будет срабатывать при получении события в конкретном канале конкретного сервиса интеграции. В результате прикладное решение сможет получать данные из СШП. При разработке обработчика получения сообщения следует учитывать особенности поведения, которые зависят от флажка [В транзакции](#) (см. [здесь](#)).
- Для отладки созданного программного интерфейса, разработчику прикладного решения необходимо предоставить доступ к развернутому тестовому внешнему сервису интеграции. Тестовый внешний сервис интеграции должен предоставлять точно такие же возможности, что и реальная СШП, которая будет использоваться в производственной системе.

3. Порядок действий при внедрении:

- С помощью стандартной функции управления сервисами интеграции указать адреса используемых внешних сервисов интеграции.
- С помощью стандартной функции управления сервисами интеграции указать признаки использования сервисов интеграции (при необходимости).

17.8.4. Использование сервиса интеграции

Чтобы отправить сообщение, необходимо создать объект типа [СообщениеСервисаИнтеграции](#). Для этого предназначен метод [СоздатьСообщение\(\)](#) менеджера соответствующего сервиса интеграции. Объект типа [СообщениеСервисаИнтеграции](#) используется как при отправке, так и при получении сообщений. В связи с этим некоторые свойства данного объекта доступны только на чтение, а некоторым свойствам могут быть присвоены новые значения. Рассмотрим свойства объекта более подробно:

- [ДатаОтправки](#) - устанавливается в момент отправки сообщения. Недоступно для изменения из встроенного языка.
- [ДатаУстаревания](#) - позволяет получить момент времени, после наступления которого считается, что сообщение устарело и не должно обрабатываться. Сообщение считается устаревшим, если [ДатаУстаревания](#) сообщения меньше текущей универсальной даты компьютера, который выполняет обработку сообщений. Значение свойства можно установить только в момент создания сообщения методом [СоздатьСообщение\(\)](#), после чего изменить это значение невозможно.

Удаление устаревших сообщений выполняется из очереди во время выполнения фоновых заданий отправки и получения сообщений.

Хранится в формате UTC. При установке/получении значения выполняется преобразование с учетом часового пояса текущего сеанса.

- [Идентификатор](#) - уникальный идентификатор, который используется для однозначной идентификации сообщения. Данный идентификатор может использоваться при формировании цепочек «запрос-ответ». В этом случае идентификатор входящего сообщения следует установить в качестве значения свойства [ИдентификаторСообщенияЗапроса](#).
- [ИдентификаторСообщенияЗапроса](#) - уникальный идентификатор, который позволит принимающей стороне определить, ответом на какое сообщение выступает данное сообщение. Можно изменить только для сообщения, которое еще не отправлено в канал.
- [КодОтправителя](#) и [КодПолучателя](#) - строковые идентификаторы, которые позволяют определить отправителя и получателя сообщения. Идентификаторы соответствуют кодам из справочника участников интеграции выбранного сервиса интеграции.

При необходимости, можно указать одновременно несколько получателей сообщения. Для этого необходимо в свойство `КодПолучателя` записать строку, где коды получателей разделены символом «;». Значения свойств можно изменить только для сообщения, которое еще не отправлено в канал.

- **Параметры** - с помощью данного свойства можно установить для нового сообщения специфический набор параметров, который будет использоваться получателем для обработки сообщения. Имя параметра и его значение - значения строкового типа.
- **РазмерТела** - данное свойство показывает размер тела сообщения. До момента отправки свойство показывает количество данных, которое записано в сообщение. После отправки - сколько данных отправлено в шину.

Для того чтобы установить в сообщение какие-либо данные (или получить какие-либо данные из сообщения), предназначен метод `ПолучитьТелоКакПоток()`. В результате работы метода возвращается значение типа `Поток` (подробнее см. [здесь](#)). При отправке сообщения в этот поток можно записывать произвольные данные, которые будут переданы в сообщении, а при чтении сообщения из этого потока можно считывать данные сообщения. Записывать данные можно только в поток, который получен для созданного, но не отправленного сообщения. Отправленное и полученное сообщения можно только читать. Изменять позицию в потоке нельзя в любом случае.

После того, как сообщение сформировано, его необходимо отправить получателю. Для этого необходимо использовать метод менеджера канала сервиса интеграции `ОтправитьСообщение()`. Для вызова метода следует использовать канал того сервиса интеграции, для которого было создано сообщение. Не поддерживается возможность создать сообщение для одного сервиса интеграции, а отправить его в канал другого сервиса интеграции. Сообщение попадает в очередь сообщений, которая расположена в базе данных, с которой работает система «1С:Предприятие».

Состояние канала сервиса интеграции можно проверить с помощью метода `ПолучитьСостояние()`. Под состоянием понимается статус подключения канала к внешнему сервису интеграции.

Для работы с очередью сообщений предназначены методы менеджера сервисов интеграции `ВыполнитьОбработку()` и `ОстановитьОбработку()`. Метод `ВыполнитьОбработку()` предназначен для запуска системных фоновых заданий, которые занимаются отправкой и получением сообщений из внешних сервисов интеграции. Если необходимо прервать работу с сообщениями, то это можно сделать двумя способами: прекратить выполнение регламентного задания или вызвав метод менеджера сервисов интеграции `ОстановитьОбработку()`.

Как было сказано ранее (см. [здесь](#)), сообщение хранится в очереди, которая находится в информационной базе, до тех пор, пока:

- Сообщение не будет получено внешним сервисом интеграции (для исходящего канала).
- Сообщение не будет обработано информационной базой (для входящего канала).

Система предоставляет возможность получить и удалить сообщения, которые находятся в очереди какого-либо канала сервиса интеграции. Другими словами, для получения или удаления доступны только те сообщения, которые еще не отправлены во внешний сервис интеграции (исходящие сообщения) или не обработаны механизмами загрузки системы «1С:Предприятие» (входящие сообщения). Для получения списка сообщений, находящихся в очереди, служит метод `ВыбратьСообщения()` менеджера канала сервиса интеграции. При получении списка сообщений можно задать критерии отбора, которые ограничат список получаемых сообщений, а также общее количество получаемых сообщений. Для установки отбора необходимо создать структуру (которая будет передана в качестве параметра метода `Отбор`). Параметр `МаксимальноеКоличество` указывает, сколько всего сообщений (из тех, которые соответствуют заданному отбору) нужно получить. Если в сообщении были указаны параметры (с помощью свойства сообщения `Параметры`), то для отбора по значениям этих параметров необходимо задавать следующим образом:

[Копировать в буфер обмена](#)

```
ОтборСообщений = Новый Структура;  
ОтборПараметров = Новый Соответствие;  
ОтборПараметров.Вставить("Параметр1", "Значение1");  
ОтборПараметров.Вставить("Параметр2", "Значение2");  
ОтборСообщений.Вставить("Параметры", ОтборПараметров);  
массивРезультат = СервисыИнтеграции.ШинаДанных.СрочныеСообщения.ВыбратьСообщения(ОтборСообщений, 10);
```

В данном примере:

- `ШинаДанных` - сервис интеграции, созданный в прикладном решении.
- `СрочныеСообщения` - это канал сервиса интеграции, сообщения которого необходимо получить.

В результате примера будет получен массив сообщений, который содержит не более 10 сообщений, причем в каждом сообщении будут существовать следующие параметры:

- параметр `Параметр1` установлен в значение `Значение1`;
- параметр `Параметр2` установлен в значение `Значение2`.

Если необходимо удалить несколько сообщений, то для этого следует использовать метод `УдалитьСообщения()`. Удаление сообщения означает, что это сообщение не будет получено внешним сервисом интеграции (для исходящего канала) и не будет обработано прикладным решением (для входящего канала). Список удаляемых сообщений определяется несколькими способами, с помощью параметра метода:

- Параметр равен значению `Неопределено` - удаляются все сообщения из очереди канала.
- Массив идентификаторов сообщений - удаляются указанные сообщения. Идентификатор сообщения находится в свойстве `СообщениеСервисаИнтеграции.Идентификатор`.
- Структура, описывающая отбор сообщений. Состав отбора удаляемых сообщений аналогичен составу отбора метода

ВыбратьСообщения(). В этом случае будут удалены все сообщения в канале, которые удовлетворяют заданному отбору.

17.8.5. Настройка и активность

Каждый сервис интеграции имеет настройки, которые можно устанавливать или из встроенного языка или с помощью стандартной обработки **Управление сервисами интеграции**. Описание стандартной обработки приводится в ее справке. Данный раздел описывает изменение настроек сервиса интеграции с помощью встроенного языка.

Получение и установка настроек сервиса регистрации выполняются с помощью пары методов менеджера сервиса интеграции: **ПолучитьНастройки()** и **УстановитьНастройки()**. В обоих случаях используется объект **НастройкиСервисаИнтеграции**. Рассмотрим свойства этого объекта более подробно:

- **АдресВнешнегоСервисаИнтеграции** - позволяет указать адрес внешнего сервиса интеграции.
- **ИмяПользователяВнешнегоСервисаИнтеграции** и **ПарольПользователяВнешнегоСервисаИнтеграции** позволяют указать параметры аутентификации, используя которые платформа «1С:Предприятие» будет подключаться к внешнему сервису интеграции. Другими словами - это параметры пользователя внешнего сервиса интеграции. Задаются в том случае, если внешний сервис интеграции обладает механизмом авторизации.
- **ИмяПользователя** - позволяет указать пользователя системы «1С:Предприятие», от имени которого будет выполняться системное фоновое задание обработки полученных сообщений. Указанный пользователь будет определять права доступа. Если пользователь не указан, то используемый пользователь зависит от варианта работы системы «1С:Предприятие», а права доступа определяются свойством конфигурации **ОсновныеРоли**. Если это свойство не заполнено - контроль прав доступа не выполняется.

17.8.6. Организация работы сервисов интеграции

17.8.6.1. Получение сообщений

17.8.6.1.1. Общая информация

В общем случае, процесс получения (и обработки) сообщения из внешнего сервиса интеграции выглядит следующим образом:

- При выполнении метода менеджера сервисов интеграции **ВыполнитьОбработку()** платформа создает сетевое соединение с внешним сервисом интеграции. Данное соединение удерживается в течение 2 минут. В момент каждого вызова метода **ВыполнитьОбработку()** таймаут начинает отсчитываться от момента вызова метода. Если вызов метода не выполнялся в течение 2 минут - сетевое соединение закрывается, и взаимодействие с внешним сервисом интеграции прерывается. Данное поведение не зависит от того, в каком варианте работы информационной базы (файловом или клиент-серверном) выполняется работа с сервисами интеграции. В связи с этим, рекомендуется вызывать метод **ВыполнитьОбработку()** с помощью регламентного задания, которое должно срабатывать 1 раз в минуту.
- За обработку сообщений, поступающих от внешнего сервиса интеграции, отвечают несколько фоновых заданий. Специфика их работы зависит от используемого варианта информационной базы (файловый или клиент-серверный). Эта специфика будет рассмотрена подробнее в следующих разделах. В общем описании работы специфика файлового варианта не будет отдельно рассматриваться.
- Как только внешний сервис интеграции оповещает платформу о появлении нового сообщения, начинают свою работу три фоновых задания:
 - Фоновое задание получения сообщений выполняется как реакция на уведомление от внешнего сервиса интеграции. Фоновое задание получает сообщения, помещает его в очередь сообщений информационной базы, уведомляет внешний сервис интеграции о получении сообщения и завершает свою работу.
 - На следующем этапе запускается фоновое задание обработки очереди сообщений. Данное фоновое задание анализирует очередь сообщений и запускает фоновые задания обработки полученных сообщений. При запуске фоновым заданием передается идентификатор обрабатываемого сообщения, а также указывается, от имени какого пользователя, и в какой области данных должно работать фоновое задание. После обработки всей очереди фоновое задание завершается.
 - Последним этапом является выполнение фоновое задания обработки полученного сообщения. Это фоновое задание выполняет обработку конкретного сообщения, при этом фоновое задание выполняется от имени конкретного пользователя и в конкретной области данных. Именно из этого фоновое задание происходит вызов обработчика получения сообщения внешнего сервиса интеграции, которые размещен в модуле объекта сервиса интеграции. Вызов обработчика полученного сообщения всегда выполняется в транзакции.

Наличие трех фоновых заданий призвано решить несколько задач:

- Обеспечить подобные схемы работы в файловом и клиент-серверном вариантах работы.
- Реализовать минимальные задержки в получении сообщений от внешних сервисов интеграции.
- Обеспечить исполнение фоновое задания обработки конкретного сообщения в конкретной области данных и от имени определенного пользователя, и сделать это с минимальными накладными расходами.
- Снизить накладные расходы при получении сообщений от внешних сервисов интеграции.

17.8.6.1.2. Файловый вариант

Каждое фоновое задание обработки сообщений всегда обрабатывает одно сообщение и завершает работу. Если сообщения для обработки нет, то ожидание сообщений не выполняется. Следует помнить, что в файловом варианте фоновые задания всегда

выполняются последовательно и это выполнение обеспечивается одним экземпляром клиентского приложения. Вся последовательность действий, описанная ниже, будет выполняться для каждого (определенного в конфигурации) сервиса интеграции.

- В момент получения нового сообщения (через установленное сетевое соединение), платформа запускает фоновое задание получения сообщений. Данное фоновое задание запускается с максимальным приоритетом. Другими словами, вне зависимости от того, сколько фоновых заданий ожидают своего выполнения в момент поступления сообщения от внешнего сервиса интеграции, фоновое задание получения сообщений будет всегда запущено после завершения выполнения текущего фонового задания (если таковое есть).
- Фоновое задание получения сообщений выполняет получение одного сообщения. Сообщение записывается в очередь сообщений базы данных. Запускается (а фактически - планируется запуск) фоновое задание обработки получения сообщений. Внешний сервис интеграции уведомляется об успешном получении сообщения. Фоновое задание получения сообщений завершается.
- Фоновое задание обработки очереди сообщений получает из очереди одно сообщение. Определяются параметры обработки этого сообщения: имя пользователя и пароль, от имени которых будет обрабатываться сообщение, параметры области данных, в рамках которой будет выполняться обработка сообщения. Затем планируется запуск фонового задания обработки полученных сообщений, которому передается полученное сообщение. Фоновое задание запускается с ранее определенными параметрами. Затем фоновое задание обработки очереди сообщений анализирует, есть ли в локальной очереди необработанные сообщения или нет. Если сообщения есть - фоновое задание обработки очереди сообщений планирует свой повторный запуск. После завершения всех вышеописанных действий фоновое задание обработки очереди завершает свою работу.
- Фоновое задание обработки полученных сообщений вызывает обработчика получения сообщения внешнего сервиса интеграции, который размещен в модуле объекта сервиса интеграции. Если сообщение получено успешно, то сообщение в очереди помечается как обработанное. Если код на встроенном языке «отказался» от получения сообщения (произошла ошибка или обработчик завершен с параметром **Отказ**, установленным в значение **Истина**), то выполняется отмена транзакции. В этом случае попытка обработки сообщения будет предпринята повторно. После выполнения этих действий фоновое задание завершает свою работу.

17.8.6.1.3. Клиент-серверный вариант

При работе клиент-серверного варианта для каждого сервиса интеграции, определенного в конфигурации, запускается все три фоновых задания работы с сообщениями. Однако фоновое задание обработки полученных сообщений после завершения своей работы не завершает свою работу, а «засыпает» на время 20 секунд. Если в течение 20 секунд потребуются обработать новое сообщение от внешнего сервиса интеграции, то обработка будет выполнена достаточно быстро, т. к. кластеру серверов не потребуется тратить время на создание нового сеанса фонового задания. Если следующее сообщение поступило более чем через 20 секунд после поступления предыдущего сообщения, то фоновое задание будет запущено заново.

- В момент получения нового сообщения (через установленное сетевое соединение), платформа передает управление фоновому заданию получения сообщений. Это фоновое задание получает все сообщения по «своему» каналу, записывает их в очередь сообщений базы данных и уведомляет фоновое задание обработки полученных сообщений. Затем внешний сервис интеграции уведомляется об успешном получении сообщения.
- Фоновое задание обработки очереди сообщений получает управление после получения очередного сообщения. Для каждого из полученных сообщений определяются параметры запуска фонового задания обработки полученных сообщений. Затем фоновому заданию обработки полученных сообщений передается для обработки идентификатор сообщения, которое требуется загрузить.
- Фоновое задание обработки полученных сообщений вызывает обработчика получения сообщения внешнего сервиса интеграции, который размещен в модуле объекта сервиса интеграции. Если сообщение получено успешно, то сообщение в очереди помечается как обработанное. Если код на встроенном языке «отказался» от получения сообщения (произошла ошибка или обработчик завершен с параметром **Отказ**, установленным в значение **Истина**), то выполняется отмена транзакции. В этом случае попытка обработки сообщения будет предпринята повторно. После выполнения этих действий фоновое задание «завершает» свою работу (с учетом особенностей, описанных в начале раздела).

Таким образом, ключевое отличие между файловым и клиент-серверным вариантом заключается (фактически) в устройстве механизма работы с фоновыми заданиями. В файловом варианте в каждый момент времени работает только одно фоновое задание и фоновые задания выполняются последовательно. В клиент-серверном варианте фоновые задания могут работать параллельно.

17.8.6.2. Отправка сообщений

Перед непосредственно отправкой, сообщение записывается в очередь сообщений, которая располагается в базе данных информационной базы. Запись в очередь сообщений выполняется при вызове метода **ОтправитьСообщение()**. Если вызов метода выполняется в транзакции, то запись в очередь выполняется при успешном завершении транзакции. При вызове метода **ВыполнитьОбработку()** запускается системное фоновое задание обработки очереди сообщений, которое занимается физической отправкой сообщений внешнему сервису интеграции. Если сообщение отправлено удачно - оно удаляется из очереди сообщений. Если отправка не удалась - сообщение остается в очереди до тех пор, пока не будет отправлено или не устареет, при этом отправка остальных сообщений в канал не выполняется, пока не будет выполнена отправка первого сообщения.

Сообщения будут отправляться только в случае, если зафиксирована транзакция в информационной базе, в рамках которой созданы сообщения. Очередность отправки сообщений в канал сервиса интеграции определяется по следующим правилам:

- Сообщения отправляются по порядку фиксации транзакций в информационной базе.
- Если в рамках одной транзакции создано несколько сообщений - они отправляются в порядке создания.

Не определена очередность отправки между различными каналами и сервисами интеграции. Не определена очередность

отправки между различными областями данных одного канала.

17.8.7. Сервисы интеграции и разделение данных

Описание особенностей работы сервисов интеграции в режиме разделения данных см. [здесь](#).

17.9. Буфер обмена

17.9.1. Общая информация

Буфер обмена - это универсальный механизм операционной системы, который позволяет обмениваться данными как между различными приложениями, так и между различными элементами одного приложения. Как правило, буфер обмена предназначен для интерактивного применения пользователем, но существует большое количество сценариев, когда пользователь только инициирует использование буфера обмена, а остальные действия выполняются программным образом. Например, можно скопировать в буфер обмена несколько файлов и вставить их в какую-либо форму. В форме прикладное решение перехватит операцию вставки, получит вписок файлов из буфера обмена и сразу выполнит с этими файлами какие-либо действия.

Система «1С:Предприятие» предлагает программный интерфейс для работы с буфером обмена, который позволяет помещать данные в буфер обмена, получать данные из буфера обмена и проверять, что буфер обмена содержит данные определенного формата. Также имеется возможность перехватывать вставку некоторых типов данных из буфера обмена на уровне формы или целиком прикладного решения.

Работа с буфером обмена поддерживается только на стороне клиентского приложения.

17.9.2. Работа с буфером обмена

Система «1С:Предприятие» предоставляет доступ к буферу обмена с помощью свойства глобального контекста `СредстваБуфераОбмена`. В документации, для упрощения, будет опускаться префикс метода `СредстваБуфераОбмена`. В примерах и реальных приложениях, очевидно, такие пропуски недопустимы.

Использование буфера обмена возможно только в том случае, если этот механизм поддерживается в данном клиентском приложении. Для выполнения проверки доступности следует использовать метод `ИспользованиеДоступно()`.

В общем, всю работу с буфером обмена можно разделить на две большие группы действий: получение данных из буфера обмена и помещение данных в буфер обмена. Рассмотрим эти действия более подробно.

Для получения данных из буфера обмена предназначен метод `ПолучитьДанныеАсинх()`. В качестве параметра метода всегда необходимо указать, в каком формате мы ожидаем получить данные. В качестве значения параметра может быть или значение системного перечисления `СтандартныйФорматДанныхБуфераОбмена` или MIME-тип в виде строки. Т. к. используется асинхронный метод, то он всегда возвращает объект типа `Обещание`. Выполнение обещания приведет к тому, что из обещания будет получено значение из буфера обмена или `Неопределено`. `Неопределено` может быть получено по разным причинам: в буфере обмена нет значения требуемого формата или пользователь запретил работу с буфером обмена (в веб-клиенте). Для того, чтобы убедиться в том, что в буфере обмена есть данные в нужном нам формате, служит метод `СодержитДанныеАсинх()`. Метод возвращает значение `Истина`, если в буфере обмена есть данные нужного формата.

Исходя из вышесказанного можно сделать вывод, что методы надо применять в следующем порядке:

1. Вначале проверить, что в буфере обмена есть данные в нужном формате: `СодержитДанныеАсинх()`.
2. В случае наличия данных - получить их: `ПолучитьДанныеАсинх()`.

Система «1С:Предприятие» предоставляет возможность перехвата интерактивной вставки значения из буфера обмена для картинки и ссылки на файл. Этот перехват можно выполнить с помощью обработчика события `ПриВставкеИзБуфераОбмена`. Этот обработчик может располагаться или в модуле формы клиентского приложения или в модуле приложения. Схема использования обработчика аналогична другим подобным обработчикам: если на каком-то уровне событие полностью обработано, дальше оно не передается. Событие обрабатывается следующим образом:

- На уровне активного элемента форма: если активный элемент формы может принять из буфера обмена находящиеся там данные - обработка на этом заканчивается.
- Если для формы указан обработчик события `ПриВставкеИзБуфераОбмена` - вызывается этот обработчик. Если при выходе из обработчика формальный параметр `СтандартнаяОбработка` установлен в значение `Ложь` - обработка на этом заканчивается.
- Вызывается обработчик события `ПриВставкеИзБуфераОбмена` модуля клиентского приложения, если он указан.

Данные остальных типов передаются в активный элемент формы непосредственно.

В качестве примера рассмотрим обработчик формы клиентского приложения, который вызывается в том случае, когда пользователь нажимает сочетание клавиш `Ctrl + V` (вставить) на форме. Обработчик «предполагает», что в форму таким образом помещается одна или несколько ссылок на файлы, которые являются документами с расширением `.docx`. Эти документы необходимо разместить в базе данных. Пример не является законченным, а предназначен только для демонстрации работы обработчика и некоторых способов работы с файлами.

```
&НаСервереБезКонтекста
Процедура ПриВставкеИзБуфераОбменаНаСервере (Знач ДанныеФайлов)
    // Здесь должен находиться программный код, который
    // выполняет помещение содержимого файлов в базу данных
КонецПроцедуры
```

[Копировать в буфер обмена](#)

&НаКлиенте

Асинх Процедура ПриВставкеИзБуфераОбмена (Значение, СтандартнаяОбработка)

ДанныеФайлов = Новый Соответствие;

Если ТипЗнч(Значение) = Тип("Массив") Тогда

Для каждого ЭлементБуфера Из Значение Цикл

Если ТипЗнч(ЭлементБуфера) = Тип("СсылкаНаФайл") И НРег(ЭлементБуфера.Расширение) = ".docx" Тогда

ДанныеФайла = Ждать ЭлементБуфера.ПолучитьКакДвоичныеДанныеАсинх();

ДанныеФайлов.Вставить(ЭлементБуфера.Файл.Имя, ДанныеФайла);

КонецЕсли;

КонецЦикла;

КонецЕсли;

Если ДанныеФайлов.Количество() <> 0 Тогда

ПриВставкеИзБуфераОбменаНаСервере(ДанныеФайлов);

КонецЕсли;

КонецПроцедуры

Для помещения данных в буфер обмена следует использовать два метода ПоместитьДанныеАсинх() и ПоддерживаетсяФорматДанныхАсинх(). Для собственно размещения данных используется метод ПоместитьДанныеАсинх() и объект ЭлементБуфераОбмена. Помещать можно или один ЭлементБуфераОбмена или массив таких элементов.

При работе в ОС Linux, функционирующей на основе X11-сервера, помещение данных в буфер обмена является асинхронной операцией. В связи с этим, в клиентских приложениях возможна ситуация, когда получение содержимого буфера обмена сразу после помещения в него каких-либо данных, может вернуть данные, отличные от только-что помещенных. Принимая во внимание указанную выше асинхронность, такая ситуация считается нормальной и никак не связана с асинхронными методами самой платформы "1С:Предприятие".

Объект ЭлементБуфераОбмена содержит помещаемые данные в свойстве Данные (это может быть картинка, двоичные данные или строка) и тип помещаемых данных в свойстве ФорматДанных (системное перечисление СтандартныйФорматДанныхБуфераОбмена или MIME-тип в виде строки).

Однако, прежде чем поместить какие-то данные в буфер обмена, надо проверить - можно ли это сделать. Другими словами - требуемый вам формат поддерживается буфером обмена? Проверить это можно с помощью метода ПоддерживаетсяФорматДанныхАсинх(), передав ему в качестве параметра требуемый тип данных.

В качестве примера рассмотрим метод, который помещает в буфер обмена внешнюю навигационную ссылку текущего объекта (в форме которого размещен метод). Ссылка размещается в двух форматах: текстовом (только навигационная ссылка) и в виде HTML-фрагмента (тег <a>, где в качестве представления выступает представление помещаемой навигационной ссылки).

[Копировать в буфер обмена](#)

&НаКлиенте

Асинх Процедура ПоместитьНавигационнуюСсылкуВБуферОбмена (Команда)

Если СредстваБуфераОбмена.ИспользованиеДоступно() Тогда

ВнутренняяСсылка = ЭтотОбъект.Окно.ПолучитьНавигационнуюСсылку();

ВнешняяСсылка = ПолучитьВнешнююНавигационнуюСсылку(, ВнутренняяСсылка);

ЭлементыБуфераОбмена = Новый Массив;

Если СредстваБуфераОбмена.ПоддерживаетсяФорматДанных(СтандартныйФорматДанныхБуфераОбмена.Текст) Тогда

ДляБуфераОбмена = Новый ЭлементБуфераОбмена(СтандартныйФорматДанныхБуфераОбмена.Текст, ВнешняяСсылка);

ЭлементыБуфераОбмена.Добавить(ДляБуфераОбмена);

КонецЕсли;

Если СредстваБуфераОбмена.ПоддерживаетсяФорматДанных(СтандартныйФорматДанныхБуфераОбмена.HTML) Тогда

Ссылки = Новый Массив;

Ссылки.Добавить(ВнутренняяСсылка);

Представления = ПолучитьПредставленияНавигационныхСсылок(Ссылки);

ТекстHTML = "" + Представления[0].Представление() + "";

ДляБуфераОбмена = Новый ЭлементБуфераОбмена(СтандартныйФорматДанныхБуфераОбмена.HTML, ТекстHTML);

ЭлементыБуфераОбмена.Добавить(ДляБуфераОбмена);

КонецЕсли;

Результат = Ждать СредстваБуфераОбмена.ПоместитьДанныеАсинх(ЭлементыБуфераОбмена);

КонецЕсли;

КонецПроцедуры

Проверить работу метода можно в любом приложении, которое позволяет выполнить так называемую специальную вставку, когда приложение показывает все форматы данных в буфере обмена и предлагает выбрать нужный пользователю. Это может быть, например, редактор текстов Microsoft Word или LibreOffice Writer или какой-то другой аналогичный продукт.

СОВЕТ. Если требуется получить информацию о том, какой MIME-тип соответствует каким расширениям файлов (и, косвенно, каким данным), то можно начать получение информации по этой ссылке: <https://www.npmjs.com/package/mime-db> (на английском языке).